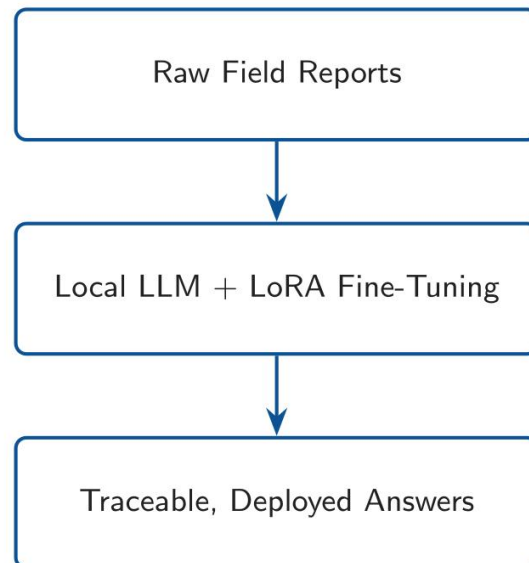


Fine-Tuning Local LLM for Drilling & Completions

A hands-on guide to fine-tuning and deploying local language models for drilling and completions engineering



Djimra Stephane Soulanoudjingar

Fine-Tuning Local LLM for Drilling & Completions

A hands-on guide to fine-tuning and deploying local language models for drilling and completions engineering

Djimra Stephane Soulanoudjingar

September 2026

Table of contents

Welcome	1
Before and after	1
What this becomes	1
Who this book is for	3
What you will build	3
Management view: why this matters	4
How this book works	5
The visual language of this book	5
Companion repository	6
Preface	7
Why this book exists	7
What this protects	7
Why “industrial”	7
How to read this book	8
A note on data	8
Acknowledgements	11
I. Part 0 — Preparing Your Local LLM Workshop	13
Part 0: Preparing Your Local LLM Workshop	15
Learning objectives	15
AI terms you’ll meet in this book	15
0.1 Why this chapter exists	16
0.2 Install Python	17
0.3 Clone this repository	18
0.4 Create and activate a virtual environment	18
0.5 Install requirements	19
0.6 Check your hardware	19
0.7 Choose your workshop (editor or notebook)	20
0.8 Run the setup check	20
0.9 Optional: install a local model runtime	21
0.10 First success checkpoint	21
0.11 Troubleshooting	21
What can you do now that you couldn’t do before?	22
Suggested next step	22

II. Part I — Foundations	23
1. Loading and Running Your First Local LLM	25
1.1. Learning objectives	25
1.2. Operational Problem	25
1.3. Example report excerpt	26
1.4. Theory	26
1.5. Why not train a model from scratch on our own reports?	27
1.6. Why not just use a cloud AI assistant?	27
1.7. Implementation	28
1.7.1. Step 1: Load the model and tokenizer	28
1.8. What problem are we solving?	28
1.9. Inputs	28
1.10. Expected Output	28
1.11. What just happened?	29
1.12. Production Reality	29
1.12.1. Step 2: Build a chat-formatted prompt and generate a reply	29
1.13. What problem are we solving?	29
1.14. Inputs	29
1.15. Expected Output	29
1.16. What just happened?	31
1.17. Practical exercise	31
1.18. Field notes	31
1.19. Challenge exercise	32
1.20. Key takeaways	32
1.21. Repository files	32
1.22. What can you do now that you couldn't do before?	33
1.23. Suggested next step	33
2. Turning Drilling & Completions Reports into Training Examples	35
2.1. Learning objectives	35
2.2. Operational Problem	35
2.3. Example report excerpt	36
2.4. Theory	36
2.5. Why not just write our own summaries?	36
2.6. Implementation	37
2.6.1. Step 1: Extract structured fields from one report	37
2.7. What problem are we solving?	37
2.8. Inputs	37
2.9. Expected Output	37
2.10. What just happened?	38
2.11. Production Reality	38
2.11.1. Step 2: Turn one report's fields into training examples	38
2.12. What problem are we solving?	38
2.13. Inputs	38
2.14. Expected Output	38
2.15. What just happened?	40
2.15.1. Step 3: Build and save the full training set — minus one report, on purpose	40

2.16. What problem are we solving?	40
2.17. Inputs	40
2.18. Expected Output	40
2.19. What just happened?	41
2.20. Practical exercise	42
2.21. Field notes	42
2.22. Challenge exercise	43
2.23. Key takeaways	43
2.24. Repository files	43
2.25. What can you do now that you couldn't do before?	44
2.26. Suggested next step	44
3. Baseline Prompting: What the Model Gets Wrong Before Fine-Tuning	45
3.1. Learning objectives	45
3.2. Operational Problem	45
3.3. Example baseline question	46
3.4. Theory	46
3.5. Why not just eyeball a few answers?	47
3.6. When fine-tuning is, and is not, the right tool	47
3.7. Implementation	48
3.7.1. Step 1: Turn Chapter 2's training examples into baseline questions	48
3.8. What problem are we solving?	48
3.9. Inputs	48
3.10. Expected Output	49
3.11. What just happened?	49
3.12. Production Reality	49
3.12.1. Step 2: Ask the base model, with the real answer hidden	50
3.13. What problem are we solving?	50
3.14. Inputs	50
3.15. Expected Output	50
3.16. What just happened?	51
3.17. Production Reality	51
3.17.1. Step 3: Run the full training baseline and save it for Chapter 5	51
3.18. What problem are we solving?	51
3.19. Inputs	51
3.20. Expected Output	51
3.21. What just happened?	52
3.22. Training recall vs. generalization	52
3.23. Practical exercise	53
3.24. Field notes	53
3.25. Challenge exercise	54
3.26. Key takeaways	54
3.27. Repository files	54
3.28. What can you do now that you couldn't do before?	55
3.29. Suggested next step	55
4. Tokenization and Embeddings for Domain Fine-Tuning	57
4.1. Learning objectives	57

Table of contents

4.2. Operational Problem	57
4.3. Example training excerpt	58
4.4. Theory	58
4.5. Why “the model has seen jargon before” isn’t evidence	59
4.6. Implementation	59
4.6.1. Step 1: See exactly how the tokenizer splits oilfield shorthand	59
4.7. What problem are we solving?	59
4.8. Inputs	59
4.9. Expected Output	59
4.10. What just happened?	60
4.11. Production Reality	61
4.11.1. Step 2: Check the base model’s own raw embeddings for these terms	61
4.12. What problem are we solving?	61
4.13. Inputs	61
4.14. Expected Output	61
4.15. What just happened?	62
4.16. Production Reality	62
4.16.1. Step 3: Check a purpose-built sentence-embedding model too	63
4.17. What problem are we solving?	63
4.18. Inputs	63
4.19. Expected Output	63
4.20. What just happened?	64
4.21. Practical exercise	64
4.22. Field notes	64
4.23. Challenge exercise	65
4.24. Key takeaways	65
4.25. Repository files	65
4.26. What can you do now that you couldn’t do before?	66
4.27. Suggested next step	66
5. Your First LoRA Fine-Tune	67
5.1. Learning objectives	67
5.2. Operational Problem	67
5.3. Example: what the model actually trains on	68
5.4. Theory	68
5.5. Why LoRA instead of updating the whole model?	70
5.6. Implementation	70
5.6.1. Step 1: Format a training example and mask the prompt out of the loss	70
5.7. What problem are we solving?	70
5.8. Inputs	70
5.9. Expected Output	71
5.10. What just happened?	71
5.11. Production Reality	72
5.11.1. Step 2: Wrap the base model with a LoRA adapter	72
5.12. What problem are we solving?	72
5.13. Inputs	72
5.14. Expected Output	72
5.15. What just happened?	73

5.16. Production Reality	73
5.16.1. Step 3: Fine-tune, then measure recall <i>and</i> generalization	73
5.17. What problem are we solving?	73
5.18. Inputs	73
5.19. Expected Output	74
5.20. What just happened?	75
5.21. Practical exercise	76
5.22. Field notes	76
5.23. Challenge exercise	77
5.24. Key takeaways	77
5.25. Repository files	77
5.26. What can you do now that you couldn't do before?	78
5.27. Suggested next step	78
 III. Part II — Industrialising the System	 79
 6. A Data Quality Gate for Training Data	 81
6.1. Learning objectives	81
6.2. Operational Problem	81
6.3. Example: the same answer, two different reports	82
6.4. Theory	82
6.5. Why “just remove duplicates” falls short	82
6.6. Implementation	83
6.6.1. Step 1: Extract fields from all 76 reports and check completeness	83
6.7. What problem are we solving?	83
6.8. Inputs	83
6.9. Expected Output	83
6.10. What just happened?	84
6.11. Production Reality	84
6.11.1. Step 2: Find field values duplicated across different reports	84
6.12. What problem are we solving?	84
6.13. Inputs	84
6.14. Expected Output	85
6.15. What just happened?	85
6.16. Production Reality	86
6.16.1. Step 3: Classify duplicates and produce the final gate report	86
6.17. What problem are we solving?	86
6.18. Inputs	86
6.19. Expected Output	86
6.20. What just happened?	87
6.21. Practical exercise	87
6.22. Field notes	87
6.23. Challenge exercise	88
6.24. Key takeaways	88
6.25. Repository files	88
6.26. What can you do now that you couldn't do before?	89
6.27. Suggested next step	89

7. Formatting and Chunking a Training Set at Scale	91
7.1. Learning objectives	91
7.2. Operational Problem	91
7.3. Example: one report, ten training examples instead of two	92
7.4. Theory	92
7.5. Why not just use the whole report as one training example?	92
7.6. Implementation	93
7.6.1. Step 1: Parse the TIME BREAKDOWN section into time-window entries	93
7.7. What problem are we solving?	93
7.8. Inputs	93
7.9. Expected Output	93
7.10. What just happened?	94
7.11. Production Reality	94
7.11.1. Step 2: Split long entries into word-boundary chunks	94
7.12. What problem are we solving?	94
7.13. Inputs	95
7.14. Expected Output	95
7.15. What just happened?	95
7.16. Production Reality	96
7.16.1. Step 3: Build the full training set, respecting Chapter 6 and Chapter 2	96
7.17. What problem are we solving?	96
7.18. Inputs	96
7.19. Expected Output	96
7.20. What just happened?	97
7.21. Practical exercise	98
7.22. Field notes	98
7.23. Challenge exercise	98
7.24. Key takeaways	99
7.25. Repository files	99
7.26. What can you do now that you couldn't do before?	100
7.27. Suggested next step	100
8. Fine-Tuning at Scale with Checkpointing and Experiment Tracking	101
8.1. Learning objectives	101
8.2. Operational Problem	101
8.3. Example: a run directory after training	102
8.4. Theory	102
8.5. Why not just rerun Chapter 5's script on more data?	102
8.6. Implementation	103
8.6.1. Step 1: Load Chapter 7's training set and hold out by report	103
8.7. What problem are we solving?	103
8.8. Inputs	103
8.9. Expected Output	103
8.10. What just happened?	104
8.11. Production Reality	104
8.11.1. Step 2: Checkpoint every epoch, log every epoch	104
8.12. What problem are we solving?	104
8.13. Inputs	104

8.14. Expected Output	104
8.15. What just happened?	106
8.16. Production Reality	106
8.16.1. Step 3: Simulate a crash, then resume from disk alone	106
8.17. What problem are we solving?	106
8.18. Inputs	107
8.19. Expected Output	107
8.20. What just happened?	107
8.21. Production Reality	108
8.22. Practical exercise	108
8.23. Field notes	109
8.24. Challenge exercise	109
8.25. Key takeaways	110
8.26. Repository files	110
8.27. What can you do now that you couldn't do before?	111
8.28. Suggested next step	111
9. Hybrid System: Combining Fine-Tuning with Retrieval	113
9.1. Learning objectives	113
9.2. Operational Problem	113
9.3. Example: the same question, with and without retrieval	114
9.4. Theory	114
9.5. Why not dense embeddings?	115
9.6. Implementation	115
9.6.1. Step 1: Build a retrieval corpus from every report	115
9.7. What problem are we solving?	115
9.8. Inputs	115
9.9. Expected Output	115
9.10. What just happened?	116
9.11. Production Reality	116
9.11.1. Step 2: Index with BM25 and retrieve	117
9.12. What problem are we solving?	117
9.13. Inputs	117
9.14. Expected Output	117
9.15. What just happened?	118
9.16. Production Reality	118
9.16.1. Step 3: Combine retrieval with the fine-tuned model	118
9.17. What problem are we solving?	118
9.18. Inputs	118
9.19. Expected Output	118
9.20. What just happened?	119
9.21. Production Reality	120
9.22. Practical exercise	120
9.23. Field notes	120
9.24. Challenge exercise	121
9.25. Key takeaways	121
9.26. Repository files	121
9.27. What can you do now that you couldn't do before?	122

9.28. Suggested next step	122
10. Traceable Outputs and Hallucination Mitigation	123
10.1. Learning objectives	123
10.2. Operational Problem	123
10.3. Example: a citation that's true, and an answer that isn't	124
10.4. Theory	124
10.5. Why trusting a citation isn't enough	125
10.6. Implementation	125
10.6.1. Step 1: Score an answer against a single source	125
10.7. What problem are we solving?	125
10.8. Inputs	125
10.9. Expected Output	125
10.10. What just happened?	126
10.11. Production Reality	127
10.11.1. Step 2: Keep only the sources an answer is actually traceable to	127
10.12. What problem are we solving?	127
10.13. Inputs	127
10.14. Expected Output	127
10.15. What just happened?	128
10.16. Production Reality	128
10.16.1. Step 3: Run the check against Chapter 9's real test cases	128
10.17. What problem are we solving?	128
10.18. Inputs	129
10.19. Expected Output	129
10.20. What just happened?	130
10.21. Practical exercise	130
10.22. Field notes	130
10.23. Challenge exercise	131
10.24. Key takeaways	131
10.25. Repository files	131
10.26. What can you do now that you couldn't do before?	132
10.27. Suggested next step	132
11. Evaluating a Fine-Tuned Domain Model	133
11.1. Learning objectives	133
11.2. Operational Problem	133
11.3. Example: one answer, asked eight different questions	134
11.4. Theory	134
11.5. Why one number isn't enough	135
11.6. Implementation	135
11.6.1. Step 1: Build a real, held-out evaluation set	135
11.7. What problem are we solving?	135
11.8. Inputs	135
11.9. Expected Output	135
11.10. What just happened?	136
11.11. Production Reality	136
11.11.1. Step 2: Score two ways — exact-match and partial credit	136

11.12	What problem are we solving?	136
11.13	Inputs	136
11.14	Expected Output	136
11.15	What just happened?	137
11.16	Production Reality	137
11.16.1	Step 3: Perplexity — a metric that doesn't need a correct answer	138
11.17	What problem are we solving?	138
11.18	Inputs	138
11.19	Expected Output	138
11.20	What just happened?	139
11.21	Practical exercise	139
11.22	Field notes	139
11.23	Challenge exercise	140
11.24	Key takeaways	140
11.25	Repository files	140
11.26	What can you do now that you couldn't do before?	141
11.27	Suggested next step	141
12.	Detecting Drift Across Model Versions	143
12.1.	Learning objectives	143
12.2.	Operational Problem	143
12.3.	Example: two checkpoints, two different verdicts	143
12.4.	Theory	144
12.5.	Why comparing one metric isn't enough	144
12.6.	Implementation	144
12.6.1.	Step 1: Summarize one model version across all three metrics	144
12.7.	What problem are we solving?	144
12.8.	Inputs	145
12.9.	Expected Output	145
12.10	What just happened?	145
12.11	Production Reality	145
12.11.1	Step 2: Compare two version summaries	145
12.12	What problem are we solving?	145
12.13	Inputs	145
12.14	Expected Output	146
12.15	What just happened?	146
12.16	Production Reality	146
12.16.1	Step 3: Run it across all 3 real checkpoints	146
12.17	What problem are we solving?	146
12.18	Inputs	147
12.19	Expected Output	147
12.20	What just happened?	147
12.21	Practical exercise	147
12.22	Field notes	148
12.23	Challenge exercise	148
12.24	Key takeaways	148
12.25	Repository files	149
12.26	What can you do now that you couldn't do before?	149

12.27Suggested next step	149
13. Continuous Fine-Tuning: Keeping the Model Current	151
13.1. Learning objectives	151
13.2. Operational Problem	151
13.3. Example: the same question this book has asked all along, one more time	152
13.4. Theory	152
13.5. Why “we retrained” isn’t the same claim as “it’s better”	152
13.6. Implementation	153
13.6.1. Step 1: Split the archive by a chronological cutoff	153
13.7. What problem are we solving?	153
13.8. Inputs	153
13.9. Expected Output	153
13.10What just happened?	154
13.11Production Reality	154
13.11.1Step 2: Train the current model, then continue on the new batch	154
13.12What problem are we solving?	154
13.13Inputs	154
13.14Expected Output	154
13.15What just happened?	155
13.16Production Reality	155
13.16.1Step 3: Compare before deciding to deploy	155
13.17What problem are we solving?	155
13.18Inputs	155
13.19Expected Output	156
13.20What just happened?	156
13.21Practical exercise	156
13.22Field notes	157
13.23Challenge exercise	157
13.24Key takeaways	158
13.25Repository files	158
13.26What can you do now that you couldn’t do before?	159
13.27See it side by side	159
13.28Suggested next step	159
IV. Appendices	161
Appendix A: Environment Setup	163
1. The training dataset	163
2. Rendering the book with Quarto	163
3. Hardware notes	164
4. A note on data handling	164
5. Troubleshooting	164
Appendix A1: Using Jupyter Notebook	167
Install the notebook	167
Start the notebook	167

Open a chapter notebook	167
Run a cell	168
Stop the notebook server	168
Appendix A2: Using VS Code	169
Install VS Code and the Python extension	169
Open the project folder	169
Select the virtual environment as your interpreter	169
Run a chapter script	169
Run the tests	169
Troubleshooting	170
Appendix A3: Using PyCharm Community	171
Install PyCharm Community	171
Open the project folder	171
Point PyCharm at your virtual environment	171
Run a chapter script	171
Run the tests	171
Troubleshooting	172
Appendix A4: Using Positron	173
Install Positron	173
Open the project folder	173
Select the Python interpreter	173
Run a chapter script	173
Preview a chapter while writing	173
Troubleshooting	174
Appendix A5: Terminal Only	175
Activate your environment	175
Edit a file	175
Run a chapter script	175
Run the tests	175
Render the book (optional)	176
Troubleshooting	176
Appendix B: Drilling, Completions & Fine-Tuning Glossary	177
B.1 Common oilfield abbreviations	177
B.2 Fine-tuning and local LLM vocabulary	178
Appendix C: Known Limitations	181
C.1 Report parsing breaks on a new report layout (Chapters 2, 6, and 7)	181
C.2 The faithfulness check compares words, not meaning (Chapter 10)	182
C.3 CPU training times don't demonstrate production scale (Chapters 8 and 13)	183
C.4 Resuming a checkpoint forgets the optimizer's memory (Chapters 8 and 13)	183
C.5 Everything in this book is tested on one small model (Chapters 1-13)	184
C.6 The tokenizer's fixed vocabulary splits oilfield shorthand apart (Chapter 4)	184
References	185

Welcome

Every operator has its own language on the rig. Morning report shorthand, internal asset codes, and practical field judgment calls rarely show up in public data. When you ask a general-purpose AI a complex drilling or completions question, the result is often polished and plausible, yet subtly wrong: the right textbook terms, but the wrong wellsite logic.

Fine-tuning fixes this, but it cannot be treated as a black box. Success requires high-quality operational data, the right training strategy, and clear validation metrics to prove the model genuinely improved rather than just shifted its phrasing.

This book builds, from scratch and in plain Python, a fine-tuned local language model purpose-built for drilling and completions engineering — one you build chapter by chapter and understand end to end. You will not start with theory. You will start by loading a small local model and watching it get something in your domain wrong. By the end, you will have a model that has genuinely learned from your own operational text, runs entirely on your own machine, and comes with an evaluation harness that tells you, with numbers, whether it actually got better.

Before and after

Today, a general local model answers a drilling or completions question the way it would answer any question: plausibly, generically, and without any real grounding in how your operation actually talks about pressure tests, completions sequencing, or a stuck-pipe event. Getting it to behave like it works *here* usually means one of two things: an enormous prompt stuffed with context every single time, or a model that has actually been trained on your language.

By Chapter 5, you'll have run your first parameter-efficient fine-tune — a small, targeted update to a local model's weights — and be able to compare its answers to the un-fine-tuned baseline side by side. By the end of the book, that model is evaluated, checked for hallucination and drift, and wired into a pipeline that keeps it current as new reports arrive.

What this becomes

The scripts in this book are the engine. Its optional companion app (`book/app/`) puts the same evidence on one screen, reading whatever real checkpoint you've actually trained: a Model Playground for the same base-vs-fine-tuned comparison side by side, a Before vs. After Evaluation page that runs this book's own metrics live across the full held-out set, a Dataset Explorer for the real training examples behind the result, and a Failure Analysis page that reruns the book's own documented failure cases — including a fluent, confident wrong answer — against your checkpoint. Nothing in it is pre-recorded; see `app/README.md` once you're ready to run it, after Chapter 5 gives you a first real checkpoint to load.

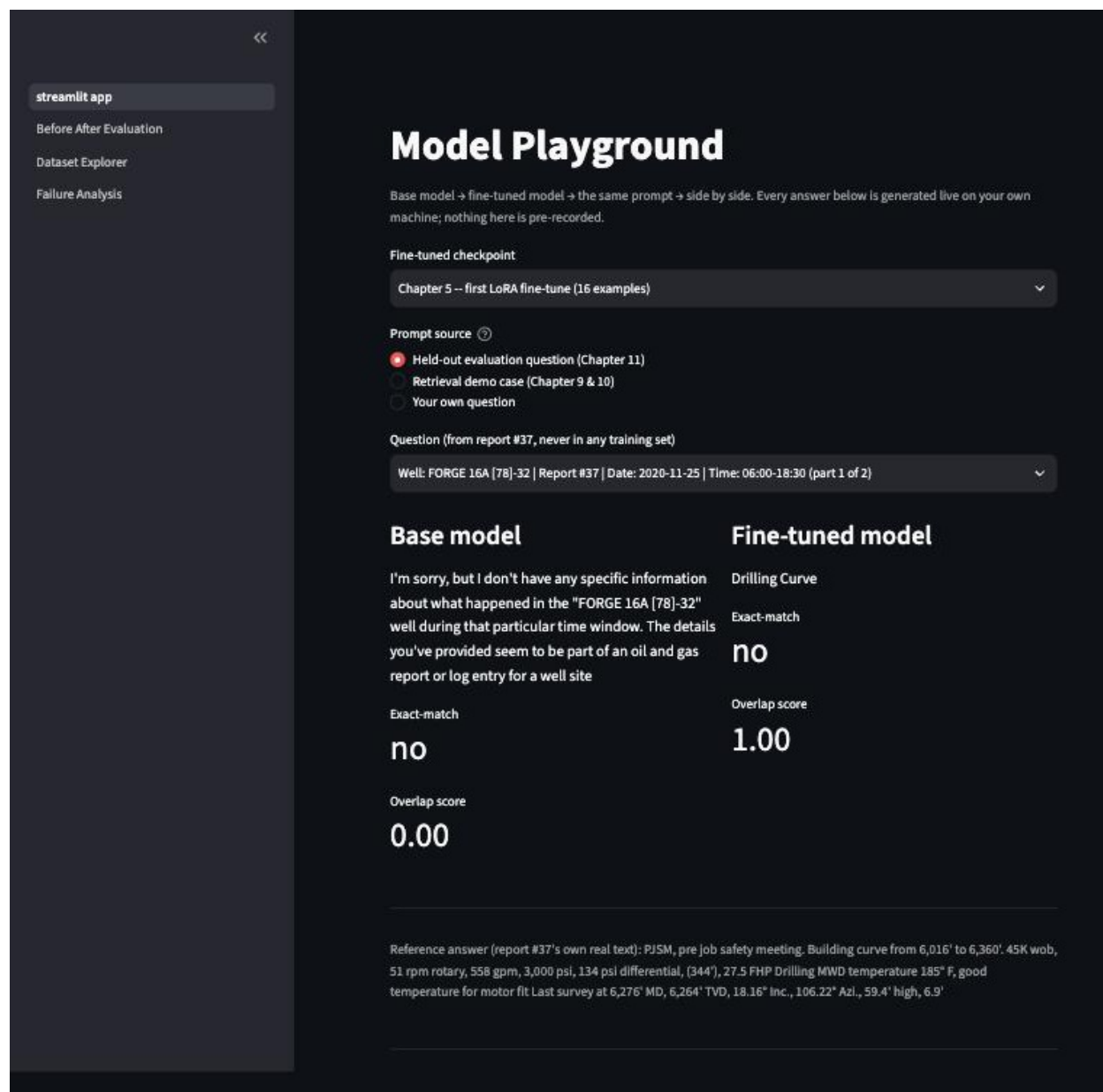


Figure 1.: Model Playground, screenshotted from a real local run: report #37's held-out question, answered live by both the base model and Chapter 5's first fine-tuned checkpoint, with the real exact-match and overlap scores for each.

Who this book is for

You should read this book if you are a drilling engineer, completions engineer, intervention engineer, production engineer, petroleum data scientist, or part of a digital transformation team in energy, and you:

- Have never written a line of Python in your life — or tried once and gave up.
- Have strong operational experience and are comfortable in Excel.
- Have never fine-tuned a model or worked with a local LLM runtime.
- Are curious about AI and automation, and are willing to type code examples yourself rather than just read about them.
- Want a model that works on your own operational language, not a generic assistant.

No prior programming or AI/ML background is assumed. Every concept — even one as basic as what a function is — is introduced only after you’ve hit the problem it solves, never before. The goal of this book isn’t to turn you into a machine learning researcher. It’s to make you capable of building and trusting a private, local, domain-specific model.

One expectation worth setting early: this book’s own code is written and tested against one real report format (Utah FORGE’s Daily Drilling Report). There is no single format across operators, contractors, and basins, so the goal here isn’t to hand you a tool that already expects your archive’s layout — it’s to teach you the pipeline stage by stage, on real data, well enough that adapting each stage to your own reports is a rewrite you understand, not a mystery you’re stuck on. See [Appendix C](#) once you’ve finished the book for exactly where that adaptation work starts.

What you will build

Chapter	You will be able to...
0	Set up a working local LLM environment, checking your hardware along the way
1	Load and run a local base model
2	Turn raw drilling and completions reports into training examples
3	Show, with evidence, where the base model gets domain questions wrong
4	Understand tokenization and embeddings well enough to prepare training data correctly
5	Run your first LoRA fine-tune and compare it to the baseline
6–13	Scale the system to larger, cleaner datasets, combine fine-tuning with retrieval, evaluate rigorously, detect drift, and keep the model current as new reports arrive

One picture for the whole route above, distinct from each chapter’s own two-box opening diagram: where you start, all 13 chapters grouped into Part I and Part II, and the one honest checkpoint in the middle that keeps this map from overselling itself.

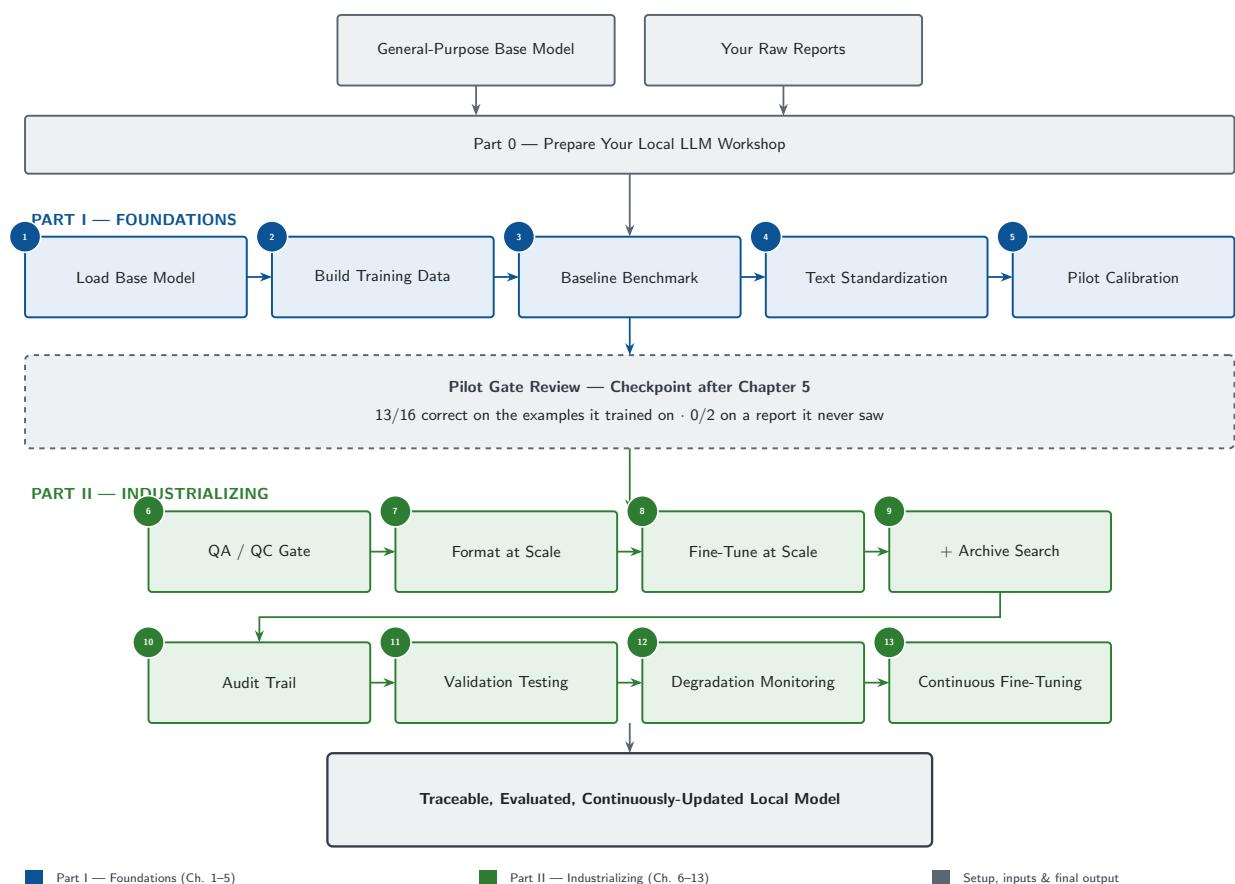


Figure 2.: The reader’s route through the book: Part I’s five foundation chapters reach a real, honest checkpoint (13/16 on trained examples, 0/2 on a held-out report) before Part II’s eight chapters industrialize the system.

Management view: why this matters

If you’re deciding whether to fund or adopt work like this, here’s the case in plain terms — and its limits.

- **Domain fluency.** A fine-tuned model responds in the vocabulary and operational logic your team actually uses, instead of generic boilerplate.
- **Private deployment.** Everything in this book runs locally, so confidential reports and the model trained on them never need to leave your environment.
- **Repeatability.** The training pipeline is code, not a one-off manual effort — it can be re-run as new data arrives (Chapter 13).
- **Measurable improvement.** Chapter 11’s evaluation harness gives you numbers, not impressions, for whether fine-tuning actually helped.

- **Build vs. buy.** The core is a few hundred lines of standard Python plus open-weight models. The trade-off against a vendor product is control and inspectability versus vendor support and maintenance.
- **Known limits.** Fine-tuning teaches style and domain patterns; it is not a substitute for retrieval when the model needs to cite a specific, current document verbatim — Chapter 9 covers combining the two. See [Appendix C](#) for six concrete gaps — from brittle report parsing to a faithfulness check that misses a reversed operation — worth checking before you point this pipeline at your own archive. That faithfulness check in particular is a teaching tool, not a validated safety system — never use it, unsupervised, as the check on whether a live-well handover summary is safe to act on.

This supports an engineer’s judgment; it doesn’t replace it. A more fluent model is only as good as the review process around how its answers get used.

How this book works

Every chapter solves one problem you’d actually hit fine-tuning a model for this domain, and ends with something you can run. There is no chapter you read passively — you will type code, run it against real training data, and see the model change. Theory appears only when it explains *why* the code in front of you behaves the way it does.

We follow one rule throughout: **measured improvement over AI novelty**. A fine-tune that isn’t evaluated is just a guess dressed up as progress — so this book insists on comparing before and after, every time.

The visual language of this book

A few recurring elements appear in every chapter, explained once here so they never need repeating:

- **Progress bar** — `###..... 3 / 13` at the top of every chapter, showing exactly how far you are through the book.
- **Estimated time and difficulty badge** (Beginner, Intermediate, Advanced) — next to the progress bar, so you know the commitment before you start.
- **CHECKPOINT** — a checklist near the end of each chapter, confirming the specific skills you just gained.
- **WHAT YOU BUILT** — a green box naming the concrete artifact (a script, a fine-tuned checkpoint, an evaluation report) you now have, distinct from the conceptual “Key takeaways” earlier in the chapter.
- **Field notes** — a real, independently-verified vignette showing where a chapter’s tool succeeds or breaks against real training data.

You’ll also meet four recurring engineers, each asking the question that opens a chapter’s **Operational Problem**:

Name	Role
Oumy	Drilling Engineer
Carlos	Completions Engineer

Welcome

Name	Role
Sarah	Intervention Engineer
Farid	Production Engineer

They're a framing device, not a source of data: whichever question Oumy, Carlos, Sarah, or Farid asks, every number and result used to answer it must come from a real, reproducible run of this book's own code — never invented on their behalf.

Companion repository

All code, training data, and notebooks referenced in this book live in the [companion repository](#) on GitHub. Every chapter lists exactly which files you need under **Repository files**.

Stuck on something — a setup error, a package that won't install, a script that fails partway through? The [Troubleshooting Guide](#) in that same repository collects the errors readers hit most often, with the likely cause for each.

If you've never installed Python or don't know which editor to use, start with **Part 0 — Preparing Your Local LLM Workshop**: it works with any of five common setups (Jupyter Notebook, VS Code, PyCharm Community, Positron, or a terminal alone) and ends with your first script actually running. If you already have Python and a working setup, jump straight to Chapter 1.

Let's load our first local model.

Preface

Why this book exists

I wrote this book because I kept seeing the same gap on drilling and completions teams considering local LLMs: a general-purpose model is fluent, but it doesn't actually know how *this* operation talks — report shorthand, sequencing conventions, the judgment calls a completions engineer makes without writing out the reasoning. You can paper over that with an enormous prompt every time, or you can fine-tune a small local model on your own operational text so it has actually learned the domain. This book teaches the second path, end to end, in plain Python.

Parameter-efficient fine-tuning methods like LoRA (Hu et al. 2021) and its quantized successor QLoRA (Dettmers et al. 2023) made this practical on ordinary hardware — a full retrain of a foundation model is out of reach for almost everyone; adapting a few percent of its parameters on a consumer GPU is not. This book builds on that shift: you fine-tune a small, already instruction-tuned open-weight model (Wei et al. 2021), evaluate it honestly, and keep it current, all running locally.

What this protects

Fine-tuning on operational text raises the same stakes as any system trained on a company's own data: what goes into the model can, in principle, come back out of it. This book treats that as a first-class constraint, not an afterthought — training data handling, what stays local, and what an evaluation harness should catch before a fine-tuned model is trusted are covered starting in Part I, not bolted on at the end.

A fine-tuned model can also sound confident while being wrong — the well-documented failure mode usually called hallucination (Ji et al. 2023) — which is a separate risk from data leakage and gets its own detection tooling starting in Chapter 10.

A fine-tuned model doesn't replace an engineer's judgment. It replaces having to explain your operation's conventions to a generic assistant every single time. Deciding what a result means, or whether a model's answer is safe to act on, stays yours to make.

Why “industrial”

A demo that fine-tunes a model on a handful of hand-picked examples is easy to build and easy to find online. An industrial system is different. It has to:

- Handle real, messy training data — inconsistent formatting, duplicate or near-duplicate reports, missing fields.
- Prove the fine-tune actually improved something, with numbers, not impressions.
- Never silently regress on cases the base model already got right.
- Be auditable and reproducible by someone who was not in the room when it was trained.

Every chapter in Part II exists because a toy fine-tuning script fails at one of these requirements. We fix them one at a time, in the order you would actually hit them in the field.

How to read this book

This book borrows the practical-first, project-based spirit of *Python Crash Course* (Matthes 2023) — small wins early, theory only when the code in front of you needs it, a working artifact at the end of every chapter — but assumes considerably less going in. You do not need to have written Python before. You do not need to know what a function or a loop is before Chapter 1; each concept is explained in plain language, translated into terms an engineer already knows, exactly when you need it to solve the problem in front of you.

If you’ve never installed Python or set up a project folder before, start with Part 0 — it gets you to a working setup in whichever editor or notebook tool you’re most comfortable with, with no assumption that you’ve done this before.

Read it in order the first time. Each chapter assumes the code from the previous one exists and works — this is a build, not a reference manual. After your first pass, use it as a reference: jump to the chapter that matches the problem you currently have (Chapter 6 if your training data is messy, Chapter 11 if you’re not sure the fine-tune helped, Chapter 12 if you suspect a newer version regressed).

Type the code yourself. Do not copy-paste from the companion repository until you have typed it once and watched it run. This is slower and it is worth it — and if you finish this book able to say “I can build this,” rather than “I understand the theory,” it did its job.

A note on data

Daily drilling and completions reports are typically operationally sensitive and confidential — which is exactly why this book reuses the same source as the author’s previous book on retrieval: real, publicly available Daily Drilling and completion reports from **Utah FORGE**, a Department of Energy-funded enhanced geothermal system research well (FORGE 16A [78]-32). No anonymisation was applied, because none was needed. See [book/datasets/README.md](#) for the exact layout and policy.

That said, the *technique* in this book generalises directly to a confidential archive: everything from Chapter 6 onward — data quality gates, evaluation, drift detection — exists precisely because production training data is messier and more sensitive than this public one. If you point this book’s code at your own organisation’s reports instead, do not commit them to a public repository, and see [Appendix A](#) for a data-handling checklist.

One thing that does *not* generalise automatically: Chapter 2’s and Chapter 7’s extraction code reads specific field labels at specific positions in Utah FORGE’s own report layout, and Chapter 6’s data quality gate runs that same extraction code unchanged over the full archive. There is no single “Daily Drilling Report” format across operators, contractors, and basins, so that extraction code will need rewriting — new labels, new patterns — against your own archive’s actual layout before anything downstream of it works. Doing that rewrite means editing regular-expression patterns directly, which is a real programming task even though the rest of this book doesn’t assume you can do one yet. That adaptation is not a gap to route around; it is the point. This book teaches you the full pipeline end to end, on one real dataset, so you have the understanding to rebuild each stage against your own data — not a drop-in tool that already expects it. See [Appendix C](#) for this and five other concrete places this book’s code is scoped to what it was actually tested against.

Acknowledgements

This book exists because drilling and completions reports capture far more than operational summaries. Behind short, genuine entries such as “TOOH with BHA #2. Break off bit, lay down Scout RSS” (from this book’s own Utah FORGE archive) sit decisions, experience, and operational context that are obvious to the engineers involved in the operation but invisible to standard AI systems that have never seen field data.

This work depends on the global open-source community, whose open models, software, and shared research make it possible to run powerful AI tools locally and securely on a standard work laptop, without sending proprietary data to the cloud.

Finally, thanks to every drilling, completions, and operations engineer working through this material. You do not need to be a software developer or data scientist to apply these methods. If you find an error, test a method on your operational data that yields unexpected results, or find a section that assumes too much technical background, please reach out via the book’s companion website. Feedback from practitioners in the field is what keeps a guide like this practical and honest.

The engineering direction, domain interpretation, educational approach, and final technical decisions are the author’s. AI tools, including Claude, were used throughout the project to assist with drafting, editing, documentation, and technical verification. All AI-assisted work was reviewed and validated against actual operational datasets and repeatable workflows. Responsibility for the final content remains with the author.

Part I.

Part 0 — Preparing Your Local LLM Workshop

Part 0: Preparing Your Local LLM Workshop

Estimated time: 30–45 min · **Difficulty:** Beginner · No prior programming experience assumed

Learning objectives

By the end of this chapter, you will be able to:

- Install Python and confirm it works
- Get a copy of this book’s companion repository onto your own machine
- Create and activate a virtual environment for this book
- Install this book’s dependencies
- Check whether your machine has a GPU fine-tuning can use, and know what to expect if it doesn’t
- Run `setup_check.py` and get a clean bill of health before Chapter 1

AI terms you’ll meet in this book

New vocabulary shows up gradually, one or two terms per chapter, each explained with an “Engineering Translation” callout the first time it matters. This page isn’t a substitute for those — it’s a quick-reference sheet to glance back at when a term from an earlier chapter resurfaces and you just want the short version, not the full explanation again. (Appendix B has the complete glossary, oilfield abbreviations included, if you need more than this.)

Term	What it means here
Model	The software that generates an answer from a question.
Base model	The general-purpose model before any domain fine-tuning — Chapter 1’s starting point.
Fine-tuning	Updating a model’s weights on your own examples so it performs better on your specific task.
Weights	The model’s internal numbers that determine its behavior — what fine-tuning actually changes.
Token	One piece of text after the model’s tokenizer cuts it up — sometimes a whole word, sometimes a fragment — Chapter 4.

Term	What it means here
Embedding	A list of numbers standing in for a token or phrase, so the model can do math with meaning instead of letters — Chapter 4.
LoRA adapter	A small, separate set of numbers trained instead of the full model, added on top of the frozen base weights — Chapter 5.
Rank	How narrow LoRA's two small adapter tables are — smaller is cheaper to train, at the cost of a rougher approximation — Chapter 5.
Training example	One input and the answer you want the model to learn from — Chapter 2.
Epoch	One full pass through the training data.
Loss	A number measuring how wrong the model's predictions were on one pass — should fall as training continues.
Checkpoint	A saved snapshot of the model's weights at one point in training — something to resume from if a run is interrupted — Chapter 8.
Held-out data	Reports deliberately kept out of training so you have something honest to test on afterward — Chapter 2.
Inference	Asking the trained model a question and getting an answer back, as opposed to training it.
Quantization	Storing a model's weights at lower numerical precision to shrink memory and speed up inference, at some accuracy cost.
Perplexity	A numeric measure of how well a model predicts held-out text; lower is generally better — Chapter 11.

0.1 Why this chapter exists

Every chapter from here on assumes four things are already true: Python is installed, this repository is on your machine, a virtual environment for it is active, and this book's packages are installed inside that environment. None of that is difficult, but skipping or rushing it is the single most common reason a reader gets stuck before Chapter 1 even starts — usually with a confusing error that has nothing to do with drilling, completions, or language models, and everything to do with one of these four things not being true yet.

This chapter does all four, one step at a time, with no assumption that you've done any of it before.

💡 Already comfortable with Python and virtual environments?

Skip the explanations in **0.2** (installing Python), **0.6** (hardware check), and **0.7** (choosing an editor) — you already know that part. You still need to actually do **0.3** (clone this repository), **0.4** (create this book’s own virtual environment), **0.5** (install its requirements), and **0.8** (run the setup check) like everyone else — they’re specific to this book, not to Python in general — but you can move through them fast. If 0.8 comes back clean, you’re done with this chapter.

0.2 Install Python

This book targets **Python 3.11 or later** — the same version pinned in `requirements.txt` and `environment.yml`.

💡 Engineering Translation: what is Python, here?

Python is the programming language every script in this book is written in. Installing it is like installing the one piece of shop equipment every other tool in this book plugs into — nothing past this section runs without it.

macOS: Python 3 usually needs to be installed explicitly (the system’s built-in `python3` is often older or missing entirely). Download the latest 3.11+ installer from python.org/downloads and run it, or if you have [Homebrew](https://brew.sh/) installed:

```
brew install python@3.11
```

Windows: download the installer from python.org/downloads. On the first installer screen, check “Add `python.exe` to `PATH`” before clicking Install — this is the step almost everyone forgets, and it’s what Section 0.11 below assumes you didn’t.

Linux: most distributions ship Python 3, but check the version — it may be older than 3.11. On Ubuntu/Debian:

```
sudo apt update && sudo apt install python3 python3-venv python3-pip
```

Confirm it worked:

```
python3 --version
```

Expected output: something like `Python 3.11.9` (any 3.11 or later patch version is fine). If you see `command not found` or a version below 3.11, revisit the install step for your platform above.

0.3 Clone this repository

If you're comfortable with `git`:

```
git clone https://github.com/djimrastephane/llm-ft-drilling-completions-book.git
cd llm-ft-drilling-completions-book
```

Never used `git` before? Click the **Download ZIP** button on the repository's GitHub page instead, unzip it, and open a terminal inside the unzipped folder.

Expected output: a folder named `llm-ft-drilling-completions-book` (or similar) containing, among other things, a `book/` folder — everything from here on happens inside `book/`.

```
cd book
```

0.4 Create and activate a virtual environment

💡 Engineering Translation: what is a virtual environment?

A virtual environment is a private, self-contained set of Python packages just for this project — like a dedicated toolbox for this book that doesn't get mixed up with tools installed for anything else on your machine. Without it, this book's packages (and their exact versions) could conflict with something else Python-related already on your system.

From inside `book/`:

```
python3 -m venv .venv
```

What just happened? This created a `.venv` folder holding a private copy of Python and an empty toolbox, specific to this project.

Now activate it — you'll need to do this every time you open a new terminal window to work on this book:

```
source .venv/bin/activate      # macOS/Linux
.venv\Scripts\activate        # Windows (Command Prompt or PowerShell)
```

Expected result: your terminal prompt now starts with `(.venv)`. That prefix is your confirmation the virtual environment is active — if it's missing, the next step will install packages into the wrong place.

0.5 Install requirements

With `(.venv)` showing in your prompt and still inside `book/`:

```
pip install -r requirements.txt
```

What just happened? `pip` read the list of packages this book needs — `transformers`, `torch`, `peft`, and the rest — and installed them into your `.venv`. `torch` in particular is a large download (several hundred MB to a few GB depending on platform and whether a GPU build is selected), so this step can take a few minutes even on a fast connection. That's normal.

Expected output: a long scroll of `Downloading...` and `Installing collected packages...` lines, ending with something like `Successfully installed torch-... transformers-... peft-... ..` and no red `ERROR` lines.

i bitsandbytes and your platform

`requirements.txt` installs `bitsandbytes` (used for 4-/8-bit QLoRA fine-tuning) only on Linux — it has no working build for macOS. If you're on macOS or a CPU-only machine, this is expected and not an error — and it doesn't matter for this book either way: no chapter script here actually loads a quantized model. Chapter 5 explains the full-precision LoRA path every chapter uses instead.

0.6 Check your hardware

Fine-tuning, even the parameter-efficient kind this book uses, is more sensitive to your hardware than simply running a model is. This section doesn't require you to buy anything — it just sets expectations for later chapters.

```
import torch

if torch.cuda.is_available():
    print(f"CUDA GPU available: {torch.cuda.get_device_name(0)}")
elif torch.backends.mps.is_available():
    print("Apple Silicon MPS device available")
else:
    print("No GPU detected -- CPU-only.")
```

What just happened? This checks for three possibilities: an NVIDIA GPU (`cuda`), an Apple Silicon Mac's built-in accelerator (`mps`), or neither.

What this means for you:

- **CUDA GPU:** the most comfortable path. Chapter 5's fine-tune, and the heavier Part II chapters, will run in minutes.

- **Apple Silicon MPS:** works well for this book’s small models, though somewhat slower than a dedicated CUDA GPU.
- **CPU-only:** every chapter is still designed to run — you’ll use a smaller model and fewer training steps where a chapter calls for it, at the cost of longer wait times, not a dead end.

0.7 Choose your workshop (editor or notebook)

You don’t need a specific editor to follow this book. Pick whichever of these you’re most comfortable with — there is no wrong choice, and all five run the exact same code:

Environment	Recommended for	Guide
Jupyter Notebook	Learning and experimentation	Appendix A1
VS Code	General coding	Appendix A2
PyCharm Community	Larger projects	Appendix A3
Positron	Data science workflows	Appendix A4
Terminal only	Minimal setup	Appendix A5

If you’re not sure, Jupyter Notebook or VS Code are the most common starting points. You can always switch later — nothing in this book locks you into one choice.

0.8 Run the setup check

With `(.venv)` active and requirements installed, from inside `book/`:

```
python code/setup_check.py
```

Expected output:

```
Fine-Tuning Local LLM for Drilling & Completions -- setup check
```

```
[OK] Python 3.11.9
[OK] torch importable
[OK] transformers importable
[OK] peft importable
[OK] datasets importable
[OK] accelerate importable
[INFO] No GPU detected -- CPU-only. Fine-tuning chapters will still work with a small model, j
```

```
Workshop is ready.
```

(Your `[INFO]` line will instead name a CUDA GPU or MPS if you have one — see Section 0.6.)

If you instead see any `[FAIL]` line, do not continue to Chapter 1 yet — see Section 0.11, Troubleshooting, below.

0.9 Optional: install a local model runtime

Chapters 1–13 use `transformers` and `peft` directly — nothing here requires any additional software. If you’d also like a familiar, general-purpose way to run local models outside this book’s own code (useful later for quick comparisons), [Ollama](#) is a convenient optional install. Pull the same size model this book uses throughout (`Qwen2.5-1.5B-Instruct`, from Chapter 1 on) so any comparison you make is apples-to-apples, not a bigger model quietly doing better:

```
# after installing from ollama.com
ollama pull qwen2.5:1.5b-instruct
ollama serve
```

This step is entirely optional and can be skipped for now.

0.10 First success checkpoint

You are ready for Chapter 1 when:

CHECKPOINT — Part 0

- ☒ `python3 --version` reports 3.11 or later
- ☒ This repository is on your machine, with a `book/` folder
- ☒ Your terminal prompt shows `(.venv)`
- ☒ `pip install -r requirements.txt` finished with no ERROR lines
- ☒ `python code/setup_check.py` printed `Workshop is ready.`

If every box above is true, you’re done with Part 0.

0.11 Troubleshooting

Symptom	Likely cause
command not found: <code>python3</code> (Windows)	Python wasn’t added to <code>PATH</code> during install — re-run the installer and check “Add python.exe to <code>PATH</code> ” (Section 0.2).
<code>python3 --version</code> shows a version below 3.11	You have an older Python 3 install alongside a newer one; try <code>python3.11 --version</code> explicitly, or reinstall from python.org .
Prompt never shows <code>(.venv)</code> after activating	Confirm you ran the activate command from <i>inside</i> <code>book/</code> , and that <code>.venv/</code> actually exists there (Section 0.4).

Symptom	Likely cause
<code>pip install -r requirements.txt</code> fails partway through	Re-run the same command — a partial download is the most common cause and simply retrying often finishes the job. If it fails on <code>torch</code> specifically, try <code>pip install torch</code> alone first, then re-run <code>pip install -r requirements.txt</code> .
<code>setup_check.py</code> reports [FAIL] <code><package> not importable</code>	<code>(.venv)</code> isn't active, or <code>pip install -r requirements.txt</code> didn't finish successfully — re-check Sections 0.4 and 0.5.
<code>ModuleNotFoundError: No module named 'torch.backends'</code> when checking hardware	Your <code>torch</code> install didn't complete — re-run <code>pip install -r requirements.txt</code> and check for <code>ERROR</code> lines.

What can you do now that you couldn't do before?

You have a working Python environment, this book's dependencies installed, and a hardware check that tells you what to expect from later fine-tuning chapters — everything Chapter 1 assumes is already in place.

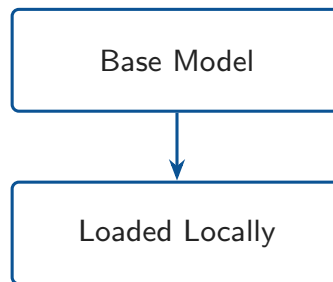
Suggested next step

Coming up in Chapter 1: with your workshop ready, Chapter 1 loads a general-purpose local model for the first time and generates its first completion — the base model you'll spend the rest of Part I fine-tuning into something that actually understands drilling and completions reports.

Part II.

Part I — Foundations

1. Loading and Running Your First Local LLM



Progress #..... 1 / 13 · Estimated time: 20–30 min · Difficulty: Beginner

1.1. Learning objectives

By the end of this chapter, you will be able to:

- Load an open-weight, instruction-tuned language model with **transformers**, entirely on your own machine
- Send it a question and read back its answer
- Explain, in plain language, why a fluent answer isn't the same as a correct one — and see it happen on your own screen

1.2. Operational Problem

Oumy, the drilling engineer, has heard her company might start using AI models to help make sense of years of drilling and completions reports. Her first question isn't about accuracy — it's about control: *“Can we run one of these models ourselves, on our own machine, so nothing about this well ever leaves the building?”*

That's this chapter's problem to solve, and only this one. Not yet whether the model understands drilling operations correctly — that's coming. Just: can you load a real, working language model on your own hardware and get an answer out of it, with nothing sent anywhere?

1.3. Example report excerpt

Later chapters turn real reports like this one into training data. Here is a genuine excerpt — quoted exactly as filed, typos and all — from this book’s own sample archive, report `Drilling_038` (2020-11-26), the real stuck-pipe day referenced throughout this book:

 Sample report excerpt – `FORGE-16A-78-32_Drilling_038_2020-11-26.pdf`

```
23:30  04:00  4.5  Drill From 6,360' to 6,507', (147') Total, 32.6' feet per hour.
                                WOB 20 TO 35k, Rotary 50, Torque 6,500, SPP 3200-
3400 GPM 560, DIFF 200-300psi
                                During the slide lost tool face and became assembly became stuck
04:00  06:00  2.0  Work pipe, circulate lube sweep, work tool back in position
                                Pipe free
```

Hold that idea — the driller lost tool face and the assembly became stuck — in mind. Later in this chapter, you’ll ask the base model about it directly, and see exactly how much (or how little) it understands.

1.4. Theory

 Engineering Translation: base model

A **base model** is the general-purpose language model before any fine-tuning on your own data — the raw, off-the-shelf unit, not yet adjusted for this job. Every later chapter compares back to this starting point.

 Engineering Translation: weights / checkpoint

A model’s **weights** (also called a **checkpoint**) are the millions of learned numbers that determine what it says — the actual “knowledge,” stored as files on disk. “Loading a model” means reading those files into memory so they can be used.

 Engineering Translation: chat template

Instruction-tuned models are trained on conversations formatted a specific way (who’s speaking, where a turn starts and ends). A **chat template** is the exact formatting recipe for a given model — get it wrong and the model still runs, just worse, because the input doesn’t look like anything it was trained on. **transformers** applies the right one for you automatically once you tell it which model you’re using.

This book’s chapters all fine-tune the same base model: **Qwen2.5-1.5B-Instruct**, an open-weight, instruction-tuned model released by Alibaba’s Qwen team, small enough to load and run on an ordinary laptop CPU. “1.5B” means 1.5 billion parameters — small by current standards, which is

1.5. Why not train a model from scratch on our own reports?

exactly why it's usable without a GPU; Chapter 5 revisits this size trade-off when you fine-tune it.

💡 Engineering Translation: pretraining

Getting to “Qwen2.5-1.5B-Instruct” from nothing takes more than one stage, both far bigger jobs than anything this book does. **Pretraining** reads a huge amount of general text and adjusts every one of the model's 1.5 billion numbers, over and over, until it can write fluent, grammatical, sensibly-reasoned language. A further stage — instruction tuning, the reason for the **-Instruct** in the name — then trains it specifically to follow prompts and hold a conversation, rather than just continue a piece of text. **Fine-tuning** — this book's whole subject, starting in Chapter 5 — takes a model that's already been through both of those stages and adjusts it further, on a much smaller amount of your own text.

1.5. Why not train a model from scratch on our own reports?

A reasonable question before going any further: if a general-purpose model doesn't know this operation's shorthand, why not just train a model on this archive's reports from the very start, instead of starting with someone else's model at all?

Because “training from scratch” means teaching a model to read and write English — grammar, common sense, how to hold a conversation — at the same time as teaching it drilling and completions. Pretraining a model like Qwen2.5-1.5B took an enormous amount of general text and compute to get right; even a large company's entire report archive, every well, every year, is nowhere near enough text on its own to teach a model language itself, let alone your domain on top of it. Fine-tuning sidesteps that problem entirely: it starts from a model that has already learned to read, write, and reason, and only asks it to learn one more, much narrower thing — how *this* operation talks: its shorthand, its sequencing, the shape of its reports. That's a small enough job to do on a laptop, which training a model from scratch never would be. It's also a different job from memorizing every fact in the archive — Chapter 3 covers that distinction directly, and Chapter 5 measures exactly where fine-tuning alone does and doesn't get there.

1.6. Why not just use a cloud AI assistant?

A fair question before writing any code: ChatGPT, Claude, and similar hosted assistants already run instruction-tuned models, and you don't have to install anything. Why bother loading one yourself?

- **Confidentiality.** A hosted assistant means your prompt — and anything you paste into it, including report excerpts — leaves your machine and goes to someone else's server. Section 4 of Appendix A covers this in more depth, but the short version: real drilling and completions reports are usually confidential, and “paste it into a chatbot” is rarely a decision you're allowed to make unilaterally.

1. Loading and Running Your First Local LLM

- **You can't fine-tune what you don't have the weights to.** This book's whole arc — building a model that actually understands *your* operation — requires updating the model's own weights (Chapter 5 onward). A hosted API gives you access to someone else's model, not the weights themselves.
- **No ongoing per-query cost.** Once downloaded, a local model runs as many times as you want at no marginal cost.

None of this means hosted assistants are bad — they're the right tool for plenty of jobs. They're just not the tool for *this* job, which needs both privacy and the ability to change the model itself.

1.7. Implementation

1.7.1. Step 1: Load the model and tokenizer

1.8. What problem are we solving?

Before you can ask the model anything, its weights need to be downloaded (first run only) and loaded into memory, alongside its tokenizer — the component that translates between text and the numeric form the model actually processes (Chapter 4 covers tokenization in depth; for now, treat it as a black box that `transformers` handles for you).

1.9. Inputs

- The model name on Hugging Face: `Qwen/Qwen2.5-1.5B-Instruct`
- An internet connection for the first run only (the model is cached locally after that)

1.10. Expected Output

No visible output yet — this step just gets the model ready in memory. On the author's machine, the very first run (including the download) took about 80 seconds; every run after that, loading from the local cache took 3–4 seconds. Yours will vary with your connection and disk speed, but a first run under a few minutes and cached runs under 10 seconds are both normal.

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

MODEL_NAME = "Qwen/Qwen2.5-1.5B-Instruct"

tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
model = AutoModelForCausalLM.from_pretrained(MODEL_NAME,
    ↪ torch_dtype=torch.float32)
```

1.11. What just happened?

`AutoTokenizer` and `AutoModelForCausalLM` each downloaded (or reused the cached copy of) everything needed for this specific model, then loaded it into memory. `torch_dtype=torch.float32` is the CPU-safe numeric precision this book uses throughout, on CPU, GPU, or Apple Silicon alike — Chapter 5 shows how to move the same full-precision model onto a GPU (`model.to("cuda")`) if you have one, without changing this precision. (Depending on your installed `transformers` version, this argument may print a harmless deprecation notice suggesting `dtype=` instead — both work identically for this book’s purposes.)

1.12. Production Reality

This loads one model into memory and keeps it there for the life of the script. A real multi-user deployment has to think about things this chapter skips: what happens if the download is interrupted, whether multiple requests share one loaded model or each gets its own, and how to swap in a newer fine-tuned checkpoint without restarting everything. The book’s own companion app (`app/`) doesn’t solve any of that either — it’s a single-reader local tool, not a production service, and reuses this exact loading step directly. None of this changes how loading itself works — it’s what you’d build *around* this step for multiple concurrent users, which this book doesn’t attempt.

1.12.1. Step 2: Build a chat-formatted prompt and generate a reply

1.13. What problem are we solving?

The model doesn’t take a plain question — it expects input formatted as a conversation, following the chat template mentioned above. This step builds that formatted input, then asks the model to continue it (that’s what “generating” means: predicting what comes next, one token at a time, until it decides to stop or hits a length limit).

1.14. Inputs

- The loaded `model` and `tokenizer` from Step 1
- A question, as a plain string

1.15. Expected Output

A chunk of generated text, printed to your terminal, in the neighborhood of a paragraph or two, taking roughly 10–15 seconds on a CPU for a response this length (again, faster with a GPU, slower on older hardware).

1. Loading and Running Your First Local LLM

```
def generate_reply(model, tokenizer, user_message: str, max_new_tokens: int =
↪ 340) -> str:
    messages = [{"role": "user", "content": user_message}]
    prompt = tokenizer.apply_chat_template(
        messages, tokenize=False, add_generation_prompt=True
    )
    inputs = tokenizer(prompt, return_tensors="pt")

    output_ids = model.generate(
        **inputs,
        max_new_tokens=max_new_tokens,
        do_sample=False, # greedy decoding -- deterministic, same output every
↪ run
    )
    generated_ids = output_ids[0][inputs["input_ids"].shape[1]:]
    return tokenizer.decode(generated_ids, skip_special_tokens=True)

question = (
    "A driller's report says: 'During the slide lost tool face and "
    "became stuck.' What should the crew watch for on the next curve "
    "section?"
)
print(generate_reply(model, tokenizer, question))
```

Running this exact code prints:

Based on the information provided in the driller's report that "during the slide lost tool face and became stuck," it suggests that there was an issue with the drill bit or other tools getting caught during the descent of the drill string. This could indicate several potential issues:

1. **Tool Catching**: The drill bit or any attached tools may have become entangled due to debris, mud cake buildup, or other obstructions.
2. **Drill String Configuration**: There might be a misalignment or improper configuration of the drill string that caused the tools to get caught.
3. **Downhole Conditions**: The conditions downhole (such as pressure changes, temperature variations) could have affected the performance of the tools.

For the next curve section, the crew should pay close attention to the following:

- **Check Tool Positioning**: Ensure that all tools are properly positioned and not obstructed by anything.
- **Inspect Downhole Equipment**: Check the condition of the drill bit and any other tools to ensure they are undamaged and functioning correctly.
- **Monitor Pressure and Temperature**: Be aware of any significant changes in pressure or temperature that could affect the performance of the equipment.

- **Review Drilling Parameters**: Adjust drilling parameters if necessary to prevent further complications.

By being vigilant about these factors, the crew can help avoid similar issues and continue safely through the next curve section.

(Captured from a real run of this exact code, on CPU, with greedy decoding — you should get this exact text too, word for word; see “What just happened?” below for why.)

1.16. What just happened?

`apply_chat_template` wrapped your question in the formatting this model expects for a single user turn, then `generate` predicted the reply one token at a time until it naturally finished (or would have been cut off at `max_new_tokens`, whichever came first). `do_sample=False` means the model always picks its single most-likely next word — no randomness — which is why re-running this exact code gives you the identical answer shown above, every time. (You may also see a harmless warning about “generation flags... not valid” — that’s the model’s default config mentioning sampling settings that greedy decoding simply ignores; it’s not an error.)

Read the answer again next to the real report excerpt above. It’s fluent, confident, and organized — and it never once mentions that the crew already worked the pipe free that same shift, or references anything about WOB, torque, or SPP from the actual report. It’s a generic answer to a specific question. That gap is this entire book.

1.17. Practical exercise

Beginner

Try it yourself: open `code/chapter_01/load_local_model.py`, change the `question` variable in `main()` to something from your own role — Carlos (completions), Sarah (intervention), or Farid (production) would each ask something different — and re-run the script.

You’ll know it worked when: you get a fluent, plausible-sounding answer that doesn’t reference any specific report, well, or number from your own operation — because it can’t. It has none of that yet.

1.18. Field notes

 Field notes: the model doesn’t know ‘trip out of hole’

Query: “In one sentence, what does ‘trip out of hole’ mean?”

Result (greedy decoding): “‘Trip out of hole’ is an idiomatic expression that means to fall or stumble unexpectedly into something difficult or unpleasant.”

Result (sampled decoding, same question, different run): “The phrase ‘trip out of

1. Loading and Running Your First Local LLM

hole’ is not standard English and doesn’t have a widely recognized meaning in common usage...”

Why: “trip out of hole” (commonly abbreviated **TOOH**, and its synonym **POOH** — “pull out of hole” — both appear in this book’s own [glossary](#)) is basic, everyday oilfield terminology: pulling the drill string out of the wellbore. It appears throughout the sample archive **Drilling_038** above is drawn from. Qwen2.5-1.5B-Instruct’s general training data apparently didn’t include enough real oilfield operational text to have learned it — so it confidently guesses at an English idiom instead, or in the sampled run, denies the phrase means anything at all.

Lesson: fluent and confident is not the same as correct. The base model doesn’t yet know your domain’s most basic vocabulary, let alone its judgment calls — closing exactly that gap is what the rest of this book does. (Reproduce this yourself with `code/chapter_01/challenge/challenge.py`, below.)

1.19. Challenge exercise

Intermediate

Challenge: `model.generate()`’s `do_sample` argument controls whether the model always picks its single most likely next word (`do_sample=False`, what this chapter used) or samples with some randomness (`do_sample=True`). Write a script that asks the same question both ways and prints both answers side by side. A reference solution — including the exact “trip out of hole” comparison from Field notes above — is at `code/chapter_01/challenge/challenge.py`.

1.20. Key takeaways

- A “local” model means the weights and the computation both live on your own machine — nothing is sent anywhere once it’s downloaded.
- `transformers`’ `AutoModelForCausalLM` and `AutoTokenizer` handle downloading, caching, and loading a model with a few lines of code.
- A chat template formats your question the way the model was trained to expect it; `apply_chat_template` handles this for you.
- `do_sample=False` (greedy decoding) is deterministic — the same input always produces the same output — which is why this book can show you exact expected output and mean it.
- Fluent output is not the same as grounded output. The base model can write confidently about something it has never actually learned.

1.21. Repository files

File	Purpose
<code>code/chapter_01/load_local_model.py</code>	Loads the base model and generates a reply to one question
<code>code/chapter_01/challenge/challenge.py</code>	Reference solution: greedy vs. sampled decoding, side by side

1.22. What can you do now that you couldn't do before?

File	Purpose
<code>code/chapter_01/build_sample_training_set.py</code>	Reproduces <code>datasets/sample_training_set/</code> from the full archive
<code>datasets/sample_training_set/</code>	The 10-report curated Utah FORGE subset, including <code>Drilling_038</code>
<code>notebooks/chapter_01_explore.ipynb</code>	Interactive companion notebook

CHECKPOINT — Chapter 1

- ☒ Loaded an open-weight local model with `transformers`
- ☒ Sent it a question through its chat template and read back a reply
- ☒ Observed, on a real run, that a fluent answer can still be operationally ungrounded

WHAT YOU BUILT

`load_local_model.py` — a working script that loads Qwen2.5-1.5B-Instruct entirely locally and answers a question. Every later chapter in Part I builds directly on `load_model_and_tokenizer()` and `generate_reply()` from this file.

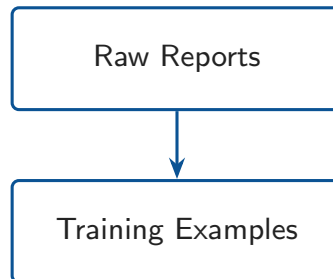
1.22. What can you do now that you couldn't do before?

You can load and run a real language model on your own machine, with no account, no API key, and no data leaving your computer — and you've seen direct, reproducible evidence of exactly what it does and doesn't understand yet.

1.23. Suggested next step

Coming up in Chapter 2: the base model has no idea what “trip out of hole” means because it never saw enough text like `Drilling_038` during its own training. Chapter 2 turns reports like that one into the instruction/response training examples that later chapters will actually fine-tune the model on.

2. Turning Drilling & Completions Reports into Training Examples



Progress ##..... 2 / 13 · Estimated time: 30–40 min · Difficulty: Beginner

2.1. Learning objectives

By the end of this chapter, you will be able to:

- Extract structured fields from a real report PDF with `pdfplumber`
- Explain what an instruction/response training example is, and why its output should come from the report itself rather than a written summary
- Turn a folder of reports into a training-ready `.jsonl` file
- Recognize when a report doesn't match your extractor's assumptions, and handle that case instead of silently getting it wrong
- Explain why one recognized, well-formed report should still be deliberately held out of training, before fine-tuning ever enters the picture

2.2. Operational Problem

Oumy, still turning over what Chapter 1 showed her, asks the obvious next question: *“The report file itself already says what happened, plainly — ‘Present Operations,’ ‘Activity Planned.’ Why can’t we just point the model at that?”*

Because a model doesn't learn from being pointed at a folder of PDFs. It learns from *training examples* — a specific, structured shape a fine-tuning process (Chapter 5) can actually consume. This chapter turns that folder of PDFs into exactly that shape, using nothing but the reports' own words.

2.3. Example report excerpt

The same real report from Chapter 1, `Drilling_038` (2020-11-26), has two short, plain-language fields sitting right at the top of the form, before any of the technical tables:

 Sample report excerpt – `FORGE-16A-78-32_Drilling_038_2020-11-26.pdf`

PRESENT OPERATIONS: PIPE FREE, TRIP OUT OF HOLE FOR BHA INSPECTION
ACTIVITY PLANNED: INSPECT BHA, TRIP IN HOLE DRILL CURVE

These two fields — a snapshot of “what’s happening right now” and “what’s planned next” — exist on every drilling report in this book’s archive. They’re the raw material this chapter turns into training examples.

2.4. Theory

 Engineering Translation: instruction/response pair

A **training example** is like a single flashcard: the question you’d ask (the *instruction*, plus optional *input* context), and the correct answer (the *output*), written down together so a model can study from it later. This book uses that three-part shape — **instruction**, **input**, **output** — throughout.

 Engineering Translation: field extraction

Field extraction means finding one specific labeled piece of information inside a big block of text — the way you’d scan a paper form for the line that starts with “Total:” rather than reading the whole page. This chapter does it with regular expressions (*regex*): a compact pattern-matching syntax for “find text that looks like this.”

2.5. Why not just write our own summaries?

A tempting shortcut: read each report and write a one-sentence summary by hand (or ask another AI to write one), and use that as the training **output**. Resist it. The moment a summary is written by anyone other than the report itself, it can be subtly wrong — a detail dropped, a number transposed, a plausible-sounding guess where the report was actually ambiguous — and that error becomes permanent, invisible ground truth. Later chapters build a data quality gate (Chapter 6) and an evaluation harness (Chapter 11) that both assume your labels are trustworthy; if the labels themselves are fabricated, everything built on top of them inherits the mistake.

This chapter’s technique sidesteps the problem entirely: every **output** below is the report’s own field value, extracted verbatim. Nothing is summarized, paraphrased, or invented. It’s a narrower kind of training example than a rich free-text summary would be — Chapter 7 builds richer ones — but every word of it is real.

2.6. Implementation

2.6.1. Step 1: Extract structured fields from one report

2.7. What problem are we solving?

Before anything can become a training example, you need the report's raw text, and then the two specific fields inside it — PRESENT OPERATIONS and ACTIVITY PLANNED — pulled out from everything else on the form (casing tables, mud properties, survey data) that isn't relevant here.

2.8. Inputs

- One report PDF, e.g. `datasets/sample_training_set/FORGE-16A-78-32_Drilling_038_2020-11-26.pdf`

2.9. Expected Output

A dictionary of the fields this report actually contains.

```
import re
import pdfplumber

FIELD_PATTERNS = {
    "well_name": r"WELL NAME:\s*(.+?)\s+JOB:",
    "rpt_date": r"RPT DATE:\s*([\d/]+)",
    "rpt_num": r"RPT NUM\.\?:\s*(\d+)",
    "present_operations": r"PRESENT OPERATIONS:\s*(.+?)\nACTIVITY PLANNED",
    "activity_planned": r"ACTIVITY PLANNED:\s*(.+?)\n",
}

def extract_text(pdf_path):
    with pdfplumber.open(pdf_path) as pdf:
        return pdf.pages[0].extract_text()

def extract_fields(text):
    fields = {}
    for name, pattern in FIELD_PATTERNS.items():
        match = re.search(pattern, text)
        fields[name] = match.group(1).strip() if match else None
    return fields

text = extract_text(
    ↪ "datasets/sample_training_set/FORGE-16A-78-32_Drilling_038_2020-11-26.pdf")
print(extract_fields(text))
```

2. Turning Drilling & Completions Reports into Training Examples

Running this exact code prints:

```
{'well_name': 'FORGE 16A [78]-32', 'rpt_date': '11/26/2020', 'rpt_num': '38',  
  'present_operations': 'PIPE FREE, TRIP OUT OF HOLE FOR BHA INSPECTION',  
  'activity_planned': 'INSPECT BHA, TRIP IN HOLE DRILL CURVE'}
```

2.10. What just happened?

`pdfplumber` read the PDF and returned its text as one long string — column layout flattened, so lines that appear side by side on the page (like a table’s column headers and a row of values) can end up on separate lines here. `extract_fields` then used regex to find each labeled field regardless of exactly where it landed in that string.

2.11. Production Reality

This regex assumes a specific, fixed field layout — the one every `Drilling_NNN` report in this archive happens to share. `pdfplumber` itself doesn’t get everything right either: scanned or handwritten reports don’t have extractable text at all (Chapter 6 covers detecting that). And even for a clean, digital report, a different operator’s form, or even this same operator’s *completion* reports, can use different field labels entirely — which is exactly what you’ll run into next.

2.11.1. Step 2: Turn one report’s fields into training examples

2.12. What problem are we solving?

Extracted fields aren’t training examples yet — they need to be reshaped into the `instruction / input / output` triples a fine-tuning process expects, and a report that doesn’t have both fields needs to be skipped rather than silently producing a broken example.

2.13. Inputs

- The `extract_fields()` dictionary from Step 1, for every report in `datasets/sample_training_set/`

2.14. Expected Output

Two training examples per successfully-parsed report; a printed notice naming any report that was skipped.

```

from datetime import datetime

def normalize_date(raw_date):
    return datetime.strptime(raw_date, "%m/%d/%Y").strftime("%Y-%m-%d")

def build_examples_for_report(pdf_path):
    text = extract_text(pdf_path)
    fields = extract_fields(text)

    if not fields["present_operations"] or not fields["activity_planned"]:
        return [] # unrecognized field layout -- see Production Reality above

    context = (
        f"Well: {fields['well_name']} | Report #{fields['rpt_num']} | "
        f"Date: {normalize_date(fields['rpt_date'])}"
    )
    return [
        {
            "instruction": "What are the present operations reported on this  

↪ well?",
            "input": context,
            "output": fields["present_operations"],
        },
        {
            "instruction": "What activity is planned next?",
            "input": context,
            "output": fields["activity_planned"],
        },
    ]

```

Running `build_examples_for_report()` on `Drilling_038` prints:

```

[
  {
    "instruction": "What are the present operations reported on this well?",
    "input": "Well: FORGE 16A [78]-32 | Report #38 | Date: 2020-11-26",
    "output": "PIPE FREE, TRIP OUT OF HOLE FOR BHA INSPECTION"
  },
  {
    "instruction": "What activity is planned next?",
    "input": "Well: FORGE 16A [78]-32 | Report #38 | Date: 2020-11-26",
    "output": "INSPECT BHA, TRIP IN HOLE DRILL CURVE"
  }
]

```

2.15. What just happened?

Each report becomes two training examples, one per field, sharing the same `input` context (well, report number, date) so the model can learn to condition its answer on which report is being asked about. `normalize_date` just makes every date consistently YYYY-MM-DD, since this archive's own reports don't all format dates the same way.

2.15.1. Step 3: Build and save the full training set — minus one report, on purpose

2.16. What problem are we solving?

One report is a proof of concept; Chapter 5's fine-tune needs the whole sample archive, saved to a file the next chapters can load without re-running PDF extraction every time. But if Chapter 5 trains on every report and then tests on those same reports, a good score would only prove the model *memorized* 18 flashcards — not that fine-tuning taught it anything that generalizes. This step reserves one drilling report, `Drilling_037` (2020-11-25), out of the training set entirely, so Chapter 5 has a report the model has genuinely never seen to test against.

2.17. Inputs

- Every PDF in `datasets/sample_training_set/`

2.18. Expected Output

A `.jsonl` file (one JSON object per line — the standard format this book uses for training data) plus summary lines reporting the held-out report, how many examples were built, and how many reports were skipped.

```
import json
from pathlib import Path

SAMPLE_SET_DIR = Path("datasets/sample_training_set")
OUTPUT_PATH = Path("datasets/training_examples/sample_training_examples.jsonl")
HELD_OUT_REPORT = "FORGE-16A-78-32_Drilling_037_2020-11-25.pdf"

def build_training_examples(sample_dir=SAMPLE_SET_DIR):
    examples, skipped, held_out = [], [], []
    for pdf_path in sorted(sample_dir.glob("*.pdf")):
        if pdf_path.name == HELD_OUT_REPORT:
            held_out.append(pdf_path.name)
            continue
        report_examples = build_examples_for_report(pdf_path)
        if report_examples:
```

```

        examples.extend(report_examples)
    else:
        skipped.append(pdf_path.name)
    if held_out:
        print(f"Reserved {len(held_out)} held-out report(s) for Chapter 5's
        ↪ generalization check: {held_out}")
    if skipped:
        print(f"Skipped {len(skipped)} report(s) with an unrecognized field
        ↪ layout: {skipped}")
    return examples

def save_examples_jsonl(examples, output_path=OUTPUT_PATH):
    output_path.parent.mkdir(parents=True, exist_ok=True)
    with output_path.open("w", encoding="utf-8") as f:
        for example in examples:
            f.write(json.dumps(example) + "\n")

examples = build_training_examples()
save_examples_jsonl(examples)
print(f"Built {len(examples)} training examples -> {OUTPUT_PATH}")

```

Running this exact code prints:

```

Reserved 1 held-out report(s) for Chapter 5's generalization check: ['FORGE-16A-
78-32_Drilling_037_2020-11-25.pdf']
Skipped 1 report(s) with an unrecognized field layout: ['FORGE-16A-78-32_Completion_003_2021-
01-06.pdf']
Built 16 training examples -> datasets/training_examples/sample_training_examples.jsonl

```

2.19. What just happened?

Of the ten sample reports, one (Completion_003) is a *completion* report using different field labels (Present Ops: / Next 24 Hours: instead of PRESENT OPERATIONS: / ACTIVITY PLANNED:) and gets correctly skipped rather than silently producing a garbage example. A second, Drilling_037, is a perfectly normal, recognized drilling report — it isn't skipped for a data-quality reason, it's *deliberately withheld* from training. That leaves eight reports, each contributing two examples — 16 total. `datasets/training_examples/` is generated, not committed to this repository — regenerate it any time by re-running this script (see `book/.gitignore`).

💡 Engineering Translation: held-out data

Held-out data is data you deliberately keep out of training so you have something honest to test on afterward — like a well test string never touched during a workover, kept clean specifically so its before/after readings mean something. A model tested only on the exact data it trained on can't tell you whether it learned anything general; it can only tell you whether it

2. Turning Drilling & Completions Reports into Training Examples

memorized.

This is a deliberately simple first training set — every **output** is a single field value copied verbatim from a report, not a rich summary or judgment call. That’s on purpose (see “Why not just write our own summaries?” above), but it also means this training set mostly teaches field recall, not operational reasoning. Later chapters build richer training examples (Chapter 7) and a proper evaluation harness beyond one held-out report (Chapter 11); this chapter’s job is just to get a first, honest training/held-out split in place before fine-tuning ever starts.

2.20. Practical exercise

Beginner

Try it yourself: run `python code/chapter_02/build_training_examples.py` from inside book /, then open the generated `datasets/training_examples/sample_training_examples.jsonl` in a text editor.

You’ll know it worked when: the script prints `Built 16 training examples`, the file has exactly 16 lines, the first one matches report #3 (`Drilling_003`, 2020-10-22) — the earliest report in the sample set — and no line anywhere in the file mentions `Report #37`.

2.21. Field notes

 **Field notes:** one field, not the whole story

Query/Action: extracting fields for reports #49 and #50 — the same packers-fail-to-fishing sequence referenced in the author’s previous book.

Result: report #49’s `activity_planned` is literally “FISHING OPERATIONS” — confirming that real event directly from this archive. Report #50’s `present_operations`, the very next day, is “TRIP IN HOLE WITH DIRECTIONAL BHA” — not a restatement of “fishing” at all, even though report #49 had just announced it as the plan.

Why: the reports say (report #49, `activity_planned`): “FISHING OPERATIONS”; (report #50, `present_operations`): “TRIP IN HOLE WITH DIRECTIONAL BHA”. Each field is a narrow, honest snapshot of *one* report at *one* moment — it was never written to also summarize what happened the next day.

Lesson: even perfectly clean, extractive ground truth like this only answers questions about a single report in isolation. Connecting the story across reports — the plan in #49 to the actual outcome by #50 — needs more than two fields from one PDF. That’s exactly why Part II exists: Chapter 9’s hybrid retrieval is what eventually lets a question span more than one report.

2.22. Challenge exercise

Intermediate

Challenge: extend `build_examples_for_report()` to also handle the completion report's different field layout (`Present Ops: / Next 24 Hours:`, and a two-digit-year date format), so 9 of the 10 sample reports contribute examples instead of 8 (`Drilling_037` stays reserved either way — see Step 3). A reference solution is at `code/chapter_02/challenge/challenge.py` — it produces 18 training examples total, 0 reports skipped.

2.23. Key takeaways

- A training example in this book is an `instruction / input / output` triple — a single flashcard the model can study from.
- Using a report's own self-reported fields as the `output` avoids fabricating ground truth; the label is never anyone's guess.
- `pdfplumber` plus a handful of regex patterns reliably extracts structured fields from a fixed-form report — but only the form layout it was written for.
- Different report types (drilling vs. completion) can use different field labels even within the same archive. Detect and skip what your extractor doesn't recognize, rather than silently producing a bad example.
- Skipping and holding out are different things done for different reasons: a report gets *skipped* because your extractor can't parse it; a report gets *held out* on purpose, even though it parses fine, so there's an honest, unseen test left once fine-tuning starts.

2.24. Repository files

File	Purpose
<code>code/chapter_02/build_training_examples.py</code>	Extracts fields and builds instruction/response training examples
<code>code/chapter_02/challenge/challenge.py</code>	Reference solution: also handles the completion report's field layout
<code>datasets/sample_training_set/</code>	The 10 source report PDFs this chapter reads
<code>datasets/training_examples/sample_training_examples.jsonl</code>	Generated output (not committed — regenerate by running the script)
<code>notebooks/chapter_02_explore.ipynb</code>	Interactive companion notebook



CHECKPOINT — Chapter 2

- ☒ Extracted structured fields from a real report PDF with `pdfplumber`
- ☒ Built instruction/response training examples using a report's own words as ground truth
- ☒ Detected and skipped a report with an unrecognized field layout, instead of silently mishandling it

2. Turning Drilling & Completions Reports into Training Examples

- ☒ Reserved one recognized, well-formed report as held-out data, instead of training on everything available

💡 WHAT YOU BUILT

`build_training_examples.py` — turns 8 of the 9 drilling-format reports in `datasets/sample_training_set/` into 16 real training examples, saved to `datasets/training_examples/sample_training_examples.jsonl`. The 9th, `Drilling_037`, is reserved untouched for Chapter 5's held-out generalization check. Chapter 5 fine-tunes on data built exactly this way, at a larger scale.

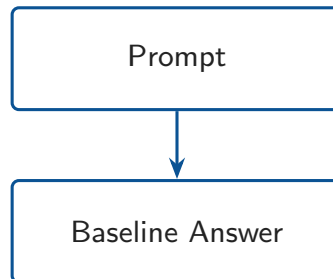
2.25. What can you do now that you couldn't do before?

You can turn a folder of real, inconsistently-formatted report PDFs into a training-ready dataset without inventing a single label — and you know precisely which reports your extractor handles and which it doesn't, instead of hoping.

2.26. Suggested next step

Coming up in Chapter 3: before spending any time actually fine-tuning, Chapter 3 runs a proper baseline test — the base model against a batch of real questions drawn from the 16 training examples you just built — so you have hard, reproducible evidence of exactly what it gets wrong, and something concrete to compare Chapter 5's fine-tuned model against afterward. `Drilling_037`, held out here, stays untouched until Chapter 5.

3. Baseline Prompting: What the Model Gets Wrong Before Fine-Tuning



Progress ###..... 3 / 13 · Estimated time: 20–30 min · Difficulty: Beginner

3.1. Learning objectives

By the end of this chapter, you will be able to:

- Turn Chapter 2’s training examples into a fixed batch of *training* baseline questions, with each example’s own *output* withheld from the model
- Run the unmodified base model against that exact batch and save its answers to a reusable results file
- Explain why a baseline must reuse the *same* prompts a later fine-tuned model will be tested on, not a different, looser sample
- Describe, from a real run against this book’s own archive, the specific way this base model fails when it isn’t given a report’s content — and why that’s different from wild hallucination
- Explain why a training baseline alone can’t prove fine-tuning generalizes, and why Chapter 5 needs a separate check against Chapter 2’s held-out report

3.2. Operational Problem

Sarah, the intervention engineer, is skeptical: *“Before we spend any time fine-tuning this thing, how do we even know it’s wrong today? And once it’s fine-tuned, how do we prove it actually got better, instead of just feeling better?”*

Chapter 1 showed the base model producing a fluent, generic answer with no report-specific detail. That was one question, read once, judged by eye. It’s not something you can compare against later, and it isn’t built from the same training examples Chapter 5 will actually fine-tune on. This chapter

3. Baseline Prompting: What the Model Gets Wrong Before Fine-Tuning

fixes both problems: it takes the exact instruction/input prompts Chapter 2 built, asks the base model every one of them with the real answer hidden, and saves the results to a file — a fixed, reproducible baseline Chapter 5's fine-tuned model gets held against, not a vibe.

One caveat up front, because it matters for how you read Chapter 5's result later: everything in this chapter runs on the same 16 examples Chapter 5 trains on. That makes it a **training baseline**, not a test of generalization — proof the model *can learn these exact examples*, not proof it learned anything beyond them. Chapter 2 reserved one report, **Drilling_037**, specifically so Chapter 5 has something the model genuinely never trained on to test against too.

3.3. Example baseline question

Chapter 2 turned report **Drilling_038** (2020-11-26 — the stuck-pipe day from Chapter 1) into this training example:

 Training example – from Chapter 2

```
{
  "instruction": "What activity is planned next?",
  "input": "Well: FORGE 16A [78]-32 | Report #38 | Date: 2020-11-26",
  "output": "INSPECT BHA, TRIP IN HOLE DRILL CURVE"
}
```

A training example and a baseline question are the same prompt used two different ways. Fine-tuning (Chapter 5) shows the model the **output** so it can learn from it. A baseline never does — this chapter sends the model only the **instruction** and **input**, then compares whatever it says back against the **output** it was never shown.

3.4. Theory

 Engineering Translation: baseline

A **baseline** is a fixed, saved measurement taken *before* you change anything — like recording a well's pressure before an intervention, so you have something concrete to compare the after-reading against. A baseline you can't reproduce, or that used different questions each time, isn't useful for that comparison.

 Engineering Translation: exact-match scoring

This chapter's **matched_expected** check asks one narrow question: does the model's answer contain the report's exact expected value, word for word? It's the simplest possible scoring rule — strict enough to never give false credit, but too strict to reward an answer that's correct in substance but phrased differently. Chapter 11 builds a more forgiving evaluation harness; this chapter deliberately starts with the crude, honest version.

3.5. Why not just eyeball a few answers?

A tempting shortcut: run the model on two or three questions, read the replies, and decide informally whether it “seems okay.” That’s exactly what Chapter 1 did, and it doesn’t scale into evidence. You can’t compare “seemed okay” across a before-and-after fine-tune, you can’t tell whether one lucky answer is representative of the other seventeen, and you have nothing saved to point to later if someone asks how you know fine-tuning helped.

This chapter’s harness fixes all three: the same 16 training questions, every time; every answer saved to `datasets/training_examples/training_baseline_results.jsonl`; and a single number — how many answers contained the report’s actual value — that Chapter 5 can reproduce and compare against after fine-tuning.

3.6. When fine-tuning is, and is not, the right tool

Every question this chapter asks has the same shape: “what does *this specific report* say?” — a fact lookup. That’s a deliberate, narrow choice (Chapter 2 explains why), but it’s worth being honest about what it does and doesn’t tell you about fine-tuning in general, before Chapter 5 spends real time on it.

Fine-tuning (Chapter 5 onward) is genuinely good at:

- Teaching a model your operation’s vocabulary and shorthand (the POOH/BHA-style terms Chapter 4 tests directly)
- Adapting its writing style to match how your reports are actually written, instead of a generic assistant’s voice
- Replacing vague, generic-sounding answers with ones that reflect your domain’s actual patterns and judgment calls
- Making “did this get better?” measurable and reproducible, which is the entire point of this chapter’s harness

It is a weaker tool for:

- Recalling one specific fact from one specific report it may or may not have trained on — memorizing 16 answers is not the same as building a lookup system, and Chapter 5 shows exactly where that distinction breaks down
- Citing which source document an answer came from — a fine-tuned model’s weights don’t carry provenance
- Replacing a proper document search / retrieval system for questions that span reports it never trained on
- Proving anything about generalization from a small training set on its own — that claim needs a held-out check, not just a lower loss number

At a glance, the two lists above come down to this:

3. Baseline Prompting: What the Model Gets Wrong Before Fine-Tuning

Question	Fine-tuning	Retrieval
Teach the model your shorthand and vocabulary?	Yes	No
Match your reports' own writing style?	Yes	No
Recall one specific fact from one specific report?	No	Yes
Cite which source document an answer came from?	No	Yes
Answer questions about reports it never trained on?	No	Yes
Prove it generalizes, on its own?	No	n/a

That's a table worth being able to redraw from memory — it's the argument for why this book eventually uses both together (Chapter 9), not a case for picking one and discarding the other.

This book starts with fine-tuning anyway, because *teaching the model your domain* is a real, necessary step even for a system that will later add retrieval on top of it. Chapter 9 is where those two techniques combine — fine-tuning for how your operation talks and reasons, retrieval for grounding answers in a specific, citable report. Neither replaces the other; Chapter 5's own results are the concrete evidence for why.

3.7. Implementation

3.7.1. Step 1: Turn Chapter 2's training examples into baseline questions

3.8. What problem are we solving?

Each Chapter 2 training example is a dictionary with `instruction`, `input`, and `output`. A baseline question is just the first two, joined into the single string the model actually receives — the `output` must never appear in it, or the test proves nothing.

3.9. Inputs

- One training example (as built by `build_training_examples()` in Chapter 2)

3.10. Expected Output

A plain-text prompt string, with no trace of the expected answer.

```
import sys
from pathlib import Path

BOOK_ROOT = Path(__file__).resolve().parents[2]
sys.path.insert(0, str(BOOK_ROOT / "code" / "chapter_01"))
sys.path.insert(0, str(BOOK_ROOT / "code" / "chapter_02"))

from build_training_examples import build_training_examples
from load_local_model import MODEL_NAME, generate_reply, load_model_and_tokenizer

def build_prompt(example: dict) -> str:
    return f"{example['instruction']}\n{example['input']}"

examples = build_training_examples()
print(build_prompt(examples[0]))
```

Running this exact code prints:

```
What are the present operations reported on this well?
Well: FORGE 16A [78]-32 | Report #3 | Date: 2020-10-22
```

3.11. What just happened?

`build_prompt` concatenates `instruction` and `input` with a newline — nothing more. `build_training_examples()` is the exact same function Chapter 2 uses to build the training set, imported directly rather than copied, so this chapter's questions are guaranteed to be the same ones Chapter 5 eventually trains on.

3.12. Production Reality

Every one of these questions only ever gives the model the report's *metadata* — well name, report number, date — never the report's actual text. That's deliberate: it isolates exactly what fine-tuning is for (Chapter 5 bakes each report's specific facts into the model's weights, so it no longer needs the text handed to it). A separate, equally valid question — can the model answer correctly when it *is* given the report text? — is this chapter's challenge exercise, and it previews why Chapter 9 eventually combines fine-tuning with retrieval instead of picking one or the other.

3. Baseline Prompting: What the Model Gets Wrong Before Fine-Tuning

3.12.1. Step 2: Ask the base model, with the real answer hidden

3.13. What problem are we solving?

For one baseline question, get the base model's answer and check it against the report's real value — the smallest unit of this chapter's harness.

3.14. Inputs

- The loaded `model` and `tokenizer` from Chapter 1
- One training example

3.15. Expected Output

A dictionary recording the question, the hidden expected answer, the model's actual answer, and whether they matched.

```
def ask_baseline(model, tokenizer, example: dict, max_new_tokens: int = 60) -> dict:
    ↪ prompt = build_prompt(example)
    ↪ answer = generate_reply(model, tokenizer, prompt,
    ↪ max_new_tokens=max_new_tokens)
    ↪ expected = example["output"]
    ↪ return {
        "instruction": example["instruction"],
        "input": example["input"],
        "expected": expected,
        "base_model_answer": answer,
        "matched_expected": expected.strip().lower() in answer.strip().lower(),
    }
```

```
model, tokenizer = load_model_and_tokenizer(MODEL_NAME)
print(ask_baseline(model, tokenizer, examples[0]))
```

Running this exact code prints:

```
{
  "instruction": "What are the present operations reported on this well?",
  "input": "Well: FORGE 16A [78]-32 | Report #3 | Date: 2020-10-22",
  "expected": "RIGGING UP WITH FRONTIER CREWS AND SUPERVISION AND JW JONES
  ↪ TRUCKING",
  "base_model_answer": "The report you've provided appears to be for a well
  ↪ called \"FORGE 16A [78]-32\" and it is dated October 22, 2020. The specific
  ↪ details of what operations were reported are not explicitly stated in your
  ↪ message. However, based",
```

```
"matched_expected": false
}
```

3.16. What just happened?

The model was asked exactly what Chapter 2’s `input` field contains — well name, report number, date — and nothing else. It answers honestly that “the specific details... are not explicitly stated,” rather than inventing a plausible-sounding operation. That’s a meaningfully *better* failure than confident fabrication would be, but it’s still a failure: the harness needed the report’s real value, and didn’t get it.

3.17. Production Reality

`max_new_tokens=60` caps the reply short on purpose, so this chapter’s 16-question run finishes in a couple of minutes on CPU. A shorter cap also means some replies (like the one above) get cut off mid-sentence rather than reaching a natural stopping point — acceptable here because `matched_expected` only checks for the report’s specific value, which would appear (or not) well before token 60 in every one of this archive’s short, single-line field outputs.

3.17.1. Step 3: Run the full training baseline and save it for Chapter 5

3.18. What problem are we solving?

One question proves the harness works. Chapter 5 needs all 16, saved somewhere it can load without re-running the base model.

3.19. Inputs

- Every training example from `build_training_examples()`

3.20. Expected Output

A summary line reporting how many of the 16 answers matched, and a saved `.jsonl` file with one result per question.

3. Baseline Prompting: What the Model Gets Wrong Before Fine-Tuning

```
import json

TRAINING_BASELINE_OUTPUT_PATH = BOOK_ROOT / "datasets" / "training_examples" /
    ↪ "training_baseline_results.jsonl"

def run_baseline(model, tokenizer, examples: list[dict]) -> list[dict]:
    return [ask_baseline(model, tokenizer, example) for example in examples]

def save_results_jsonl(results: list[dict], output_path: Path =
    ↪ TRAINING_BASELINE_OUTPUT_PATH) -> None:
    output_path.parent.mkdir(parents=True, exist_ok=True)
    with output_path.open("w", encoding="utf-8") as f:
        for result in results:
            f.write(json.dumps(result) + "\n")

results = run_baseline(model, tokenizer, examples)
save_results_jsonl(results)

matched = sum(r["matched_expected"] for r in results)
print(f"Training baseline: {matched}/{len(results)} answers contained the
    ↪ report's actual value verbatim")
print(f"Saved -> {TRAINING_BASELINE_OUTPUT_PATH}")
```

Running this exact code prints:

```
Training baseline: 0/16 answers contained the report's actual value verbatim
Saved -> datasets/training_examples/training_baseline_results.jsonl
```

3.21. What just happened?

Every one of the 16 questions built from the 8 recognized, non-held-out sample reports came back with `matched_expected: false`. Not one exact number, status, or operation name from the archive appears in any answer — because the model was never given any of them, and it has no other way to know which specific well this is. 0/16 is this chapter's training baseline. Chapter 5 reruns this exact harness against the fine-tuned model and compares its score directly against this one — but that comparison alone only shows training recall. See below.

3.22. Training recall vs. generalization

A 0/16 baseline is easy to beat: fine-tune on these same 16 examples, rerun this harness, and the score will very likely improve — a small model with a handful of LoRA-adjustable parameters

(Chapter 5) can often just memorize 16 short flashcards. That improvement would be real, but it would only prove the model *learned these exact examples*, not that it learned anything that transfers to a report it has never seen.

That’s exactly why Chapter 2 reserved `Drilling_037` out of training entirely. Chapter 5 has to report two separate numbers, not one: training-set recall on these 16 examples, and a held-out generalization score on `Drilling_037`’s own 2 examples — a report the fine-tuned model never trained on. Only the second number is evidence of learning that generalizes; the first is evidence of memorization capacity. Neither number alone tells the full story, and Chapter 11 later replaces this one-report check with a proper evaluation harness — but even a single honest held-out report beats zero.

3.23. Practical exercise

Beginner

Try it yourself: run `python code/chapter_03/baseline_prompting.py` from inside `book/`, then open `datasets/training_examples/training_baseline_results.jsonl` and read through all 16 answers.

You’ll know it worked when: the script prints `Training baseline: 0/16`, the file has exactly 16 lines, none of the model’s answers contain a specific operation, status, or number from the actual report (most will instead say something like “not explicitly stated”), and no line mentions `Report #37` — that report is reserved, not tested here.

3.24. Field notes

 Field notes: the model invents the wrong kind of document

Query: report #49’s “What activity is planned next?” question — the same fishing-operations report referenced in Chapter 2’s Field notes.

Result: *“The text you provided appears to be from an online forum or discussion board where someone has posted their progress on a project called ‘FORGE 16A’ with the report number #49 and the date of December 7, 2020. Without more context about what this refers...”*

Why: report #49’s real `activity_planned` value is “FISHING OPERATIONS” — the model was never given that, only the metadata string “Well: FORGE 16A [78]-32 | Report #49 | Date: 2020-12-07”. Rather than guess at an oilfield operation, it guesses at what *kind of document* would contain a bare project name and report number — and lands on “an online forum or discussion board post,” which is a reasonable guess for generic internet text and a completely wrong one for a daily drilling report.

Lesson: this base model doesn’t just lack this well’s facts — it doesn’t yet recognize the *shape* of the document it’s being asked about. That’s a second, deeper gap fine-tuning has to close, beyond memorizing individual report values.

3.25. Challenge exercise

Intermediate

Challenge: `baseline_prompting.py` only ever gives the model each report’s metadata, never its actual text — so 0/16 mostly proves the model has no way to know the answer, not that fine-tuning is the only possible fix. Extend the harness to include each report’s full extracted text (from Chapter 2’s `extract_text()`) alongside the question, and re-measure the match rate. A reference solution is at `code/chapter_03/challenge/challenge.py` — on a real run, it scores 6/16, up from 0/16, but still well short of perfect: even reading the report directly, this small base model doesn’t reliably reproduce its exact wording. Keep that number in mind for Chapter 9, which combines fine-tuning with exactly this kind of retrieved context.

3.26. Key takeaways

- A baseline is only useful if it’s fixed and reproducible: same questions, same withheld answers, saved results, every time.
- This chapter’s training baseline questions are Chapter 2’s real training examples, with the output withheld — not a separate, invented test set.
- On this book’s sample archive, the unmodified base model scores 0/16 on exact-match — not because it hallucinates wildly, but because it correctly (and unhelpfully) admits it wasn’t given the report’s content.
- A training baseline only measures memorization capacity. Proving fine-tuning generalizes needs a separate score on data the model never trained on — Chapter 2’s held-out report, `Drilling_037`, exists for exactly that reason, and Chapter 5 reports both numbers.
- `matched_expected` is a deliberately crude, honest scoring rule. Chapter 11 builds something more forgiving; this chapter’s harness exists so there’s a saved, comparable number to build on.

3.27. Repository files

File	Purpose
<code>code/chapter_03/baseline_prompting.py</code>	Builds training baseline questions from Chapter 2’s training examples, runs the base model, saves results
<code>code/chapter_03/challenge/challenge.py</code>	Reference solution: re-runs the same questions with each report’s full text included
<code>datasets/training_examples/training_baseline_results.jsonl</code>	Generated output (not committed — regenerate by running the script)
<code>notebooks/chapter_03_explore.ipynb</code>	Interactive companion notebook

3.28. What can you do now that you couldn't do before?

CHECKPOINT — Chapter 3

- ☒ Turned Chapter 2's training examples into training baseline questions with the answer withheld
- ☒ Ran the base model against all 16 questions and saved every answer
- ☒ Measured a real, reproducible training baseline score (0/16) to compare Chapter 5's fine-tuned model against
- ☒ Can explain why this training baseline alone won't prove generalization, and what Chapter 5's held-out check adds

WHAT YOU BUILT

baseline_prompting.py — runs the unmodified base model against Chapter 2's 16 training examples with every answer hidden, and saves the results to `datasets/training_examples/training_baseline_results.jsonl`. Chapter 5 reruns this same harness against the fine-tuned model for a training-recall comparison, and runs a second, separate check against the held-out report for a generalization comparison.

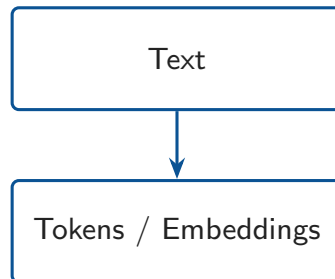
3.28. What can you do now that you couldn't do before?

You can produce a fixed, reproducible number for exactly how well the base model performs on your own training questions — something to point to, not just a feeling — and you have it saved to a file Chapter 5 will compare against directly. You can also tell the difference between “the model learned its training set” and “the model generalizes,” and you know which one this chapter's number actually proves.

3.29. Suggested next step

Coming up in Chapter 4: fine-tuning (Chapter 5) works by adjusting how the model represents text internally — but “text” isn't what a model actually reads. Chapter 4 opens up tokenization and embeddings, the layer underneath every prompt this chapter sent, so LoRA fine-tuning doesn't feel like a black box when you get there. Chapter 5 then reruns this chapter's training baseline against the fine-tuned model, and runs its held-out counterpart against `Drilling_037` for the first time.

4. Tokenization and Embeddings for Domain Fine-Tuning



Progress ##### 4 / 13 · Estimated time: 30–40 min · Difficulty: Intermediate

4.1. Learning objectives

By the end of this chapter, you will be able to:

- Explain what a tokenizer actually does to text before the model ever sees it, and show it splitting real oilfield shorthand into pieces
- Explain what an embedding is, and the difference between a raw, per-token embedding and a whole-phrase (contextual) embedding
- Show, from a real run, whether the base model’s raw embeddings or a general-purpose sentence-embedding model recognize this book’s own oilfield acronyms as equivalent to their spelled-out meaning
- Explain why neither result is evidence the model is “broken” — and what it means for what fine-tuning (Chapter 5) actually has to fix

4.2. Operational Problem

Farid, the production engineer, pushes back on Chapter 1’s finding: *“Sure, this small model didn’t know ‘trip out of hole’ — but ChatGPT and every other LLM I’ve used seem to handle abbreviations and jargon fine elsewhere. Isn’t that basically a solved problem by now?”*

It’s a fair challenge, and answering it honestly means looking one layer lower than any question-and-answer test can show. Before a model generates anything, your text gets cut into pieces and looked up against a fixed table of numbers — and that happens the same way whether the model has seen your domain’s jargon a million times or never once. This chapter looks directly at that layer: what

4. Tokenization and Embeddings for Domain Fine-Tuning

pieces this book's oilfield shorthand actually gets cut into, and whether the numbers those pieces get looked up against already know anything about what that shorthand means.

4.3. Example training excerpt

A real instruction/response pair, built the same way Chapter 2's `Drilling_038` examples were, but this one drawn from a genuine TIME BREAKDOWN entry in `Drilling_017` (2020-11-05), quoted exactly as filed:

 Sample training example – FORGE-16A-78-32_Drilling_017_2020-11-05.pdf

```
{
  "instruction": "Summarize what operation this report excerpt describes.",
  "input": "TOOH (trip out of hole) with BHA #2. Break off bit, lay down
    ↳ Scout RSS (rotary steerable).",
  "output": "A planned bottom hole assembly change: BHA #2 is pulled out of
    ↳ hole, its bit is broken off, and the rotary steerable tool is laid down
    ↳ ahead of picking up the next BHA."
}
```

TOOH and BHA sit right in the `input` above. Before this chapter is over, you'll know exactly how the model's tokenizer splits both of those, and how close — if at all — its own numbers place them next to what they actually mean.

4.4. Theory

 Engineering Translation: token

A **token** is the actual chunk of text a model reads — not necessarily a whole word. A **tokenizer** is the fixed rulebook that decides where those chunks break. It doesn't know your domain, your abbreviations, or your intent; it just mechanically applies the same splitting rules to everything, the way a shipping label printer wraps text at a fixed column width whether or not that's where a word actually ends.

 Engineering Translation: vocabulary

A model's **vocabulary** is the fixed list of pieces its tokenizer is allowed to produce — tens of thousands of them, decided once when the model was built and never updated by fine-tuning. If a term isn't one whole piece on that list, it gets cut into smaller pieces that are.

 Engineering Translation: embedding

An **embedding** is a list of numbers — a coordinate — that stands in for a token or a phrase, so the model can do math with meaning instead of with letters. Two coordinates being close together is supposed to mean “these mean similar things,” the same way two points close together on a map are supposed to mean “these places are near each other.” That promise only holds if the model actually learned to place them that way — nothing about having a coordinate guarantees it’s a *good* one.

4.5. Why “the model has seen jargon before” isn’t evidence

Farid’s instinct — that a language model trained on huge amounts of text has probably seen abbreviations and shorthand before — is reasonable. But “seen text like this somewhere” and “placed this specific term next to its specific meaning” are different claims, and only the second one matters for whether the model can answer a question about BHA correctly. This chapter tests the second claim directly, two ways: by looking at the model’s own raw, per-token embeddings, and by checking whether a dedicated, purpose-built sentence-embedding model — not this book’s small base model, something specifically designed to place similar meanings close together — does any better. Both results below are from real runs, not assumptions.

4.6. Implementation

4.6.1. Step 1: See exactly how the tokenizer splits oilfield shorthand

4.7. What problem are we solving?

Before asking anything about meaning, see the literal pieces the tokenizer produces — the raw material every later layer of the model has to work with.

4.8. Inputs

- The tokenizer loaded in Chapter 1 (Qwen/Qwen2.5-1.5B-Instruct)
- A handful of oilfield acronyms from this chapter’s example, plus a few ordinary English words for contrast

4.9. Expected Output

A token count and the exact pieces for each term.

4. Tokenization and Embeddings for Domain Fine-Tuning

```
import sys
from pathlib import Path

BOOK_ROOT = Path(__file__).resolve().parents[2]
sys.path.insert(0, str(BOOK_ROOT / "code" / "chapter_01"))

from load_local_model import MODEL_NAME, load_model_and_tokenizer

OILFIELD_TERMS = ["POOH", "TOOH", "BHA", "WOB", "SPP", "RPM"]
PLAIN_TERMS = ["pipe", "hello", "drilling", "stuck"]

def tokenize_with_pieces(tokenizer, text: str) -> list[str]:
    ids = tokenizer.encode(text, add_special_tokens=False)
    return tokenizer.convert_ids_to_tokens(ids)

model, tokenizer = load_model_and_tokenizer(MODEL_NAME)
for term in OILFIELD_TERMS + PLAIN_TERMS:
    pieces = tokenize_with_pieces(tokenizer, term)
    print(f"{term!r:12} -> {len(pieces)} token(s): {pieces}")
```

Running this exact code prints:

```
'POOH'      -> 2 token(s): ['PO', 'OH']
'TOOH'      -> 2 token(s): ['TO', 'OH']
'BHA'       -> 2 token(s): ['B', 'HA']
'WOB'       -> 2 token(s): ['W', 'OB']
'SPP'       -> 2 token(s): ['S', 'PP']
'RPM'       -> 2 token(s): ['R', 'PM']
'pipe'      -> 1 token(s): ['pipe']
'hello'     -> 1 token(s): ['hello']
'drilling'  -> 2 token(s): ['dr', 'illing']
'stuck'     -> 2 token(s): ['st', 'uck']
```

4.10. What just happened?

Every oilfield acronym split into exactly two arbitrary pieces — 'BHA' became 'B' + 'HA', with no more relationship to “bottom hole assembly” than those two letters carry on their own. But notice 'drilling' and 'stuck' split too, into pieces that are just as arbitrary ('dr' + 'illing', 'st' + 'uck') — so splitting itself isn’t a domain-jargon problem, it’s just how this tokenizer’s fixed vocabulary handles anything not common enough to earn its own whole token. 'pipe' and 'hello' happened to make the cut; 'BHA' didn’t.

4.11. Production Reality

This book’s tokenizer is fixed — it came with the base model and Chapter 5’s fine-tuning doesn’t change it. A production system dealing with truly nonstandard shorthand (a specific operator’s internal codes, say) sometimes trains or extends a custom tokenizer instead of just fine-tuning on top of a generic one; that’s a heavier, rarer move this book doesn’t cover, because for the vocabulary in this archive, as Step 2 shows next, the fix doesn’t need to happen at the tokenizer level at all.

4.11.1. Step 2: Check the base model’s own raw embeddings for these terms

4.12. What problem are we solving?

Tokenizing a term is not the same as the model understanding it. This step looks up the model’s own numeric coordinate for each piece, combines them into one vector per term, and measures how close an acronym’s vector is to its spelled-out meaning’s vector.

4.13. Inputs

- The loaded model from Step 1 (specifically, `model.get_input_embeddings()`)
- Pairs of oilfield shorthand and their real meanings

4.14. Expected Output

A cosine similarity score for each pair — closer to 1.0 means “close together,” closer to 0 means “no relationship,” negative means “pointing away from each other.”

```
import torch

TERM_PAIRS = [
    ("POOH", "trip out of hole"),
    ("TOOH", "trip out of hole"),
    ("BHA", "bottom hole assembly"),
    ("BHA", "pizza delivery"),
    ("trip out of hole", "pull out of hole"),
]

def term_embedding(model, tokenizer, term: str) -> torch.Tensor:
    ids = tokenizer.encode(term, add_special_tokens=False)
    input_ids = torch.tensor([ids])
    vectors = model.get_input_embeddings()(input_ids)[0]
    return vectors.mean(dim=0)
```

4. Tokenization and Embeddings for Domain Fine-Tuning

```
def cosine_similarity(a: torch.Tensor, b: torch.Tensor) -> float:
    return torch.nn.functional.cosine_similarity(a.unsqueeze(0),
        ↪ b.unsqueeze(0)).item()

for term_a, term_b in TERM_PAIRS:
    vec_a = term_embedding(model, tokenizer, term_a)
    vec_b = term_embedding(model, tokenizer, term_b)
    print(f"{term_a!r:20} vs {term_b!r:20} -> {cosine_similarity(vec_a,
        ↪ vec_b):.3f}")
```

Running this exact code prints:

```
'POOH'           vs 'trip out of hole'    -> 0.046
'TOOH'           vs 'trip out of hole'    -> 0.026
'BHA'            vs 'bottom hole assembly' -> -0.035
'BHA'            vs 'pizza delivery'      -> -0.111
'trip out of hole' vs 'pull out of hole'    -> 0.709
```

4.15. What just happened?

The one pair that scored high — 'trip out of hole' vs. 'pull out of hole' — shares three of its four literal tokens ('out', 'of', 'hole'), so a high score there mostly reflects token overlap, not proof the model reasoned about synonyms. The real test is the acronym pairs, which share *no* tokens with their meanings: 'BHA' scored -0.035 against its own spelled-out meaning — statistically no relationship at all, and barely different from -0.111 against 'pizza delivery'. These raw, per-token embeddings are the model's very first, static lookup table, before any contextual reasoning happens — and at that layer, an acronym and its meaning are strangers.

4.16. Production Reality

Raw input embeddings are the crudest possible measurement — a mean-pooled lookup with no context at all, nothing like how the model actually reasons once text moves through its attention layers during generation. A production system checking “does the model already know this term” would never stop at this layer. Step 3 checks a proper, purpose-built embedding model instead, to see if that changes the answer.

4.16.1. Step 3: Check a purpose-built sentence-embedding model too

4.17. What problem are we solving?

Maybe the problem is specific to this small base model's raw embedding table, not embeddings in general. This step repeats the exact same comparison using `sentence-transformers`, a general-purpose model built specifically to place similar meanings close together — the same kind of tool Chapter 9's retrieval system will use.

4.18. Inputs

- The same `TERM_PAIRS` from Step 2
- `sentence-transformers`' `all-MiniLM-L6-v2` model (downloaded once, then cached)

4.19. Expected Output

The same style of cosine similarity scores, from a different, dedicated embedding model.

```
from sentence_transformers import SentenceTransformer

st_model = SentenceTransformer("all-MiniLM-L6-v2")

for term_a, term_b in TERM_PAIRS:
    vec_a, vec_b = st_model.encode(term_a), st_model.encode(term_b)
    sim = float(
        torch.nn.functional.cosine_similarity(
            torch.tensor(vec_a).unsqueeze(0), torch.tensor(vec_b).unsqueeze(0)
        )
    )
    print(f"{term_a!r:20} vs {term_b!r:20} -> {sim:.3f}")
```

Running this exact code prints:

```
'POOH'           vs 'trip out of hole'   -> 0.249
'TOOH'           vs 'trip out of hole'   -> 0.167
'BHA'            vs 'bottom hole assembly' -> 0.115
'BHA'            vs 'pizza delivery'      -> 0.128
'trip out of hole' vs 'pull out of hole'   -> 0.747
```

4.20. What just happened?

A real, dedicated embedding model does slightly better on 'P00H' (0.249, up from 0.046) — some improvement, though still a weak score for two phrases that mean exactly the same thing. But 'BHA' barely moved, and not in the direction that matters: 0.115 against its own meaning versus 0.128 against 'pizza delivery' — the unrelated phrase still scores *higher*. A general-purpose embedding model, built by people who never saw this archive, has no more reason to know that BHA means “bottom hole assembly” than the base language model did.

4.21. Practical exercise

Beginner

Try it yourself: run `python code/chapter_04/tokenize_and_embed.py` from inside `book/`, then open `datasets/full_training_set/FORGE-16A-78-32_Drilling_017_2020-11-05.pdf` and pick two more real abbreviations from its TIME BREAKDOWN table (RSS and TIH both appear, each self-glossed in parentheses right in the report, the same way T00H is above) and check how they tokenize.

You’ll know it worked when: you can point to the exact pieces each term splits into, and explain in one sentence why a short acronym isn’t necessarily “safe” from being split, even though it’s already short.

4.22. Field notes

⚠ Field notes: a blowout preventer scores lower than a birthday party

Query: how closely does each embedding approach place BOP (the industry-standard abbreviation for **blowout preventer**, a critical well-control safety device, appearing 36 times across this book’s 9 drilling reports) next to its own meaning — compared to an unrelated phrase, “birthday party”?

Result:

Comparison	Raw input embedding	Sentence embedding
BOP vs. "blowout preventer"	0.131	0.059
BOP vs. "birthday party"	-0.105	0.320

Why: the safest claim this result supports is the direct one — neither embedding approach encodes the equivalence between BOP and “blowout preventer” strongly enough for this task. Exactly *why* the sentence-embedding model happens to place “birthday party” closer is not something these two numbers prove on their own (this chapter doesn’t inspect the model’s internals to confirm a specific cause); what they do prove is that whatever 0.320 is picking up on, it isn’t domain understanding of well control.

Lesson: a safety-critical abbreviation is exactly the kind of term where “the embedding space probably has this covered” is the most dangerous assumption to make casually. Neither this chapter’s raw embeddings nor a real, dedicated embedding model can be trusted with your domain’s specific vocabulary out of the box. (Reproduce this yourself with `code/chapter_04/challenge/challenge.py`, below.)

4.23. Challenge exercise

Intermediate

Challenge: extend the comparison to real acronyms pulled from this book’s own archive instead of hand-picked examples — BOP (blowout preventer) and TVD (true vertical depth), both genuinely present across the 9 drilling reports (36 and 32 times respectively, on a real count). A reference solution is at `code/chapter_04/challenge/challenge.py` — on a real run, TVD vs. “true vertical depth” scores 0.024 raw / 0.118 sentence-embedding, in the same range as this chapter’s other acronym pairs, and the same run’s `count_term_occurrences()` reproduces both counts above (and the Field Notes’ 36 for BOP) directly from the archive.

4.24. Key takeaways

- A tokenizer mechanically splits text into fixed pieces from a vocabulary decided when the model was built — it has no awareness of your domain, and fine-tuning doesn’t change it.
- Splitting into multiple pieces isn’t unique to jargon — plenty of ordinary English words split too. The real question is whether the resulting pieces get placed anywhere near their meaning.
- Raw, per-token embeddings are the model’s crudest, first-layer lookup — a real run shows this book’s oilfield acronyms placed no closer to their own meanings than to an unrelated phrase.
- A dedicated, purpose-built sentence-embedding model does only marginally better, and for at least one safety-critical term (BOP) actually favors an unrelated phrase. Off-the-shelf embeddings, of either kind, cannot be assumed to already understand your domain’s specific shorthand.

4.25. Repository files

File	Purpose
<code>code/chapter_04/tokenize_and_embed.py</code>	Tokenizes oilfield terms and compares raw vs. sentence-embedding similarity

4. Tokenization and Embeddings for Domain Fine-Tuning

File	Purpose
<code>code/chapter_04/challenge/challenge.py</code>	Reference solution: real archive acronyms (BOP, TVD)
<code>notebooks/chapter_04_explore.ipynb</code>	Interactive companion notebook

CHECKPOINT — Chapter 4

- ☒ Inspected the exact sub-word pieces the tokenizer produces for real oilfield shorthand
- ☒ Measured the base model's raw embedding similarity between shorthand and its own spelled-out meaning
- ☒ Measured the same comparison with a dedicated sentence-embedding model, and confirmed it doesn't reliably do better

WHAT YOU BUILT

`tokenize_and_embed.py` — inspects tokenization and compares two kinds of off-the-shelf embeddings against this book's own oilfield shorthand, with real, reproducible numbers showing neither reliably understands it. This is the concrete evidence for *why* Chapter 5 has to actually update the model's weights, not just hope a bigger embedding model would fix it – though, as you'll see there, Chapter 5's fine-tune leaves this exact raw embedding table frozen and untouched; the fix happens at the attention layers downstream of it instead.

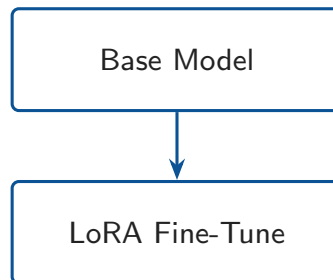
4.26. What can you do now that you couldn't do before?

You can show, with real numbers instead of intuition, exactly what a model reads before it ever generates a word — and you have direct, reproducible evidence that off-the-shelf embeddings, of any kind, can't be assumed to already understand your operation's specific shorthand.

4.27. Suggested next step

Coming up in Chapter 5: tokenization and embeddings are the fixed starting material every chapter until now has quietly relied on. Chapter 5 is where that finally changes — your first LoRA fine-tune, which updates the model's own weights using Chapter 2's 16 training examples, then reruns Chapter 3's training baseline *and* its held-out check against `Drilling_037` to see, honestly, how much actually improved.

5. Your First LoRA Fine-Tune



Progress #####..... 5 / 13 · Estimated time: 45–60 min · Difficulty: Intermediate

5.1. Learning objectives

By the end of this chapter, you will be able to:

- Explain what a LoRA adapter is: a small set of extra weights trained on top of a frozen base model, not a rewrite of the model itself
- Fine-tune the base model on Chapter 2’s 16 training examples with a plain PyTorch training loop — no framework magic hidden from you
- Rerun Chapter 3’s training baseline against your fine-tuned model and measure a real, honest improvement
- Run, for the first time in this book, a held-out generalization check against Chapter 2’s reserved report — and explain, from a real run, exactly *how* the fine-tuned model fails on it

5.2. Operational Problem

Carlos, the completions engineer, has watched this book build up evidence for four chapters — the base model doesn’t know this well’s vocabulary, doesn’t know its specific reports, and its embeddings don’t already understand the shorthand. He’s ready for the payoff, but wary of a different failure: *“If we fine-tune this on sixteen examples, are we teaching it our operation — or just teaching it to repeat sixteen sentences back at us?”*

That’s exactly the right question, and this chapter answers it two ways, not one. It fine-tunes the base model on Chapter 2’s 16 training examples, then checks *both* whether it learned those exact examples (training recall) *and* whether that learning transfers to a report it never saw during training (generalization, using Chapter 2’s reserved `Drilling_037`). Chapter 3 already scored the base model 0/16 and 0/2 on both. This chapter reruns that exact same harness against the fine-tuned model — same questions, same scoring, same code — so the comparison means something.

5.3. Example: what the model actually trains on

Fine-tuning doesn't feed the model `instruction/input/output` dictionaries directly — it feeds it the same chat-formatted text Chapter 1's `generate_reply` builds, except this time with the correct answer appended as the assistant's turn:

 What one training example looks like, chat-formatted

```
<|im_start|>system
You are Qwen, created by Alibaba Cloud. You are a helpful assistant.<|im_end|>
<|im_start|>user
What are the present operations reported on this well?
Well: FORGE 16A [78]-32 | Report #3 | Date: 2020-10-22<|im_end|>
<|im_start|>assistant
RIGGING UP WITH FRONTIER CREWS AND SUPERVISION AND JW JONES TRUCKING<|im_end|>
```

Fine-tuning's whole job is to adjust the model so that, given the `system` and `user` turns above, it becomes more likely to produce that exact `assistant` turn — and nothing else changes about how the model reads or generates text.

5.4. Theory

 Engineering Translation: LoRA adapter

LoRA stands for **Low-Rank Adaptation**. Full fine-tuning would mean updating all 1.5 billion of the base model's numbers. A **LoRA adapter** is a small, separate set of numbers — a correction that gets trained instead, while the original 1.5 billion stay frozen, untouched. LoRA does not replace the base model's weights: at inference time the model reads its usual weights *and* this correction, added on top, together — as if the correction had been written directly into the model, without the original numbers ever having changed.

“Low-rank” describes *how* that correction stays small: instead of training one full table of correction numbers — one for every connection in the layer — LoRA trains two much smaller tables. Multiply those two tables together and you get a full-size correction, the same shape as the layer it's correcting, without needing to store that full-size version separately. How narrow those two tables are is the adapter's **rank** (**r**=8 in Step 2 below) — small by design. Fewer numbers to store and train, at the cost of only approximating what a full-size correction could do.

 Engineering Translation: epoch and loss

One **epoch** is one full pass through the training data — all 16 examples, once. **Loss** is a single number measuring how wrong the model's predictions were on that pass; it should generally fall as training continues, the way a golf score should generally fall as you improve. Chapter 11 covers proper evaluation metrics; loss here is just a training-time signal, not a measure of correctness by itself.

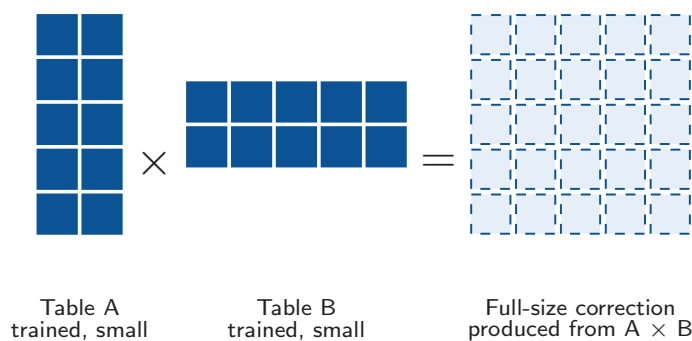
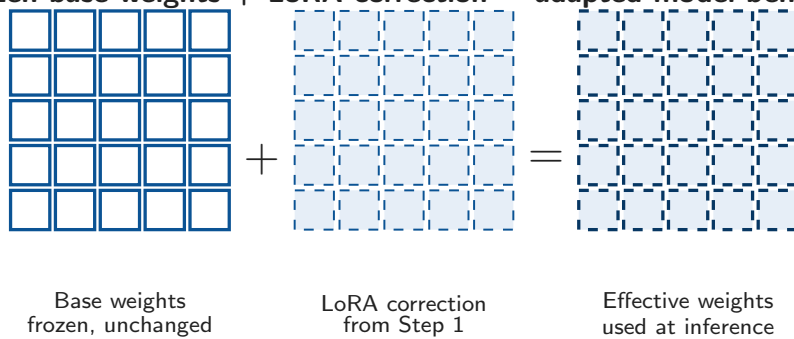
Step 1 – the correction is built from two small tables**Frozen base weights + LoRA correction = adapted model behavior**

Figure 5.1.: LoRA trains two small adapter tables. Their product creates a full-size correction, which is added to the frozen base weights. The original model is not rewritten.

5. Your First LoRA Fine-Tune

Why more than one pass? Each pass only nudges the adapter’s weights a small amount toward the correct answers — like tightening a connection in small, controlled turns instead of one big one, so it doesn’t overshoot. One epoch barely moves the model; this chapter runs 20 so those small nudges can actually accumulate into something. The loss numbers in Step 3 (4.7173 on epoch 1, falling to 0.1201 by epoch 20) are the visible trace of that accumulation.

💡 Engineering Translation: training recall vs. generalization

Training recall is how well the model reproduces the exact examples it trained on — an open-book test where it already saw the answer key. **Generalization** is how well it does on something it never saw — the real test of whether it learned anything transferable. A model can score perfectly on the first and close to zero on the second; Chapter 3 built the held-out check specifically to catch that gap.

5.5. Why LoRA instead of updating the whole model?

A full fine-tune — updating all 1.5 billion parameters — is a reasonable thing to *want*: more of the model can adapt, in principle. It’s a much harder thing to actually run: 1.5 billion trainable parameters means 1.5 billion optimizer states to keep in memory, and a saved checkpoint the same multi-gigabyte size as the base model itself, for every fine-tune you ever run. LoRA trains a small fraction of that — for this chapter’s setup, about 0.14% of the model’s parameters — and saves a corresponding adapter file measured in megabytes, not gigabytes. You’ll see the exact numbers for both in Step 2.

5.6. Implementation

5.6.1. Step 1: Format a training example and mask the prompt out of the loss

5.7. What problem are we solving?

The model shouldn’t be trained to predict the *question* — only the *answer*. This step builds the full chat-formatted text for one example, then marks every token before the assistant’s turn as “don’t compute loss here.”

5.8. Inputs

- The tokenizer loaded in Chapter 1
- One training example (`instruction`, `input`, `output`)

5.9. Expected Output

Two tensors of matching shape: the full token sequence, and a labels tensor where the prompt portion is replaced with -100 (PyTorch’s “ignore this position” marker for loss computation).

```
import torch

def build_training_ids(tokenizer, example: dict) -> tuple[torch.Tensor,
    ↪ torch.Tensor]:
    user_message = f"{example['instruction']}\n{example['input']}"
    prompt_only = tokenizer.apply_chat_template(
        [{"role": "user", "content": user_message}], tokenize=False,
    ↪ add_generation_prompt=True
    )
    full_text = tokenizer.apply_chat_template(
        [
            {"role": "user", "content": user_message},
            {"role": "assistant", "content": example["output"]},
        ],
        tokenize=False,
        add_generation_prompt=False,
    )
    prompt_ids = tokenizer(prompt_only, return_tensors="pt",
    ↪ add_special_tokens=False)["input_ids"]
    full_ids = tokenizer(full_text, return_tensors="pt",
    ↪ add_special_tokens=False)["input_ids"]
    labels = full_ids.clone()
    labels[:, : prompt_ids.shape[1]] = -100
    return full_ids, labels

input_ids, labels = build_training_ids(tokenizer, examples[0])
print(f"Full sequence: {input_ids.shape[1]} tokens, {int((labels[0] ==
    ↪ -100).sum()))} masked (prompt)")
```

Running this exact code prints:

Full sequence: 92 tokens, 71 masked (prompt)

5.10. What just happened?

92 tokens make up the full chat-formatted example; the first 71 — everything through `<|im_start|>assistant` — are masked out of the loss, leaving only the 21 tokens of the actual answer for the model to be trained on. This is the same masking idea Chapter 3’s baseline used in reverse: there, the answer was hidden from the model at inference time; here, everything *except* the answer is hidden from the loss at training time.

5.11. Production Reality

Real fine-tuning pipelines batch many examples together with padding, not one at a time — Chapter 7 covers formatting and chunking training data at scale. This chapter keeps it to one example per step deliberately, so every line of the training loop in Step 3 stays easy to read and reason about.

5.11.1. Step 2: Wrap the base model with a LoRA adapter

5.12. What problem are we solving?

Turn the frozen base model into one with a small, trainable adapter attached — everything else about the model stays exactly as Chapter 1 loaded it.

5.13. Inputs

- The loaded model from Chapter 1
- A LoraConfig naming which layers get an adapter

5.14. Expected Output

A wrapped model, plus a printed count of how many parameters are actually trainable.

```
from peft import LoraConfig, TaskType, get_peft_model

LORA_CONFIG = LoraConfig(
    r=8,
    lora_alpha=16,
    lora_dropout=0.05,
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj"],
    task_type=TaskType.CAUSAL_LM,
)

lora_model = get_peft_model(model, LORA_CONFIG)
lora_model.print_trainable_parameters()
```

Running this exact code prints:

```
trainable params: 2,179,072 || all params: 1,545,893,376 || trainable%: 0.1410
```

5.15. What just happened?

`target_modules` names the specific weight matrices inside each attention layer that get a LoRA adapter attached — `q_proj/k_proj/v_proj/o_proj` are the query, key, value, and output projections every attention layer has. `r=8` sets the adapter’s size (its “rank” — a small number by design). The result: 2,179,072 trainable parameters out of 1,545,893,376 total — 0.141%. Everything else in the model is frozen and will never change during this chapter’s training — including the raw embedding table Chapter 4 measured; BHA’s own coordinate doesn’t move. What changes instead is how the attention layers *use* that unchanged coordinate once text passes through them, which is where this chapter’s improvement actually comes from.

5.16. Production Reality

`target_modules` and `r` are real hyperparameters a production fine-tuning pipeline would sweep, not fix once — different layers targeted, or a larger rank, trade off adapter size against how much the model can actually learn. This chapter’s challenge exercise tries a smaller `target_modules` list and shows the real effect on results.

One more deliberate choice, worth naming directly: this chapter’s training loop runs entirely on CPU, the same as Chapter 1’s default. A GPU or Apple Silicon (MPS) machine can fine-tune this faster, but this book keeps every reader’s numbers reproducible against the same baseline rather than depending on whichever accelerator a given machine happens to have. If you have a GPU and want to try it, `model.to("cuda")` (or `"mps"`) before Step 2, and moving `input_ids/labels` in `build_training_ids` to the same device, is the change — the rest of this chapter’s logic doesn’t depend on which device it runs on.

5.16.1. Step 3: Fine-tune, then measure recall *and* generalization

5.17. What problem are we solving?

Actually train the adapter on Chapter 2’s 16 examples, then answer Carlos’s question with numbers: did the model learn these exact examples, and separately, did anything transfer to a report it never saw?

5.18. Inputs

- The wrapped `lora_model` from Step 2
- Chapter 2’s 16 training examples, and its held-out report’s 2 examples
- Chapter 3’s `run_baseline` harness, reused unchanged

5.19. Expected Output

A loss that falls across 20 epochs, then two before/after score comparisons.

```
def fine_tune(lora_model, tokenizer, examples: list[dict], num_epochs: int = 20)
    ↪ -> None:
    lora_model.train()
    optimizer = torch.optim.AdamW(lora_model.parameters(), lr=2e-4)
    for epoch in range(num_epochs):
        epoch_loss = 0.0
        for example in examples:
            input_ids, labels = build_training_ids(tokenizer, example)
            loss = lora_model(input_ids=input_ids, labels=labels).loss
            loss.backward()
            optimizer.step()
            optimizer.zero_grad()
            epoch_loss += loss.item()
        if (epoch + 1) % 5 == 0 or epoch == 0:
            print(f" epoch {epoch + 1}/{num_epochs}: avg loss {epoch_loss /
                ↪ len(examples):.4f}")
    lora_model.eval()

def score(results: list[dict]) -> tuple[int, int]:
    return sum(r["matched_expected"] for r in results), len(results)

print("Before fine-tuning:")
print(f" Training baseline: {score(run_baseline(model, tokenizer, examples))}")
print(f" Held-out baseline: {score(run_baseline(model, tokenizer,
    ↪ held_out_examples))}")

fine_tune(lora_model, tokenizer, examples)

print("After fine-tuning:")
print(f" Training recall: {score(run_baseline(lora_model, tokenizer,
    ↪ examples))}")
print(f" Held-out generalization: {score(run_baseline(lora_model, tokenizer,
    ↪ held_out_examples))}")
```

Running this exact code prints:

```
Before fine-tuning:
  Training baseline: 0/16
  Held-out baseline: 0/2

epoch 1/20: avg loss 4.7173
```

```
epoch 5/20: avg loss 1.2688
epoch 10/20: avg loss 0.3360
epoch 15/20: avg loss 0.1850
epoch 20/20: avg loss 0.1201
```

After fine-tuning:

Training recall: 13/16

Held-out generalization: 0/2

5.20. What just happened?

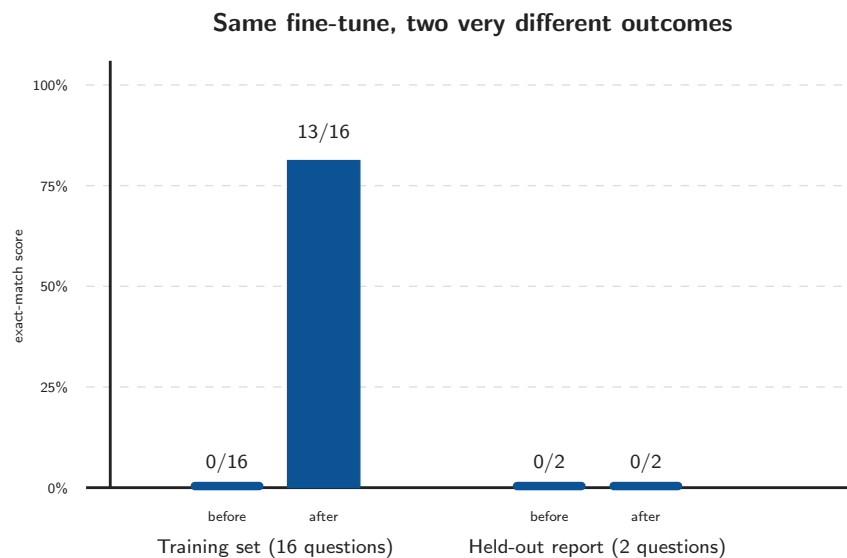


Figure 5.2.: The same fine-tune produces two very different outcomes – training recall jumps from 0/16 to 13/16, while held-out generalization stays flat at 0/2.

Be precise about what this run actually shows: fine-tuning **changed** the model, not simply “improved” it. Training recall jumped from 0/16 to 13/16 — real, measurable learning, not a training artifact. Held-out generalization stayed at 0/2, exactly where it started. Sixteen examples was enough for this small model to memorize sixteen answers; it was not enough to teach it anything that transfers to a report it never saw. That’s not a bug — it’s the honest, predictable result of a training set this size, and precisely why Chapter 2 reserved that report in the first place.

Of the 18 total questions across both checks (16 training + 2 held-out), 13 training answers came back right, which leaves 5 wrong — 3 from the training set, 2 from the held-out report. Looking at *which* answers came back wrong is more revealing than the scores alone. Every one of those 5 is a real answer from a *different* report, borrowed wholesale — never an invented guess:

Question	Expected	Model said	Actually belongs to
Report #3, activity planned	CONTINUE RIGGING U P...	CONTINUE BUILDING CURVE TO 65°	Report #36

5. Your First LoRA Fine-Tune

Question	Expected	Model said	Actually belongs to
Report #36, present ops	BUILDING CURVE SECTION...	PIPE FREE, TRIP OUT OF HOLE...	Report #38
Report #48, present ops	LAYING DOWN COOLING BHA 31	CIRCULATE FOR TEMPERATURE	Report #49
Report #37 (held-out), present ops	RIG UP SCHLUMBERGER...	PIPE FREE, TRIP OUT OF HOLE...	Report #38
Report #37 (held-out), activity planned	RUN UBI LOG...	CONTINUE BUILDING CURVE TO 65°	Report #36

The fine-tuned model didn't get vaguer or more hesitant on questions it couldn't answer — the way the honest, ungrounded base model did back in Chapter 3. It got *more* confident, and confidently wrong, reaching for the nearest memorized answer instead. That's a materially worse failure mode for a production system than "I don't know," and it's the direct consequence of training on too little, too narrow a dataset.

The principle behind all three results together: a tiny fine-tune can teach a model to prefer the *shape* of your archive — its vocabulary, its formatting, the fact that answers are short, well-specific field values — without teaching it reliable judgment over reports it hasn't seen. Shape is real and worth having. It is not the same thing as judgment, and this chapter's numbers are exactly what the difference looks like.

5.21. Practical exercise

Beginner

Try it yourself: run `python code/chapter_05/first_lora_finetune.py` from inside `book/` (it takes about 5 minutes on CPU) and watch the loss fall across the 20 epochs.

You'll know it worked when: you see `Training recall: 13/16` and `Held-out generalization: 0/2` — and you can explain, in one sentence, why those two numbers moving so differently is exactly the point.

5.22. Field notes

 Field notes: wrong, but never randomly wrong

Query: every one of this chapter's 5 wrong answers (3 training misses, both held-out misses), collected as a set.

Result: all 5 are word-for-word answers that belong to *other* real reports in the training set — "PIPE FREE, TRIP OUT OF HOLE FOR BHA INSPECTION" (really report #38's answer) shows up twice, once for report #36's question and once for the held-out report #37's question.

Why: with only 16 examples and 20 epochs of repetition, the model learned a shortcut: *produce something that looks like a report answer from this well*, without reliably learning *which*

answer belongs to *which specific report*. Its confusion isn't noise — it consistently reaches for one of a small set of memorized, real sentences.

Lesson: a fine-tuned model's mistakes are diagnostic. Random gibberish would suggest something broken in training; consistently borrowed real answers point specifically at “not enough distinct training signal to tell reports apart” — exactly what Chapter 6's data quality gate and Chapter 7's richer training examples exist to fix.

5.23. Challenge exercise

Intermediate

Challenge: Step 2 targets all four attention projections (`q_proj`, `k_proj`, `v_proj`, `o_proj`). A more common minimal LoRA setup only targets `q_proj` and `v_proj`. Rerun fine-tuning with that lighter config and see whether training recall changes. A reference solution is at `code/chapter_05/challenge/challenge.py` — on a real run, it trains 1,089,536 parameters (0.0705%, about half of this chapter's setup) and scores 10/16 training recall, down from 13/16.

5.24. Key takeaways

- A LoRA adapter trains a small fraction of a model's parameters (here, 0.141%) while the rest stay frozen — a real fine-tune, not a simulation of one, at a fraction of full fine-tuning's cost.
- Training recall and generalization are different claims that need different evidence. This chapter's model aced the first (0/16 → 13/16) and made zero progress on the second (0/2 → 0/2), on the exact same fine-tune.
- A fine-tuned model's wrong answers are worth reading, not just counting. Every miss here was a real, memorized answer from a *different* report — evidence of a specific, fixable data problem, not random model failure.
- Sixteen examples is enough to prove fine-tuning works mechanically. It is deliberately not enough to teach real operational judgment — that's what Part II's larger, higher-quality training sets are for.

5.25. Repository files

File	Purpose
<code>code/chapter_05/first_lora_finetune.py</code>	Fine-tunes a LoRA adapter on Chapter 2's training examples and reports training/held-out scores
<code>code/chapter_05/challenge/challenge.py</code>	Reference solution: a lighter <code>q_proj/v_proj</code> -only LoRA config
<code>checkpoints/chapter_05_lora/</code>	Generated adapter (not committed — regenerate by running the script)
<code>notebooks/chapter_05_explore.ipynb</code>	Interactive companion notebook

CHECKPOINT — Chapter 5

- ☒ Fine-tuned a LoRA adapter on real training data with a plain PyTorch training loop
- ☒ Measured a real improvement in training recall (0/16 → 13/16)
- ☒ Ran a held-out generalization check for the first time, and found it did *not* improve (0/2 → 0/2)
- ☒ Traced every wrong answer back to a specific, explainable cause rather than shrugging at “the model got it wrong”

WHAT YOU BUILT

first_lora_finetune.py — trains a LoRA adapter on Chapter 2’s 16 examples and saves it to `checkpoints/chapter_05_lora/` (about 8 MB, next to the base model’s ~2.9 GB of weights on disk). Reruns Chapter 3’s exact baseline harness before and after, against both the training set and the held-out report, so the comparison is honest and reproducible.

5.26. What can you do now that you couldn’t do before?

You can fine-tune a real language model on your own data and produce real, measurable evidence of what it learned — and, just as importantly, honest evidence of what it didn’t, instead of assuming a falling loss number means the job is done.

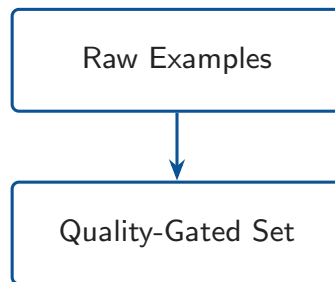
5.27. Suggested next step

Coming up in Chapter 6: this chapter’s fine-tune worked exactly as a 16-example, single-well dataset should: real memorization, no generalization. The obvious next move is more data — but simply pointing this same pipeline at hundreds of reports, without checking them first, would just memorize more inconsistencies faster. Chapter 6 builds a data quality gate: the checks a training set needs to pass *before* it’s trusted at the scale Part II is about to introduce.

Part III.

Part II — Industrialising the System

6. A Data Quality Gate for Training Data



Progress #####..... 6 / 13 · Estimated time: 30–40 min · Difficulty: Beginner

6.1. Learning objectives

By the end of this chapter, you will be able to:

- Run Chapter 2’s field extractor across the full 76-report archive instead of the 10-report sample, and read a real pass/fail rate at that scale
- Explain the difference between a hard failure (a report’s fields can’t be extracted at all — must be excluded) and a soft flag (fields extracted fine, but something about them deserves a human look before training)
- Detect field values duplicated across different reports, and distinguish likely-benign duplicates from ones that need review
- Explain why “extraction succeeded” and “this data is trustworthy to train on” are two different claims

6.2. Operational Problem

Oumy, who built Chapter 2’s small proof of concept from just 10 reports, asks the obvious next question now that Part II is about scaling that same pipeline to the full archive: *“If we just point this extractor at all 76 reports, how do we know we’re not scaling up hidden problems right along with the data?”*

Chapter 2’s extractor either finds both fields on a report or it doesn’t — a binary, visible failure Chapter 2 already handles by skipping the report. What it can’t tell you is whether two *different* reports somehow ended up with the exact same field value, which is a much quieter, easier-to-miss kind of problem. This chapter builds a gate that checks for both, run for real against all 76 reports.

6.3. Example: the same answer, two different reports

Chapter 2 and Chapter 3 both used report #49 — the day `activity_planned` read "FISHING OPERATIONS" — as a running example. Run this chapter's extractor across the *full* archive, not just the 10-report sample, and a report three weeks later turns up something worth a second look:

 Two reports, one identical field value

```
Report #49 (2020-12-07), present_operations: "CIRCULATE FOR TEMPERATURE"  
Report #70 (2020-12-28), present_operations: "CIRCULATE FOR TEMPERATURE"
```

Word-for-word identical, 21 days apart. That could be a completely routine, independently-occurring operation — or a sign something got copy-pasted or misfiled. Chapter 2's extractor has no way to even notice this happened; it only looks at one report at a time.

6.4. Theory

 Engineering Translation: data quality gate

A **quality gate** is a checkpoint data has to pass through before it's trusted for the next stage — like a pre-shift equipment check. It doesn't fix anything by itself; it stops something visibly wrong from silently making it further down the line, and it flags anything borderline for a human to actually look at.

 Engineering Translation: hard fail vs. soft flag

A **hard fail** is unambiguous enough to act on automatically — Chapter 2's "couldn't extract required fields" is one, so that report gets excluded, no judgment call needed. A **soft flag** is something worth a human's attention that isn't automatically wrong — like this chapter's duplicate values. Treating every soft flag as a hard fail would silently delete real, correct data; treating every hard fail as just a suggestion would let broken data through.

6.5. Why "just remove duplicates" falls short

The tempting, simpler rule: any field value that appears on more than one report is suspicious, so drop every report involved. Running that rule on this archive would delete report #6 ("RIGGING UP WITH FRONTIER DRILLING", identical to #5), report #33 ("DRILLING 8 3/4" PRODUCTION HOLE WITH BHA #14", identical to #32), and report #35 ("CORING", identical to #34) — three reports whose "duplicate" value is just the same operation genuinely continuing into the next day, which is completely normal in a real drilling program. This chapter's gate checks *which* reports share a value, not just *whether* they do: report numbers one apart are almost certainly a continued operation; any bigger gap is what actually deserves review.

6.6. Implementation

6.6.1. Step 1: Extract fields from all 76 reports and check completeness

6.7. What problem are we solving?

Confirm Chapter 2's extractor's real pass rate at archive scale, not just on the curated 10-report sample where every quirk was already known.

6.8. Inputs

- Every PDF in `datasets/full_training_set/`
- `extract_text` and `extract_fields` from Chapter 2, reused unchanged

6.9. Expected Output

A status (`ok` or `unrecognized_format`) attached to every report.

```
import sys
from pathlib import Path

BOOK_ROOT = Path(__file__).resolve().parents[2]
sys.path.insert(0, str(BOOK_ROOT / "code" / "chapter_02"))

from build_training_examples import extract_fields, extract_text

FULL_SET_DIR = BOOK_ROOT / "datasets" / "full_training_set"

def build_archive_records(full_dir: Path = FULL_SET_DIR) -> list[dict]:
    records = []
    for pdf_path in sorted(full_dir.glob("*.pdf")):
        text = extract_text(pdf_path)
        fields = extract_fields(text)
        status = "ok" if fields["present_operations"] and
        ↪ fields["activity_planned"] else "unrecognized_format"

        record = dict(fields)
        record["file"] = pdf_path.name
        record["status"] = status
        record["rpt_num"] = int(fields["rpt_num"]) if fields["rpt_num"] else None
        records.append(record)
    return records
```

```
records = build_archive_records()
passed = [r for r in records if r["status"] == "ok"]
failed = [r for r in records if r["status"] != "ok"]
print(f"Extraction: {len(passed)}/{len(records)} reports passed")
print(f"Failed: {[r['file'] for r in failed]}")
```

Running this exact code prints:

```
Extraction: 75/76 reports passed
Failed: ['FORGE-16A-78-32_Completion_003_2021-01-06.pdf']
```

6.10. What just happened?

75 of 76 reports — every drilling report, at full archive scale, not just the 10-report sample — extracted cleanly with Chapter 2’s exact same regex patterns. The one failure is the same completion report Chapter 2 already knew about, using different field labels entirely. That consistency is itself useful information: this archive’s drilling reports really do share one fixed layout, at scale, not just by coincidence in the small sample.

6.11. Production Reality

This archive comes from a single well, a single operator’s report template, over a few months. A multi-well, multi-operator archive would show a much lower pass rate here, with several different field layouts needing their own extractor (Chapter 2’s challenge exercise handles exactly one such case — the completion report). A production quality gate would need a library of layout patterns, not one regex per field.

6.11.1. Step 2: Find field values duplicated across different reports

6.12. What problem are we solving?

A report passing extraction doesn’t mean its *value* is trustworthy on its own — the same exact text on two different reports is invisible to a per-report check and needs comparing across the whole archive at once.

6.13. Inputs

- The `records` list from Step 1

6.14. Expected Output

Every field value that appears on more than one report, with the report numbers involved.

```
from collections import defaultdict

def find_duplicate_values(records: list[dict], field_name: str) -> dict[str,
↪ list[int]]:
    by_value = defaultdict(list)
    for record in records:
        if record["status"] == "ok" and record[field_name]:
            by_value[record[field_name]].append(record["rpt_num"])
    return {value: sorted(nums) for value, nums in by_value.items() if len(nums)
↪ > 1}

for field_name in ("present_operations", "activity_planned"):
    print(f"Duplicate {field_name} values:")
    for value, nums in find_duplicate_values(records, field_name).items():
        print(f"  reports {nums}: {value!r}")
```

Running this exact code prints:

```
Duplicate present_operations values:
reports [5, 6]: 'RIGGING UP WITH FRONTIER DRILLING'
reports [32, 33]: 'DRILLING 8 3/4" PRODUCTION HOLE WITH BHA #14'
reports [34, 35]: 'CORING'
reports [49, 70]: 'CIRCULATE FOR TEMPERATURE'
Duplicate activity_planned values:
reports [32, 33]: 'DRILL AHEAD WITH BHA #14'
reports [47, 59]: 'DRILL AHEAD'
```

6.15. What just happened?

Six duplicate groups turned up across both fields — four where the report numbers are exactly one apart (an operation continuing into the next day), and two where they aren't: #49/#70 (21 days apart) and #47/#59 (12 days apart). Nothing here is auto-corrected or removed yet — this step's only job is to surface every duplicate so Step 3 can decide which ones actually need a human's attention.

6.16. Production Reality

This only catches *exact* text matches. A near-duplicate — the same operation described with slightly different wording or a typo fixed — would slip through entirely. Catching that reliably needs a fuzzy or embedding-based similarity check (Chapter 4’s tools are directly applicable here), which is real, valuable future work this simple version deliberately leaves out.

6.16.1. Step 3: Classify duplicates and produce the final gate report

6.17. What problem are we solving?

Turn the raw duplicate list into the actual verdict this chapter opened with: which duplicates are almost certainly fine, and which ones need a person to look before this data gets trained on.

6.18. Inputs

- The duplicate groups from Step 2

6.19. Expected Output

Each duplicate group labeled `consecutive` or `needs_review`, plus a final summary.

```
def classify_duplicate(rpt_nums: list[int]) -> str:
    gaps = [b - a for a, b in zip(rpt_nums, rpt_nums[1:])]
    return "consecutive" if all(gap == 1 for gap in gaps) else "needs_review"

def run_quality_gate(full_dir: Path = FULL_SET_DIR) -> dict:
    records = build_archive_records(full_dir)
    ok_records = [r for r in records if r["status"] == "ok"]
    failed_records = [r for r in records if r["status"] != "ok"]

    duplicates = {}
    for field_name in ("present_operations", "activity_planned"):
        for value, nums in find_duplicate_values(records, field_name).items():
            duplicates[(field_name, value)] = classify_duplicate(nums), nums

    return {"total": len(records), "passed": len(ok_records), "failed":
        ↳ failed_records, "duplicates": duplicates}

report = run_quality_gate()
needs_review = {k: v for k, v in report["duplicates"].items() if v[0] ==
    ↳ "needs_review"}
```

```
consecutive = {k: v for k, v in report["duplicates"].items() if v[0] ==
    ↪ "consecutive"}
print(f"Extraction: {report['passed']}/{report['total']} passed")
print(f"Duplicates: {len(consecutive)} consecutive (no action needed),
    ↪ {len(needs_review)} need review")
for (field_name, value), (_, nums) in needs_review.items():
    print(f"    {field_name}, reports {nums}: {value!r}")
```

Running this exact code prints:

```
Extraction: 75/76 passed
Duplicates: 4 consecutive (no action needed), 2 need review
    present_operations, reports [49, 70]: 'CIRCULATE FOR TEMPERATURE'
    activity_planned, reports [47, 59]: 'DRILL AHEAD'
```

6.20. What just happened?

The gate's final verdict on this archive: 1 hard fail (excluded), 4 duplicate groups explained away as normal continued operations, and 2 flagged for a human to actually check before Chapter 7 formats this archive into training examples at scale. Neither flagged case is automatically rejected — "DRILL AHEAD" in particular is common enough phrasing that it may well be an honest coincidence, not an error. The gate's job is to surface it, not to guess.

6.21. Practical exercise

Beginner

Try it yourself: run `python code/chapter_06/data_quality_gate.py` from inside `book/`, then open reports #49 and #70 from `datasets/full_training_set/` and compare their PRESENT OPERATIONS: lines by eye.

You'll know it worked when: the script prints 75/76 passed, 4 consecutive, 2 need review, and you can say in one sentence whether you think #49/#70's match looks like a real coincidence or a data-entry duplicate.

6.22. Field notes

 Field notes: the fishing report's echo, three weeks later

Query: every duplicate `present_operations` value across the full 76-report archive.

Result: report #49 — 2020-12-07, the same day `activity_planned` read "FISHING OPERATIONS" in Chapter 2 and Chapter 3's running example — has `present_operations` "CIRCULATE FOR TEMPERATURE". So does report #70, dated 2020-12-28.

Why: the archive shows report #49 sitting inside the packers-fail-to-fishing sequence (#48–#50) already discussed in earlier chapters, while #70 is three weeks later with no narrative connection to it at all — a different point in the drilling program entirely. Circulating to stabilize wellbore temperature is a genuinely common, unremarkable operation; there’s no reason it couldn’t happen twice, unrelated, three weeks apart.

Lesson: a duplicate flag is a question, not a verdict. This one most likely resolves to “yes, coincidence, both real” on inspection — but the gate’s job is only to make sure that judgment call gets made by a person instead of silently vanishing into 75 reports’ worth of training data.

6.23. Challenge exercise

Intermediate

Challenge: add a third check: report dates should strictly increase alongside report numbers. A scrambled or misfiled archive (two reports swapped, one from the wrong well mixed in) would show up here as an out-of-order or repeated date. A reference solution is at `code/chapter_06/challenge/challenge.py` — on a real run against this book’s archive, it reports `75 reports checked, 0 issues`: every date in this archive is exactly where it should be.

6.24. Key takeaways

- A quality gate answers two different questions: which reports fail outright (hard fail, exclude automatically), and which ones are fine but worth a second look (soft flag, surface for a human).
- A naive “remove all duplicates” rule would have deleted three perfectly legitimate continued-operation reports in this archive. The *pattern* of the duplicate (consecutive vs. not) matters as much as the duplicate itself.
- “Extraction succeeded” and “this data is trustworthy to train on” are different claims — this chapter’s gate is what actually checks the second one, at the same 76-report scale Chapter 7 onward builds on.
- Even a real, verified archive like this one — genuinely clean on chronological order, with 0 issues found — is worth checking explicitly rather than assumed clean by default.

6.25. Repository files

File	Purpose
<code>code/chapter_06/data_quality_gate.py</code>	Runs extraction and duplicate-value checks across the full 76-report archive
<code>code/chapter_06/challenge/challenge.py</code>	Reference solution: a chronological-order check
<code>notebooks/chapter_06_explore.ipynb</code>	Interactive companion notebook

CHECKPOINT — Chapter 6

- ☒ Ran Chapter 2's extractor across the full 76-report archive and measured a real pass rate at scale
- ☒ Found every field value duplicated across different reports, and distinguished likely-benign duplicates from ones needing review
- ☒ Produced a final gate report separating hard fails from soft flags, instead of treating every anomaly the same way

WHAT YOU BUILT

data_quality_gate.py — checks the full 76-report archive for extraction failures and cross-report duplicate values, and separates what needs automatic exclusion from what needs a human decision. It leaves this archive with 75 reports that passed extraction, 2 of which still carry an open `needs_review` flag — not yet resolved either way. Chapter 7 builds its formatted, chunked training set on top of those 75 extractable reports, and should treat the 2 flags as a decision still owed, not a detail already handled.

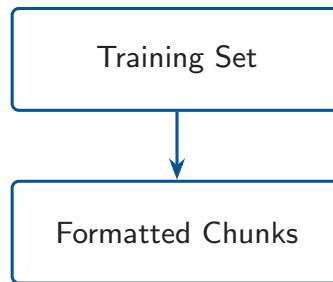
6.26. What can you do now that you couldn't do before?

You can run a real data quality check across an entire report archive before training on it, and you can tell the difference between a problem worth automatically excluding and one worth a human's judgment — instead of either training on everything blindly or manually skimming 76 reports by eye.

6.27. Suggested next step

Coming up in Chapter 7: this chapter surfaced, for all 76 reports, which ones extract cleanly and which field values deserve a second look — it didn't unilaterally decide the 2 `needs_review` cases, and it didn't turn anything into a training example yet. Those 2 flags should be accepted or excluded, by a person, before Chapter 7 runs. Chapter 2's approach, one regex extraction per report, worked fine for 10 reports typed out one at a time, but Chapter 7 needs to format and chunk 75 reports' worth of training data at scale, without re-doing that work by hand.

7. Formatting and Chunking a Training Set at Scale



Progress #####..... 7 / 13 · Estimated time: 30–40 min · Difficulty: Intermediate

7.1. Learning objectives

By the end of this chapter, you will be able to:

- Extract a report’s full operational log — not just Chapter 2’s two summary fields — into one structured entry per logged time window
- Turn each entry into its own training example, producing many more, richer examples per report than Chapter 2’s fixed two
- Split (“chunk”) any entry too long for one training example into several, without truncating or corrupting a single word
- Detect and filter a real PDF-extraction artifact that would otherwise put corrupted text straight into training data
- Build a full training set from every quality-gated, non-held-out report at once, instead of running Chapter 2’s script by hand 74 times

7.2. Operational Problem

Farid, the production engineer, looks at Chapter 2’s output and asks the obvious question: *“Two lines per report? These reports have a whole shift-by-shift log in them. Are we just throwing the rest away?”*

He’s right to ask. Every report in this archive has a **TIME BREAKDOWN** section underneath the two fields Chapter 2 used — a real, multi-entry operational log, one row per logged time window, each with its own narrative. Chapter 2’s regex never looked at it. This chapter does, across all 74 reports Chapter 6’s gate and Chapter 2’s held-out reservation leave available for training.

7.3. Example: one report, ten training examples instead of two

Report `Drilling_038` — the stuck-pipe day from Chapter 1 — has a `TIME BREAKDOWN` section with 10 separate logged time windows. One of them is the exact moment Chapter 1 quoted:

 One `TIME BREAKDOWN` entry, turned into a training example

```
{
  "instruction": "What happened on this well during this time window?",
  "input": "Well: FORGE 16A [78]-32 | Report #38 | Date: 2020-11-26 | Time:
    ↪ 23:30-04:00",
  "output": "Drill From 6,360' to 6,507', (147') Total , 32.6' feet per hour.
    ↪ WOB 20 TO 35k, Rotary 50, Torque 6,500, SPP 3200 -3400 GPM 560, DIFF
    ↪ 200 - 300psi Production Drilling Trips During the slide lost tool face
    ↪ and became assembly became stuck"
}
```

Chapter 2 turned this same report into 2 examples. This chapter turns it into 10 — one per logged time window, each with real narrative detail Chapter 2 never touched.

7.4. Theory

 Engineering Translation: chunking

Chunking means splitting one long piece of text into several smaller pieces that each fit inside one training example, instead of one oversized example or a silently truncated one. Done well, it's invisible — like cutting a long pipe run into standard joint lengths; the pipe still does its job, it just now comes in handled pieces.

 Engineering Translation: word-boundary splitting

Splitting text at a **word boundary** means never cutting in the middle of a word — only at the spaces between them. A naive character-count split (`text[:300]`) doesn't know or care where words end; a word-boundary split builds each chunk up word by word, stopping just before the next word would push it over the limit.

7.5. Why not just use the whole report as one training example?

The simplest alternative to timeline-by-timeline chunking: skip parsing entirely, and use each report's full raw text as a single training example. It's tempting, and it's worse for two concrete reasons. First, most of a report's raw text is tabular data — casing, mud properties, pump specs, BHA components — that `pdfplumber` flattens into dense, confusing lines with no narrative value at all (Step 1's exact code already skips past all of it). Second, and more importantly: one example

per report means the model only ever sees 74 examples total, each one an undifferentiated wall of text mixing several unrelated time windows together. This chapter’s approach — one example per logged time window — produces 669 focused, individually meaningful examples from those same 74 reports, each one answering a specific question about a specific window of time.

7.6. Implementation

7.6.1. Step 1: Parse the TIME BREAKDOWN section into time-window entries

7.7. What problem are we solving?

Turn each report’s TIME BREAKDOWN table — several rows of FROM TO HRS ... OPERATIONS, with narrative text that can wrap across multiple lines — into a clean list of (from_time, to_time, text) entries.

7.8. Inputs

- One report’s full extracted text (from Chapter 2’s `extract_text`)

7.9. Expected Output

A list of dictionaries, one per logged time window.

```
import re

TIMELINE_SECTION_PATTERN = re.compile(
    r"TIME BREAKDOWN\n(?:FROM TO HRS PHASE CODE NPT OPERATIONS\n)?(.*?)\nTOTAL\n"
    ↪   HRS", re.DOTALL
)

TIMELINE_ENTRY_PATTERN =
    ↪   re.compile(r"^\d{2}:\d{2}\s+(\d{2}:\d{2})\s+([\d.]+)\s+(.*)", re.MULTILINE)

def parse_timeline_entries(text: str) -> list[dict]:
    section = TIMELINE_SECTION_PATTERN.search(text)
    if not section:
        return []
    body = section.group(1)
    matches = list(TIMELINE_ENTRY_PATTERN.finditer(body))

    entries = []
    for i, match in enumerate(matches):
        start = match.end()
```

7. Formatting and Chunking a Training Set at Scale

```
        end = matches[i + 1].start() if i + 1 < len(matches) else len(body)
        raw_text = f"{match.group(4)} {body[start:end].strip()}"
        entries.append({"from_time": match.group(1), "to_time": match.group(2),
↪      "text": re.sub(r"\s+", " ", raw_text)})
    return entries

entries = parse_timeline_entries(text)
print(f"{len(entries)} timeline entries found")
print(entries[-2])
```

Running this exact code against `Drilling_038` prints:

```
10 timeline entries found
{'from_time': '23:30', 'to_time': '04:00', 'text': "Drill From 6,360' to 6,507', (147') Total
3400 GPM 560, DIFF 200 - 300psi Production Drilling Trips During the slide lost tool face and l
```

7.10. What just happened?

The regex finds every line that starts with two times and an hour count, then treats everything up to the *next* such line as that entry's narrative — which is what lets a real, multi-line entry (like this one) get captured whole, instead of stopping at the first line break. PHASE/CODE labels ("Production Drilling Trips" above) stay mixed into the narrative text rather than parsed into their own fields — `pdfplumber`'s column-flattening makes them genuinely ambiguous to separate reliably, and this chapter doesn't pretend otherwise.

7.11. Production Reality

This regex assumes the same fixed TIME BREAKDOWN layout every report in this archive happens to share — the same kind of assumption Chapter 2 made about its two fields, and the same kind of assumption that breaks the moment a different operator's report template shows up.

7.11.1. Step 2: Split long entries into word-boundary chunks

7.12. What problem are we solving?

Some entries are short ("Intermediate Drilling Lubricate Rig Rig Service"); others run past 900 characters. Every training example in this book stays a manageable size, so anything over the limit needs to become more than one example — without cutting through the middle of a word.

7.13. Inputs

- One entry's `text`, from Step 1
- A maximum length, in characters

7.14. Expected Output

A list of one or more chunks, each within the limit, that concatenate back into the original text exactly.

```
def chunk_text(text: str, max_chars: int = 300) -> list[str]:
    words = text.split()
    chunks, current, current_len = [], [], 0
    for word in words:
        added_len = len(word) + (1 if current else 0)
        if current and current_len + added_len > max_chars:
            chunks.append(" ".join(current))
            current, current_len = [], 0
            added_len = len(word)
        current.append(word)
        current_len += added_len
    if current:
        chunks.append(" ".join(current))
    return chunks

long_entry = entries[-2]["text"] # the stuck-pipe entry, 237 characters -- under
    ↪ the limit here
print(chunk_text(long_entry, max_chars=100))
```

Running this exact code prints:

```
["Drill From 6,360' to 6,507', (147') Total , 32.6' feet per hour. WOB 20 TO 35k, Rotary 50, T
'6,500, SPP 3200 -3400 GPM 560, DIFF 200 - 300psi Production Drilling Trips During the slide
'tool face and became assembly became stuck']
```

7.15. What just happened?

`chunk_text` builds each piece up one word at a time, and only closes a chunk off *before* adding a word would push it past `max_chars` — never mid-word. At this chapter's real setting, `max_chars=300`, this particular entry (237 characters) doesn't need splitting at all; forced down to 100 here just to show the mechanism working on real text.

7.16. Production Reality

Splitting at word boundaries keeps words intact, but it can still split a sentence — or a single measurement like "WOB 20 TO 35k" — across two chunks, same as it did above. A more careful chunker would prefer to break at sentence or clause boundaries first, only falling back to a mid-sentence word split when a single sentence itself exceeds the limit. This chapter's simpler version is honest about that trade-off rather than hiding it.

7.16.1. Step 3: Build the full training set, respecting Chapter 6 and Chapter 2

7.17. What problem are we solving?

Combine Steps 1 and 2 across every report — but only the reports Chapter 6's gate actually passed, and never Chapter 2's held-out report — and catch one more real problem along the way: a PDF-extraction artifact that would otherwise put corrupted text straight into training data.

7.18. Inputs

- Chapter 6's `build_archive_records()`, for status and file list
- Chapter 2's `HELD_OUT_REPORT`

7.19. Expected Output

A `.jsonl` file with one example per line, plus counts of how many examples, chunks, and filtered artifacts resulted.

```
def contains_cid_artifact(text: str) -> bool:
    return "(cid:" in text # pdfplumber's stand-in for an undecoded glyph

def build_timeline_examples_for_report(pdf_path):
    text = extract_text(pdf_path)
    fields = extract_fields(text)
    context = f"Well: {fields['well_name']} | Report #{fields['rpt_num']} | Date:
    ↪ {normalize_date(fields['rpt_date'])}"

    examples, artifacts_filtered = [], 0
    for entry in parse_timeline_entries(text):
        chunks = chunk_text(entry["text"])
        for i, chunk in enumerate(chunks):
            if contains_cid_artifact(chunk):
                artifacts_filtered += 1
                continue
```

```

        time_label = f"Time: {entry['from_time']}--{entry['to_time']}"
        if len(chunks) > 1:
            time_label += f" (part {i + 1} of {len(chunks)})"
        examples.append({"instruction": "What happened on this well during
↪ this time window?", "input": f"{context} | {time_label}", "output": chunk})
    return examples, artifacts_filtered

def build_training_set_at_scale(full_dir=FULL_SET_DIR):
    records = build_archive_records(full_dir)
    examples, skipped, artifacts_filtered = [], [], 0
    for record in records:
        if record["file"] == HELD_OUT_REPORT:
            continue
        if record["status"] != "ok":
            skipped.append(record["file"])
            continue
        report_examples, report_artifacts =
↪ build_timeline_examples_for_report(full_dir / record["file"])
        examples.extend(report_examples)
        artifacts_filtered += report_artifacts
    return examples, skipped, artifacts_filtered

examples, skipped, artifacts_filtered = build_training_set_at_scale()
chunked = sum(1 for e in examples if "part" in e["input"])
print(f"Built {len(examples)} training examples (excluding {len(skipped)}
↪ unrecognized, 1 held out)")
print(f"{chunked} examples are chunks of a longer timeline entry")
print(f"{artifacts_filtered} chunk(s) filtered out for containing undecoded-glyph
↪ artifacts")

```

Running this exact code prints:

```

Built 669 training examples (excluding 1 unrecognized, 1 held out)
125 examples are chunks of a longer timeline entry
3 chunk(s) filtered out for containing undecoded-glyph artifacts

```

7.20. What just happened?

74 reports (76 total, minus the 1 unrecognized-format report Chapter 6 already flagged, minus the 1 held-out report Chapter 5 fine-tuned against) produced 669 training examples — more than 40 times Chapter 2’s 16, from the same archive, just by reading further into each report. 125 of those needed chunking; 3 chunks got filtered entirely for containing (cid:N) markers, which turned up in exactly the report with this chapter’s single longest entry (see Field notes).

7.21. Practical exercise

Beginner

Try it yourself: run `python code/chapter_07/format_training_chunks.py` from inside book /, then open `datasets/training_examples/full_training_examples.jsonl` and search it for `R eport #49` — the fishing-operations report from Chapters 2, 3, and 6.

You'll know it worked when: the script prints `Built 669 training examples`, and report #49 has more than 2 entries in the file — direct evidence this chapter captured more of that day than Chapter 2 ever did.

7.22. Field notes

 **Field notes:** a garbled sub-table hiding inside the longest entry in the archive

Query: the single longest `TIME BREAKDOWN` entry across all 76 reports, before any chunking.

Result: report #21's 18:00–23:00 entry, 991 characters — nearly 4x the archive's median entry length — contains a step-rate-test sub-table that `pdfplumber` couldn't decode: `"Time(cid:0)WOB(K) Rotary (cid:0)Differential (cid:0)ROP 19:40 (cid:0)30 40(cid:0)(cid:0)95..."`.

Why: the source PDF embeds a small results table inside this one entry — rows of timestamped WOB/rotary/differential/ROP readings from a step-rate test. Most of this archive's tables extract cleanly (Chapter 6's gate flagged only 1 of 76 reports as an outright extraction failure, and that was a different report using a different field layout entirely); this one specific embedded table apparently uses characters `pdfplumber` can't map to real text, leaving `(cid: N)` placeholders instead — a narrower, quieter kind of extraction problem than a whole report failing outright.

Lesson: the longest, most information-dense entry in the whole archive is exactly the one most likely to contain something a simple extractor mishandles — and exactly the one chunking was already about to split into pieces anyway. Checking for corruption at the chunk level, right where the chunking already happens, catches it for free.

7.23. Challenge exercise

Intermediate

Challenge: this chapter's `chunk_text` uses `max_chars=300`. Rerun the full pipeline with a smaller limit, 150, and see how many more (shorter) examples result. A reference solution is at `code/chapter_07/challenge/challenge.py` — on a real run, it produces 855 examples with 405 chunks, up from 669/125 at `max_chars=300`.

7.24. Key takeaways

- A report has more usable training signal in it than its two headline summary fields — this archive’s `TIME BREAKDOWN` log alone turned 74 reports into 669 examples, over 40x Chapter 2’s count from the same data.
- Chunking at word boundaries keeps every word intact, at the cost of sometimes splitting a sentence or a single measurement across two examples — a real, named trade-off, not a hidden one.
- A quality check doesn’t happen once. Chapter 6’s gate checked the two summary fields; going one level deeper into each report’s narrative surfaced a real extraction artifact Chapter 6 never had a chance to see.
- Every training-set-building step in this book — Chapter 2’s, Chapter 6’s held-out and gate logic, and now this chapter’s — composes directly: this chapter imports and reuses them rather than duplicating their logic.

7.25. Repository files

File	Purpose
<code>code/chapter_07/format_training_chunks.py</code>	Parses each report’s timeline into chunked training examples at archive scale
<code>code/chapter_07/challenge/challenge.py</code>	Reference solution: a smaller <code>max_chars</code> chunk size
<code>datasets/training_examples/full_training_examples.jsonl</code>	Generated output (not committed — regenerate by running the script)
<code>notebooks/chapter_07_explore.ipynb</code>	Interactive companion notebook

CHECKPOINT — Chapter 7

- ☒ Parsed a report’s full operational timeline, not just its two summary fields
- ☒ Split an overlong entry into multiple training examples at word boundaries, without cutting through a word
- ☒ Caught a real PDF-extraction artifact before it reached training data, instead of after
- ☒ Built a 669-example training set from 74 reports, respecting both Chapter 6’s quality gate and Chapter 2’s held-out reservation

WHAT YOU BUILT

`format_training_chunks.py` — turns every quality-gated, non-held-out report’s full operational timeline into chunked training examples, saved to `datasets/training_examples/full_training_examples.jsonl`. Chapter 8 fine-tunes on this 669-example set — the “at scale” successor to Chapter 5’s 16-example proof of concept.

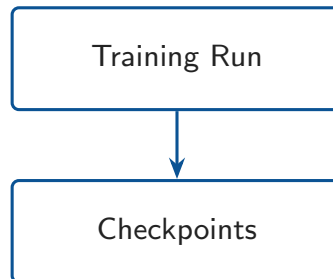
7.26. What can you do now that you couldn't do before?

You can turn an entire report archive's narrative content — not just its two most obvious fields — into a properly sized, artifact-checked training set automatically, instead of hand-extracting a couple of lines per report the way Chapter 2 did.

7.27. Suggested next step

Coming up in Chapter 8: 669 examples is a very different training run than Chapter 5's 16 — long enough that a crash partway through would be a real loss, and large enough that “did it actually learn anything” needs more than eyeballing a loss number once at the end. Chapter 8 fine-tunes on this chapter's full training set with checkpointing (so a failure doesn't mean starting over) and experiment tracking (so training behavior is visible while it's happening, not just after).

8. Fine-Tuning at Scale with Checkpointing and Experiment Tracking



Progress #####. 8 / 13 · Estimated time: 45–60 min · Difficulty: Intermediate

8.1. Learning objectives

By the end of this chapter, you will be able to:

- Fine-tune on Chapter 7’s full 669-example training set instead of Chapter 5’s 16, and explain why that changes what “just rerun it” actually costs
- Save a checkpoint every epoch, and log training metrics to a plain CSV/JSONL file you can open in any spreadsheet
- Simulate a real crash and resume training purely from a saved checkpoint on disk — not from anything still sitting in memory
- Explain why this chapter’s held-out check still splits by report, not by individual training example, and what would go wrong if it didn’t

8.2. Operational Problem

Carlos, the completions engineer who asked in Chapter 5 whether fine-tuning teaches the operation or just memorizes flashcards, has a more practical worry now that the training set has grown from 16 examples to 669: *“If this run takes half an hour and the laptop goes to sleep at minute twenty-five, did we just lose the whole thing?”*

Chapter 5’s script had no answer to that question, because at 16 examples and 5 minutes, it never needed one. At this scale, it does.

8.3. Example: a run directory after training

 What a run directory looks like on disk

```
checkpoints/chapter_08/run_20260815_142824/  
  checkpoint_1/  
    adapter_config.json  
    adapter_model.safetensors  
  checkpoint_2/  
    adapter_config.json  
    adapter_model.safetensors  
  checkpoint_3/  
    adapter_config.json  
    adapter_model.safetensors  
metrics.csv  
metrics.jsonl
```

Every one of those files exists independent of the training process — delete the terminal, close the laptop, come back tomorrow, and everything needed to resume or inspect this run is still sitting on disk exactly as it was.

8.4. Theory

 Engineering Translation: checkpoint

A **checkpoint** is a saved snapshot of the adapter’s weights at one point in training — like a documented casing point during a long run, somewhere you could pick back up from if something interrupted the job, instead of starting over from the surface. This chapter saves one after every **epoch** — Chapter 5’s term for one full pass through the entire training set, all 669 examples once.

 Engineering Translation: experiment tracking

Experiment tracking means recording what happened during training — loss, epoch, timestamp, which checkpoint — as it happens, not just remembering the final number. A CSV file with one row per epoch is experiment tracking; it doesn’t need to be a dashboard to count.

8.5. Why not just rerun Chapter 5’s script on more data?

Chapter 5’s `first_lora_finetune.py` would technically work if you just handed it Chapter 7’s 669 examples instead of 16 — nothing about the LoRA mechanics changes. What breaks is everything *around* it. Chapter 5’s run took about 5 minutes; at 669 examples, one epoch alone takes several minutes, and this chapter trains for multiple epochs. A script with no checkpointing that crashes

90% of the way through a 30-minute run loses all 90%. A script that only prints a final loss number gives you nothing to look at if training goes wrong at epoch 2, not epoch 3. Scale doesn't just mean "more of the same" — it means a training run becomes something you have to be able to interrupt, inspect, and resume.

8.6. Implementation

8.6.1. Step 1: Load Chapter 7's training set and hold out by report

8.7. What problem are we solving?

Get the same 669 training examples and the same single held-out report Chapter 2 reserved — reusing both, not rebuilding either.

8.8. Inputs

- Chapter 7's `build_training_set_at_scale()`
- Chapter 2's `HELD_OUT_REPORT` (`Drilling_037`)

8.9. Expected Output

669 training examples; the held-out report's own timeline entries, built separately.

```
import sys
from pathlib import Path

BOOK_ROOT = Path(__file__).resolve().parents[2]
sys.path.insert(0, str(BOOK_ROOT / "code" / "chapter_02"))
sys.path.insert(0, str(BOOK_ROOT / "code" / "chapter_06"))
sys.path.insert(0, str(BOOK_ROOT / "code" / "chapter_07"))

from build_training_examples import HELD_OUT_REPORT
from data_quality_gate import FULL_SET_DIR
from format_training_chunks import build_timeline_examples_for_report,
    ↪ build_training_set_at_scale

examples, skipped, artifacts_filtered = build_training_set_at_scale()
held_out_examples, _ = build_timeline_examples_for_report(FULL_SET_DIR /
    ↪ HELD_OUT_REPORT)
print(f"{len(examples)} training examples, {len(held_out_examples)} held-out
    ↪ examples")
```

Running this exact code prints:

669 training examples, 8 held-out examples

8.10. What just happened?

Nothing about the split changed from Chapter 2 — `Drilling_037` is excluded from `examples` at the report level, the same way it always has been, and its own timeline entries are built separately for `held_out_examples`. That’s deliberate: if even one chunk of report #37 had ended up in `examples` while others stayed in `held_out_examples`, the held-out check would be partly grading the model on a report it *did* train on — a leak Chapter 7’s per-report exclusion prevents by construction, not by carefully remembering to check.

8.11. Production Reality

A single held-out report gives 8 examples to check against — informative, but a small sample. Chapter 11 builds a proper evaluation harness over more held-out data; this chapter’s job is getting checkpointing and tracking right, not maximizing eval statistical power yet.

8.11.1. Step 2: Checkpoint every epoch, log every epoch

8.12. What problem are we solving?

Save the adapter’s weights after every epoch, and record what happened during that epoch, so a long run leaves behind both recoverable state and a visible history — not just whatever’s still in memory when it finishes (or doesn’t).

8.13. Inputs

- A LoRA-wrapped model (Chapter 5’s `build_lora_model`)
- The training examples and held-out examples from Step 1

8.14. Expected Output

One `checkpoint_N/` directory and one new row in `metrics.csv` per epoch.

```
import csv
import json
import time

import torch

from baseline_prompting import run_baseline
```

```

from first_lora_finetune import build_training_ids, score

LEARNING_RATE = 2e-4

def save_checkpoint(lora_model, run_dir, epoch):
    checkpoint_dir = run_dir / f"checkpoint_{epoch}"
    lora_model.save_pretrained(checkpoint_dir)
    return checkpoint_dir

class MetricsLogger:
    FIELDS = ["step", "epoch", "train_loss", "held_out_exact_match",
↪  "checkpoint"]

    def __init__(self, run_dir):
        self.csv_path = run_dir / "metrics.csv"
        self.jsonl_path = run_dir / "metrics.jsonl"
        self._rows = []

    def log(self, **kwargs):
        self._rows.append({field: kwargs.get(field, "") for field in
↪  self.FIELDS})
        self._write()

    def _write(self):
        with self.csv_path.open("w", newline="", encoding="utf-8") as f:
            writer = csv.DictWriter(f, fieldnames=self.FIELDS)
            writer.writeheader()
            writer.writerows(self._rows)
        with self.jsonl_path.open("w", encoding="utf-8") as f:
            for row in self._rows:
                f.write(json.dumps(row) + "\n")

def fine_tune_epoch(lora_model, tokenizer, examples, optimizer):
    lora_model.train()
    epoch_loss = 0.0
    for example in examples:
        input_ids, labels = build_training_ids(tokenizer, example)
        loss = lora_model(input_ids=input_ids, labels=labels).loss
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()
        epoch_loss += loss.item()
    lora_model.eval()
    return epoch_loss / len(examples)

```

8. Fine-Tuning at Scale with Checkpointing and Experiment Tracking

```
def train_with_checkpoints(lora_model, tokenizer, examples, held_out_examples,
    ↪ run_dir, logger, num_epochs, start_epoch=0):
    optimizer = torch.optim.AdamW(lora_model.parameters(), lr=LEARNING_RATE)
    for i in range(num_epochs):
        epoch = start_epoch + i + 1
        avg_loss = fine_tune_epoch(lora_model, tokenizer, examples, optimizer)
        checkpoint_dir = save_checkpoint(lora_model, run_dir, epoch)
        matched, total = score(run_baseline(lora_model, tokenizer,
    ↪ held_out_examples))
        logger.log(step=epoch * len(examples), epoch=epoch,
    ↪ train_loss=round(avg_loss, 4), held_out_exact_match=f"{matched}/{total}",
    ↪ checkpoint=checkpoint_dir.name)
        print(f" epoch {epoch}: avg loss {avg_loss:.4f}, held-out
    ↪ {matched}/{total} -> {checkpoint_dir.name}")
```

Running two epochs against the real training set prints:

```
epoch 1: avg loss 2.7550, held-out 0/8 -> checkpoint_1
epoch 2: avg loss 2.1636, held-out 0/8 -> checkpoint_2
```

8.15. What just happened?

Every epoch now leaves two things behind on disk: a `checkpoint_N/` directory holding that epoch's adapter weights, and a new row in `metrics.csv` recording the loss and held-out score at that point. Neither depends on the training process still being alive — both are just files, readable by anything, including a spreadsheet.

8.16. Production Reality

A production setup might stream these same rows to TensorBoard, Weights & Biases, MLflow, or an internal dashboard instead of a flat file — any of those add real value at bigger scale (live plots, comparing many runs, alerting). A CSV is the right starting point here: it's transparent, it's portable, and it needs nothing installed beyond Python's own `csv` module to read or write.

8.16.1. Step 3: Simulate a crash, then resume from disk alone

8.17. What problem are we solving?

Prove the checkpoints from Step 2 are actually sufficient to recover from — not just files that happen to exist, but files a genuinely fresh process can resume real training from.

8.18. Inputs

- A checkpoint directory from Step 2
- A freshly (re-)loaded base model — not the one already in memory

8.19. Expected Output

Training resumes from the saved epoch, and the loss continues falling from where it left off rather than restarting from scratch.

```
from peft import PeftModel

def load_checkpoint(model, checkpoint_dir):
    return PeftModel.from_pretrained(model, checkpoint_dir, is_trainable=True)

del model, lora_model
model, tokenizer = load_model_and_tokenizer(MODEL_NAME) # a genuinely fresh load
lora_model = load_checkpoint(model, run_dir / "checkpoint_2")

train_with_checkpoints(lora_model, tokenizer, examples, held_out_examples,
    ↪ run_dir, logger, num_epochs=1, start_epoch=2)

sample_after = score(run_baseline(lora_model, tokenizer, training_sample))
held_out_after = score(run_baseline(lora_model, tokenizer, held_out_examples))
print(f"Training sample recall (50 examples):
    ↪ {sample_after[0]}/{sample_after[1]}")
print(f"Held-out generalization: {held_out_after[0]}/{held_out_after[1]}")
```

Running this exact code prints:

Resuming training, epoch 3...

epoch 3: avg loss 1.8211, held-out 0/8 -> checkpoint_3

Training sample recall (50 examples): 0/50

Held-out generalization: 0/8

8.20. What just happened?

`del model, lora_model` followed by loading the base model again from scratch is this chapter's stand-in for a real crash and restart — nothing about the previous training process survives except what got written to `checkpoint_2/`. `PeftModel.from_pretrained` reads the adapter weights back from that directory onto the fresh model, and training continues from there. The loss keeps falling

8. Fine-Tuning at Scale with Checkpointing and Experiment Tracking

after the resume — 2.1636 → 1.8211 — not jumping back up to where it started, direct evidence the resume picked up real progress, not just a fresh start with an optimistic label.

The recall numbers are worth sitting with, though, because they’re not what Chapter 5 trained you to expect. Chapter 5’s 16-example run went from 0/16 to 13/16 training recall in 20 epochs. This chapter’s 669-example run, after 3 epochs, is still 0/50 on its training sample — loss fell by more than a third, and exact-match recall hasn’t moved at all. Both results are real; they’re not in tension. Chapter 5’s tiny, repetitive dataset let the model start reproducing exact answers quickly. This chapter’s dataset is over 40 times larger and far more varied — three passes through it is proportionally much less repetition per example than twenty passes through sixteen examples was. The loss curve says the model is learning something real about this archive’s language and structure; it just hasn’t reached the point of reproducing exact wording yet. That’s a genuinely different, and genuinely useful, thing to have measured — a loss number alone would have told you training was “working” without telling you it wasn’t yet memorizing, and exact-match alone would have told you nothing was happening at all when something clearly was.

8.21. Production Reality

This chapter’s resume restores the adapter’s *weights* from disk, but not the optimizer’s internal state (Adam’s per-parameter momentum estimates) — that state lived only in memory and is gone after the simulated crash, same as it would be after a real one. A production resume would also checkpoint and restore optimizer state for a more seamless continuation; this simpler version accepts a small, honest discontinuity in exchange for not doubling every checkpoint’s size and complexity.

8.22. Practical exercise

Beginner

Try it yourself: run `python code/chapter_08/finetune_at_scale.py` from inside `book/` (it takes about 30-35 minutes on CPU), then open the newly created `checkpoints/chapter_08/run_{timestamp}/metrics.csv` in a spreadsheet application and watch `train_loss` fall epoch over epoch.

You’ll know it worked when: you can find 3 `checkpoint_N/` directories on disk, `metrics.csv` has 3 rows, and you can point to exactly which row corresponds to the epoch that ran *after* the simulated crash.

8.23. Field notes

⚠ Field notes: the model learned the accent before the facts

Query: three different training-sample questions, each about a different report and time window, checked against `checkpoint_3`.

Result:

Report / window	Expected	Model said
#2, 06:00–08:00	"No Activity"	"Production Casing Run C sg & Cement Rig up casin g"
#4, 07:30–17:00 (part 3)	"...Rig floor is 50% rig ged up..."	"Production Casing Run C sg & Cement Rig up casin g"
#7, 06:00–07:00	"Rig Move In Rig Up & Te ar Down No activity"	"Production Casing Run C sg & Cement Rig up cemen t"

Why: all three questions are about genuinely different reports and time windows, with genuinely different correct answers — but the model gives back almost the identical phrase each time, just barely varied. "Production Casing", "Rig up", "Cement" are real, common terms across this archive's PHASE/CODE labels; after 3 epochs, the model has learned that this *kind* of question gets answered with *that kind* of phrase, without yet learning which specific phrase belongs to which specific report.

Lesson: fine-tuning at this stage doesn't fail randomly — it learns the domain's accent before it learns the domain's facts. That's a genuinely different failure mode than Chapter 5's "confidently borrows a different report's real answer," and it's exactly the kind of detail a single final loss number would hide completely.

8.24. Challenge exercise

Intermediate

Challenge: prove a saved checkpoint is genuinely self-sufficient — reload the base model completely fresh, load `checkpoint_1` from the most recent run with no other state carried over, and confirm its held-out score matches what `metrics.csv` logged for that exact epoch. A reference solution is at `code/chapter_08/challenge/challenge.py` — on a real run, it reloads a completely fresh model, loads `checkpoint_1` with no other state carried over, recomputes 0/8 on the held-out report, and confirms that matches `metrics.csv`'s logged value for epoch 1 exactly.

8.25. Key takeaways

- Scale changes what a training script needs to survive, not just how long it takes to run. A crash 25 minutes into a 30-minute run with no checkpointing costs you the whole 25 minutes.
- A checkpoint is only as good as its ability to resume training on a genuinely fresh process — this chapter proved that directly by deleting the in-memory model and reloading from disk before resuming.
- Experiment tracking doesn't require a dashboard. A CSV with one row per epoch — step, loss, held-out score, checkpoint name — is transparent, portable, and already useful.
- Falling loss and rising exact-match recall are different signals, and they don't have to move together: this chapter's loss fell by more than a third across 3 epochs while training-sample recall stayed at 0/50. Watching only one of them would have hidden real information the other one caught.
- The held-out split still happens at the report level, exactly as it has since Chapter 2. At this scale that matters more, not less: more training examples means more chances for a sloppier split to leak a few of the wrong ones across the boundary.

8.26. Repository files

File	Purpose
<code>code/chapter_08/finetune_at_scale.py</code>	Fine-tunes on Chapter 7's full training set with checkpointing and CSV/JSONL metrics logging
<code>code/chapter_08/challenge/challenge.py</code>	Reference solution: verifies a checkpoint reproduces its own logged score
<code>checkpoints/chapter_08/run_<timestamp>/</code>	Generated checkpoints and metrics (not committed — regenerate by running the script)
<code>notebooks/chapter_08_explore.ipynb</code>	Interactive companion notebook

CHECKPOINT — Chapter 8

- ☒ Fine-tuned on Chapter 7's full 669-example training set, not Chapter 5's small proof-of-concept sample
- ☒ Saved a checkpoint and logged metrics every epoch, to files readable without the training process still running
- ☒ Simulated a real crash and resumed training from a checkpoint on disk alone, and confirmed the loss kept falling, not restarting
- ☒ Kept the held-out split at the report level throughout, at this larger scale

WHAT YOU BUILT

finetune_at_scale.py — fine-tunes on Chapter 7's 669-example training set with per-epoch checkpointing to `checkpoints/chapter_08/run_<timestamp>/` and metrics logged to `metrics.csv/metrics.jsonl`, and survives a simulated mid-training crash by resuming purely from a saved checkpoint. Chapter 9 builds on this fine-tuned model directly.

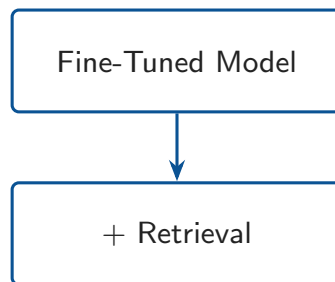
8.27. What can you do now that you couldn't do before?

You can run a fine-tuning job long enough that a crash partway through would actually matter, and trust that it can pick back up from exactly where it left off — with a plain, inspectable record of what happened at every step along the way, not just a final number.

8.28. Suggested next step

Coming up in Chapter 9: this fine-tuned model has now learned this archive's language, formatting, and shorthand at real scale — but it still can't cite which report an answer came from, and Chapter 3 already showed fine-tuning alone isn't the right tool for exact, source-grounded fact lookup. Chapter 9 combines this chapter's fine-tuned model with retrieval, so an answer can be both fluent in the domain *and* traceable back to a specific report.

9. Hybrid System: Combining Fine-Tuning with Retrieval



Progress #####... 9 / 13 · Estimated time: 45–60 min · Difficulty: Advanced

9.1. Learning objectives

By the end of this chapter, you will be able to:

- Build a real keyword-based retrieval index over the full report archive, and explain why it covers every report, held-out one included
- Explain, with a real comparison, why dense sentence embeddings — Chapter 4’s own tool — aren’t the right retrieval method for this domain, and what to use instead
- Combine retrieval with Chapter 8’s fine-tuned model so an answer comes with a citation: which report, which time window
- Explain why a retrieval query and the fine-tuned model’s generation prompt need to be built differently, not reused as the same string
- Recognize, from a real run, that finding the right source and using it faithfully are two different problems — and that this chapter only solves the first one

9.2. Operational Problem

Sarah, the intervention engineer, asks the question Chapter 3 predicted she eventually would: “*The fine-tuned model sounds like it knows our operation now. If I ask it what happened on a specific day, can I actually trust the answer — or do I still have to go find the PDF myself to check?*”

Chapter 3 already gave the honest answer in the abstract: fine-tuning alone is a weak tool for citing a source. Chapter 8 measured exactly how weak — 0/50 exact-match on its own training sample. This chapter builds the other half of the system Chapter 3 promised: a real retrieval layer that can

9. Hybrid System: Combining Fine-Tuning with Retrieval

find the report a fact actually came from, and hand it to the fine-tuned model so the answer can be grounded in something citable, not just fluent.

9.3. Example: the same question, with and without retrieval

i Report #37 – the report Chapter 8’s model never trained on

Without retrieval:

Production Casing Run Csg & Cement Rig up casing

With retrieval:

Trip out of hole with BHA #18. Stop at 5,800' and circulate to cool hole and tools.

Sources: [{'report_num': 37, 'from_time': '20:30', 'to_time': '21:30'}, ...]

The first answer is the same generic, style-only pattern Chapter 8’s Field Notes already caught — fluent, plausible, and wrong for this specific report. The second is close to word-for-word what report #37 actually says at that exact time window, a report Chapter 8’s model never saw during training. The only thing that changed between them is that the second one was handed the right three sentences from the right report before it answered.

9.4. Theory

💡 Engineering Translation: retrieval / RAG

Retrieval means searching a collection of documents for the ones most relevant to a question, without needing a language model at all — like a well-indexed filing cabinet. **RAG** (retrieval-augmented generation) means handing what you found to a language model as extra context before it answers, so the answer can be grounded in specific, real text instead of only in what the model happened to memorize.

💡 Engineering Translation: BM25 keyword search

BM25 is a scoring method for keyword search: it ranks documents by how many of the query’s words they contain, weighted so rare, specific words ("fishing", "Schlumberger") count for more than common ones ("the", "drilling"). It has no idea what any of these words *mean* — it’s just counting overlap, carefully. `rank_bm25`, already one of this book’s dependencies, implements it in a few lines.

9.5. Why not dense embeddings?

Chapter 4 already found that this archive’s vocabulary defeats general-purpose sentence embeddings at the single-term level — BOP scored *lower* against its own meaning than against "birthday party". The same problem shows up again here, at the whole-sentence level. The query "lost tool face and became stuck during the slide" — close to a direct paraphrase of report #38’s actual text — should retrieve that report immediately. With `sentence-transformers`, it doesn’t: report #38’s real entry ranks **8th** out of 677 chunks. With BM25, the same query ranks it **1st**, by a wide margin over every other chunk in the archive. For text this dense with exact technical phrasing, keyword overlap turns out to be a stronger signal than semantic similarity from a general-purpose embedding model. This chapter uses BM25 for exactly that reason — not because keyword search is more sophisticated, but because it’s what the evidence actually supports here.

9.6. Implementation

9.6.1. Step 1: Build a retrieval corpus from every report

9.7. What problem are we solving?

Get every timeline chunk from every report Chapter 6’s gate passed — including the held-out report. Retrieval isn’t training: it should be able to find facts about any report, whether or not the fine-tuned model ever saw it.

9.8. Inputs

- Chapter 6’s `build_archive_records()`
- Chapter 7’s `parse_timeline_entries, chunk_text, contains_cid_artifact`

9.9. Expected Output

Every quality-gated report’s chunks, in one flat list — more than Chapter 7’s training set, because this one keeps the held-out report in.

```
import sys
from pathlib import Path

BOOK_ROOT = Path(__file__).resolve().parents[2]
sys.path.insert(0, str(BOOK_ROOT / "code" / "chapter_02"))
sys.path.insert(0, str(BOOK_ROOT / "code" / "chapter_06"))
sys.path.insert(0, str(BOOK_ROOT / "code" / "chapter_07"))

from build_training_examples import extract_text
```

9. Hybrid System: Combining Fine-Tuning with Retrieval

```
from data_quality_gate import build_archive_records, FULL_SET_DIR
from format_training_chunks import chunk_text, contains_cid_artifact,
↪ parse_timeline_entries

def build_retrieval_corpus(full_dir=FULL_SET_DIR):
    records = build_archive_records(full_dir)
    corpus = []
    for record in records:
        if record["status"] != "ok":
            continue
        text = extract_text(full_dir / record["file"])
        for entry in parse_timeline_entries(text):
            for chunk in chunk_text(entry["text"]):
                if contains_cid_artifact(chunk):
                    continue
                corpus.append({"report_num": record["rpt_num"], "from_time":
↪ entry["from_time"], "to_time": entry["to_time"], "text": chunk})
    return corpus

corpus = build_retrieval_corpus()
print(f"{len(corpus)} retrievable chunks across {len({c['report_num'] for c in
↪ corpus})} reports")
```

Running this exact code prints:

```
677 retrievable chunks across 75 reports
```

9.10. What just happened?

677 chunks, $677 - 669 = 8$ more than Chapter 7’s training set — the held-out report’s own 8 chunks, present here and nowhere in training. Same hard exclusion as always (the 1 unrecognized-format report), same reuse of Chapter 7’s parsing and artifact-filtering — the only difference from Chapter 7’s `build_training_set_at_scale()` is which one report this function chooses to keep.

9.11. Production Reality

Every source report here is public DOE data, so indexing it all is uncomplicated. A production system over confidential well data would need access control baked into retrieval itself — not just “can this user query the model,” but “can this specific chunk even be returned to this specific user” — which this simple version doesn’t attempt.

9.11.1. Step 2: Index with BM25 and retrieve

9.12. What problem are we solving?

Turn the flat chunk list into something searchable, and get back the most relevant chunks for a real question.

9.13. Inputs

- The corpus from Step 1
- A query string

9.14. Expected Output

The top-k chunks, ranked by BM25 score, each labeled with its source report and time window.

```
import numpy as np
from rank_bm25 import BM25Okapi

def build_bm25_index(corpus):
    return BM25Okapi([chunk["text"].lower().split() for chunk in corpus])

def retrieve(query, corpus, bm25, k=3):
    scores = bm25.get_scores(query.lower().split())
    top_indices = np.argsort(-scores)[:k]
    return [{"corpus[i]", "score": float(scores[i])} for i in top_indices]

bm25 = build_bm25_index(corpus)
for r in retrieve("trip out of hole stop at 5800 circulate cool hole and tools",
    ↪ corpus, bm25, k=3):
    print(f" [{r['score']:.2f}] Report #{r['report_num']}
    ↪ {r['from_time']}-{r['to_time']}: {r['text'][:70]}")
```

Running this exact code prints:

```
[18.07] Report #37 20:30-21:30: Production Drilling Trips Trip out of hole with BHA #18. Stop
[14.98] Report #28 01:30-04:30: Make up Ulterra U616M 6 PDC bit Stop and circulate at 3,000 f
[14.13] Report #39 21:00-22:00: Production Drilling Cond Mud & Circ Circulate hole to cool d
```

9.15. What just happened?

The top result is report #37's actual chunk, by a wide score margin over the next-best match — and report #37 is the one report Chapter 8's fine-tuned model never trained on. Retrieval doesn't need the model to have seen something to find it; it only needs the something to be in the corpus, which Step 1 made sure it was.

9.16. Production Reality

677 chunks fit entirely in memory, so scoring every one of them against a query, every time, is instant. A production archive with millions of documents needs a real search index (Elasticsearch, or a proper BM25 implementation with precomputed term indexes) instead of scoring everything from scratch per query — a different implementation of the same idea, not a different idea.

9.16.1. Step 3: Combine retrieval with the fine-tuned model

9.17. What problem are we solving?

Get a grounded, cited answer out of Chapter 8's fine-tuned model — and avoid a real mistake this chapter's own first draft made: using the model's templated input as the retrieval query.

9.18. Inputs

- Chapter 8's fine-tuned checkpoint
- A retrieval **query** — a real information need, not the templated **instruction/input_context** the fine-tuned model expects

9.19. Expected Output

An answer, plus the specific report(s) and time window(s) it was grounded in.

```
def build_grounded_prompt(instruction, input_context, retrieved_chunks):
    sources = "\n".join(f"[Report #{c['report_num']}],"
    ↪ {c['from_time']}-{c['to_time']}: {c['text']}" for c in retrieved_chunks)
    return f"{instruction}\n{input_context}\n\nRelevant report
    ↪ excerpts:\n{sources}"

def answer_with_retrieval(model, tokenizer, instruction, input_context, query,
    ↪ corpus, bm25, k=3):
    retrieved = retrieve(query, corpus, bm25, k=k)
    prompt = build_grounded_prompt(instruction, input_context, retrieved)
```

```

answer = generate_reply(model, tokenizer, prompt, max_new_tokens=60)
return {"answer": answer, "sources": [{"report_num": c["report_num"],
    ↪ "from_time": c["from_time"], "to_time": c["to_time"]} for c in
    ↪ retrieved]}

instruction = "What happened on this well during this time window?"
input_context = "Well: FORGE 16A [78]-32 | Report #37 | Time: 20:30-21:30"

# The mistake: using the templated input itself as the retrieval query
print("Metadata-only query:")
for r in retrieve(f"{instruction} {input_context}", corpus, bm25, k=3):
    print(f"  Report #{r['report_num']}")

# The fix: a query describing the actual information need
query = "trip out of hole stop at 5800 circulate cool hole and tools"
result = answer_with_retrieval(lora_model, tokenizer, instruction, input_context,
    ↪ query, corpus, bm25)
print(f"\nGrounded answer: {result['answer']}")
print(f"Sources: {result['sources']}")

```

Running this exact code prints:

Metadata-only query:

```

Report #25
Report #54
Report #54

```

```

Grounded answer: Trip out of hole with BHA #18. Stop at 5,800' and circulate to cool hole and
Sources: [{'report_num': 37, 'from_time': '20:30', 'to_time': '21:30'}, {'report_num': 28, 'fr

```

9.20. What just happened?

The metadata-only query — the templated `instruction` and `input_context` the fine-tuned model was trained on — retrieves the *wrong* reports entirely: #25 and #54, nowhere near report #37. That string is nearly identical across hundreds of chunks ("What happened on this well during this time window? Well: ... | Report #NN | Time: ...") — it carries metadata, not the actual question. The real query finds report #37 immediately, and the grounded answer that comes back is close to word-for-word what that report actually says. Retrieval and generation need different inputs here: what finds the right document and what the fine-tuned model expects to see are not the same string, and treating them as interchangeable is the single easiest way for a hybrid system like this one to quietly fail.

9.21. Production Reality

This chapter hand-writes each `query` to demonstrate the mechanism. A real system needs the query to come from somewhere — the user’s own natural-language question, most directly, possibly rewritten or expanded first. Turning “what a user actually typed” into “a good retrieval query” is its own real subfield (query reformulation); this chapter assumes a reasonable query is already in hand.

9.22. Practical exercise

Beginner

Try it yourself: run `python code/chapter_09/hybrid_rag_finetune.py` from inside `book/` (needs Chapter 8’s checkpoint — run that chapter’s script first if you haven’t), and compare the “without retrieval” and “with retrieval” answers for each of the 4 test cases.

You’ll know it worked when: the script prints `Retrieval found the correct report in the top 3 for 4/4 test queries`, and you can point to at least one case where the grounded answer contains a specific number or name that only appears in the retrieved report text.

9.23. Field notes

 Field notes: finding the source and using it faithfully are different problems

Query: all 4 of this chapter’s test cases, comparing what the retrieved source actually said against what the grounded answer said.

Result:

Report	Retrieval	Grounded answer matches source?
#37 (held-out)	found	Yes — near word-for-word
#38 (stuck pipe)	found	No — answer ignores the retrieved text entirely
#21 (step rate test)	found	No — answer is unrelated to the retrieved content
#49 (fishing)	found	Partial — topically right, not the specific detail

Why: retrieval succeeded in all 4 cases — the correct report was always in the top 3. But handing the model the right text doesn’t force it to use that text. For #38, the model was given the exact “lost tool face and became stuck” passage and generated an unrelated answer about tripping in hole anyway. Nothing in this chapter’s setup makes the model’s answer *accountable* to the context it was given.

Lesson: a hybrid system’s citation only means “this is what was retrieved,” not “this is what the answer is actually based on.” Those can silently diverge, and this chapter has no way to catch it when they do — checking whether an answer is actually consistent with its cited source is exactly Chapter 10’s job, not this one’s.

9.24. Challenge exercise

Intermediate

Challenge: rerun this chapter’s 4 test queries using dense sentence-transformer embeddings (Chapter 4’s tool) instead of BM25, and compare retrieval accuracy directly. A reference solution is at `code/chapter_09/challenge/challenge.py` — on a real run, it scores 3/4 (missing exactly the #38 stuck-pipe query, the same one that ranked 8th in this chapter’s “Why not dense embeddings?” section), against BM25’s 4/4 on the same queries.

9.25. Key takeaways

- Retrieval should cover everything, held-out data included — a fine-tuned model’s blind spots are exactly where retrieval needs to work, not where it’s optional.
- For this archive’s dense technical phrasing, keyword search (BM25) beat general-purpose dense embeddings at finding the right source — the same domain-vocabulary gap Chapter 4 found at the single-term level shows up again at the sentence level. Four test queries is a real result, not a toy one, but it’s still too small to call this a statistically powered comparison — read it as consistent with BM25’s known strength on exact-jargon matching, not as proof dense embeddings lose on this archive in general.
- A retrieval query and a fine-tuned model’s expected input are different things. Reusing one as the other — the templated metadata string as a search query — silently retrieves the wrong documents.
- Successful retrieval doesn’t guarantee grounded generation. This chapter measured both separately and found them genuinely different: retrieval succeeded 4/4 times; the model faithfully used what it was given only some of that time.

9.26. Repository files

9. Hybrid System: Combining Fine-Tuning with Retrieval

File	Purpose
<code>code/chapter_09/hybrid_rag_finetune.py</code>	Builds a BM25 retrieval index over the full archive and combines it with Chapter 8's fine-tuned model
<code>code/chapter_09/challenge/challenge.py</code>	Reference solution: dense-embedding retrieval, compared directly against BM25
<code>notebooks/chapter_09_explore.ipynb</code>	Interactive companion notebook

🔥 CHECKPOINT — Chapter 9

- ☑ Built a retrieval corpus covering every quality-gated report, including the one the fine-tuned model never trained on
- ☑ Chose BM25 over dense embeddings based on a real, measured comparison, not by default
- ☑ Combined retrieval with Chapter 8's fine-tuned model to produce answers with citations
- ☑ Measured retrieval accuracy and generation faithfulness separately, and found they're not the same thing

💡 WHAT YOU BUILT

hybrid_rag_finetune.py — a BM25 retrieval index over all 75 quality-gated reports (held-out report included), combined with Chapter 8's fine-tuned checkpoint to produce cited, source-grounded answers. Chapter 10 picks up exactly where this chapter's Field Notes left off: checking whether an answer is actually consistent with what it cites.

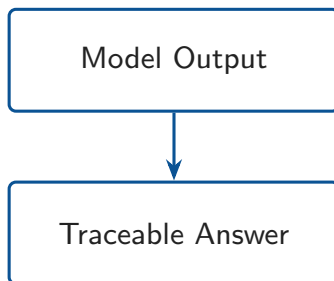
9.27. What can you do now that you couldn't do before?

You can answer a question using both a model that's learned your domain's language and a search over your actual reports, together — and you can point to exactly which report and time window an answer came from, instead of asking the model to simply be right.

9.28. Suggested next step

Coming up in Chapter 10: this chapter's Field Notes found a real gap — retrieval can find the right source and the model can still write an answer that ignores it. Chapter 10 builds the check this chapter is missing: verifying an answer is actually consistent with the report it cites, and flagging it when it isn't, instead of trusting a citation just because one was attached.

10. Traceable Outputs and Hallucination Mitigation



Progress #####... 10 / 13 · Estimated time: 30–40 min · Difficulty: Intermediate

10.1. Learning objectives

By the end of this chapter, you will be able to:

- Explain why a citation Chapter 9 attaches to an answer means “this is what was retrieved,” not “this is what the answer is actually based on” — and why those two claims can silently diverge
- Build a real, inspectable faithfulness check that scores an answer against each individual source chunk it was shown
- Run that check against Chapter 9’s own 4 test cases and find a real case where an answer looks grounded but is actually traceable to the *wrong* report
- Explain the limits of a word-overlap faithfulness check: what kind of mistake it can and can’t catch

10.2. Operational Problem

Oumy, the drilling engineer who built Chapter 6’s quality gate to catch bad training data before anything gets trained on it, looks at Chapter 9’s hybrid system and asks the natural next question: “*We check the data going in. How do we check the answer coming out?*”

Chapter 9’s own Field Notes already found the specific problem this chapter has to fix: across its 4 test cases, retrieval found the correct report every time, but the fine-tuned model’s answer was only actually consistent with that report’s text once. A citation Chapter 9 attaches to an answer only ever meant “this is what retrieval handed the model” — nothing in Chapter 9 checks whether the model’s answer *used* it.

10.3. Example: a citation that's true, and an answer that isn't

Ask Chapter 9's system about report #21's step rate test, and it retrieves report #21's real text in its top 3 — the actual test, timestamped and logged:

i Report #21, 15:30-16:00 – the report actually asked about

```
Step Rate Test: 5K increments, 5 mins each
set 800 gpm, 40 rotary WOB
Depth Differential
20K 4,946 110
25K 4,946 130
...
```

The grounded answer that comes back is fluent and specific: *“Test choke manifold at 5000psi.”* Every word of it is real — but it isn't from report #21. It's lifted from a different report retrieval also handed the model in the same prompt:

i Report #27, 06:00-11:00 – retrieved alongside #21, and where this answer actually came from

```
PJSM, pre job safety meeting
Production Drilling
Test B.O.P.
Test BQPE 250 psi low 5000 psi high, test casing 1500 psi,
test choke manifold 5000 psi
```

Report #27 is a blowout-preventer pressure test — a completely different operation. The model didn't invent “5000psi” out of thin air, so this isn't the obviously-wrong-and-fluent failure Chapter 3 first showed. It's worse: a real number, from a real report, attached to the wrong question. Chapter 9's citation for this answer would have listed report #21 right alongside #27 — both were retrieved — with no way to tell which one the answer actually came from.

10.4. Theory

💡 Engineering Translation: faithfulness / grounding

An answer is **grounded** in a source if it's actually built from that source's content — not just handed the source alongside an unrelated guess. **Faithfulness** is how well an answer matches what a specific source actually says. A hybrid system can be grounded in general (it retrieved real reports) while being unfaithful in the specific answer it gave (it didn't actually use them) — that gap is exactly what report #21's example above shows.

 Engineering Translation: hallucination

A **hallucination** is a fluent, confident model output that isn't actually supported by real evidence. Chapter 1 showed the obvious kind — confident nonsense with nothing behind it. Report #21's example above is the harder kind: a hallucination made of entirely real words, borrowed from the wrong place. Both are hallucinations; only one of them is easy to spot by reading the answer alone.

10.5. Why trusting a citation isn't enough

The tempting shortcut: if an answer comes with a citation — a report number and time window retrieval actually found — treat it as trustworthy. Report #21's example shows exactly why that fails: the citation Chapter 9 would print alongside this answer is *true* (report #21 really was retrieved, really is relevant to the question) and the *answer* is still wrong. A citation only proves retrieval did its job. It proves nothing about whether the generation step actually used the report it names, instead of a different one sitting in the same prompt. Catching that gap needs something that looks at the answer's own words, not just the list of sources it was handed.

10.6. Implementation

10.6.1. Step 1: Score an answer against a single source

10.7. What problem are we solving?

Turn “does this answer match this source?” into a real, checkable number — something simple enough to read and trust, not a second opaque model making an opaque judgment call.

10.8. Inputs

- One generated **answer** (a string)
- One candidate **source_text** (a string) — a single retrieved chunk

10.9. Expected Output

A score from 0.0 to 1.0: the fraction of the answer's meaningful words that also appear somewhere in the source.

```

import re

# Words that appear in every prompt's own instruction/input template
# ("What happened on this well during this time window?"), not because
# the retrieved text was actually used. Counting them as "supported"
# would make every answer look more grounded than it is.
STOPWORDS = {
    "a", "an", "and", "are", "as", "at", "be", "been", "but", "by", "during",
    "for", "from", "happened", "in", "is", "it", "its", "of", "on", "or",
    "report", "the", "this", "time", "to", "was", "well", "were", "what",
    "which", "window", "with",
}

def content_words(text: str) -> set[str]:
    words = re.findall(r"[a-z0-9']+", text.lower())
    return {w for w in words if w not in STOPWORDS and len(w) > 2}

def faithfulness_score(answer: str, source_text: str) -> float:
    answer_words = content_words(answer)
    if not answer_words:
        return 0.0
    return len(answer_words & content_words(source_text)) / len(answer_words)

print(faithfulness_score("Test choke manifold at 5000psi", "test choke manifold
↪ 5000 psi"))
print(faithfulness_score("Test choke manifold at 5000psi", "Step Rate Test: 5K
↪ increments, 5 mins each, 800 gpm, 40 rotary WOB"))

```

Running this exact code prints:

```

0.75
0.25

```

10.10. What just happened?

The exact same generated answer scores 0.75 against report #27's text and only 0.25 against report #21's — the report it was actually supposed to be about. The number alone already tells the story Chapter 9 could only tell by reading both answers by eye: this answer is faithful to *something*, just not the thing it was asked about.

0.75, not 1.0, is itself worth noticing: the answer's own "5000psi" (no space) doesn't match report #27's "5000 psi" (two words) as a token, even though a person reading both would call them the same number. A word-overlap check is exactly this literal — Production Reality below has more on what that costs.

10.11. Production Reality

This is a plain word-overlap check, not a semantic one. It will miss a genuine paraphrase (an answer that says the same thing in different words scores low even when it's correct), and it can't detect a directional or factual inversion buried inside otherwise-matching words — Field Notes below shows a real case of exactly that. A production system checking higher-stakes claims would pair this with a real entailment or fact-verification model; this chapter's version is deliberately simple enough to read every line of and trust what it's actually measuring.

! This is a teaching tool, not a safety check

`faithfulness_score` exists so you can see, in code you can read start to finish, what “grounded” does and doesn't guarantee — it is not validated for deciding whether an automated answer is safe to act on. A field engineer relying on a model-generated summary for a shift handover cannot treat a high score here as a stand-in for reading the source report. Field Notes below is the proof: a genuinely dangerous reversal — the report said *trip in hole*, the answer said *trip out of hole* — still scored 0.80, comfortably above this chapter's own 0.5 threshold. Use this score as a prompt for a qualified person to go check the source, never as an unsupervised pass/fail on a live well operation.

10.11.1. Step 2: Keep only the sources an answer is actually traceable to

10.12. What problem are we solving?

Chapter 9's `answer_with_retrieval` cites every chunk retrieval handed the model, whether or not the answer used it. Turn that into a citation list that only survives if the answer is actually faithful to it.

10.13. Inputs

- The loaded, fine-tuned model and tokenizer from Chapter 8/9
- `instruction`, `input_context`, and a retrieval query (Chapter 9)
- The retrieval corpus and bm25 index (Chapter 9)

10.14. Expected Output

The generated answer, every retrieved chunk's faithfulness score, and a `verified_sources` list containing only the chunks that actually cross a faithfulness threshold.

```
def answer_with_traceable_sources(model, tokenizer, instruction, input_context,
    ↪ query, corpus, bm25, k=3, threshold=0.5) -> dict:
    retrieved = retrieve(query, corpus, bm25, k=k)
    prompt = build_grounded_prompt(instruction, input_context, retrieved)
```

```
answer = generate_reply(model, tokenizer, prompt, max_new_tokens=60)

scored = sorted(
    (**chunk, "faithfulness": faithfulness_score(answer, chunk["text"]))
    ↪ for chunk in retrieved),
    key=lambda c: c["faithfulness"],
    reverse=True,
)
verified_sources = [c for c in scored if c["faithfulness"] >= threshold]

return {"answer": answer, "retrieved": scored, "verified_sources":
    ↪ verified_sources, "grounded": len(verified_sources) > 0}
```

10.15. What just happened?

`retrieve` and `build_grounded_prompt` are Chapter 9's own functions, reused unchanged — this step doesn't touch how retrieval or generation work, only what happens to the result afterward. Note the naming shift: Chapter 9's `build_grounded_prompt` just means “built from retrieved text,” true whether or not the model actually used it; this step's own `"grounded"` field is the stricter claim — true only once at least one source has passed the faithfulness check below. Same word, two different chapters, deliberately different strictness; watch for which one is being made wherever you see it. `threshold=0.5` is a plain default, not tuned to any one answer; Step 3 shows what it does across all 4 of Chapter 9's real test cases.

10.16. Production Reality

`k` (how many chunks get retrieved and shown to the model) directly controls how much unrelated material sits in the same prompt as the right answer, and report #21's example is exactly what happens when that goes wrong: the correct report was in the prompt, and the model still reached for a different one next to it. A production system would tune `k` against real faithfulness measurements like this one, not just against retrieval accuracy.

10.16.1. Step 3: Run the check against Chapter 9's real test cases

10.17. What problem are we solving?

Apply Step 2's function to all 4 of Chapter 9's test cases and see, with real numbers, how often “grounded” actually means “grounded in the right report.”

10.18. Inputs

- Chapter 9's TEST_CASES (4 real report questions) and INSTRUCTION
- The fine-tuned checkpoint, retrieval corpus, and BM25 index

10.19. Expected Output

Per-case answers, faithfulness scores against every retrieved chunk, and which sources (if any) survive as verified citations.

```
for label, input_context, query, target_report in TEST_CASES:
    result = answer_with_traceable_sources(lora_model, tokenizer, INSTRUCTION,
    ↪ input_context, query, corpus, bm25)
    print(f"{label}: {result['answer']}")
    print(f"   grounded: {result['grounded']}")
    for chunk in result["retrieved"]:
        mark = "verified" if chunk["faithfulness"] >= 0.5 else "not used"
        print(f"       [{mark:>8}] Report #{chunk['report_num']} (faithfulness
        ↪ {chunk['faithfulness']:.2f})")
```

Running this exact code prints:

Report #37 (held-out): Trip out of hole with BHA #18. Stop at 5,800' and circulate to cool hole
grounded: True

```
[verified] Report #37 (faithfulness 1.00)
[verified] Report #28 (faithfulness 0.78)
[not used] Report #39 (faithfulness 0.44)
```

Report #38 (stuck pipe): Production Drilling Trips Trip in hole with drill pipe from 10,490' to
grounded: False

```
[not used] Report #38 (faithfulness 0.44)
[not used] Report #60 (faithfulness 0.22)
[not used] Report #61 (faithfulness 0.11)
```

Report #21 (step rate test): Test choke manifold at 5000psi
grounded: True

```
[verified] Report #27 (faithfulness 0.75)
[not used] Report #21 (faithfulness 0.25)
[not used] Report #17 (faithfulness 0.25)
```

Report #49 (fishing): Trip out of hole with fishing bha
grounded: True

```
[verified] Report #49 (faithfulness 0.80)
[not used] Report #49 (faithfulness 0.40)
[not used] Report #50 (faithfulness 0.20)
```

10.20. What just happened?

Four answers, three different real outcomes:

- **#37** is genuinely faithful, and correctly verified against the right report — the check confirms what Chapter 9 already showed.
- **#38** is correctly flagged **grounded: False**. No retrieved chunk crosses the threshold, matching Chapter 9's own human judgment that this answer ignores the retrieved text entirely — automated now, not eyeballed.
- **#21** is the case a plain **grounded: True/False** flag would have gotten wrong. It reads as grounded — and it is, just not in the report the question was actually about. Only checking **verified_sources** against the *target* report catches this; the boolean alone would have silently counted it as a success.

#49 is worth a closer look too — see Field Notes.

10.21. Practical exercise

Beginner

Try it yourself: run `python code/chapter_10/traceable_outputs.py` from inside `book/` and compare its printed output to the transcript above.

You'll know it worked when: you see **grounded: False** for report #38 and **grounded: True** for report #37, and you can explain in one sentence why report #21's case is more concerning than #38's, even though both were flagged by Chapter 9 as failures.

10.22. Field notes

 Field notes: right words, wrong direction

Query: report #49's fishing-operation answer, checked word by word against its own correctly-verified source.

Result: the grounded answer says *"Trip out of hole with fishing bha"* — faithfulness 0.80 against report #49's actual text, well above threshold, correctly verified. Report #49's real text says:

Production Drilling Trips

Pick up Fishing BHA # 33 and trip in hole to 7,996'

The report says **trip in hole**. The answer says **trip out of hole** — the opposite direction — and drops the specific depth, 7,996', entirely.

Why: "out" and "in" are both short, common words. This chapter's check only tests whether an answer's words appear *somewhere* in the source; it has no notion of which words modify which, so "trip", "fishing", and "bha" all matching is enough to score high even though the one word that actually changes the meaning of the sentence flipped.

Lesson: a high faithfulness score means an answer’s vocabulary came from the right place — not that every claim in it is correct. #21’s wrong-report case and #49’s flipped-direction case are two different failure modes this simple check catches differently: one it flags correctly by score alone, the other it verifies with a score that’s honestly still too high. Both are worth knowing about; only one is currently caught automatically.

10.23. Challenge exercise

Intermediate

Challenge: does this chapter’s check also catch hallucination when there’s no retrieval context at all — Chapter 9’s original, un-grounded baseline answers? Regenerate each of the 4 test cases *without* retrieval, then score each answer against its own target report’s real text using this chapter’s `faithfulness_score`. A reference solution is at `code/chapter_10/challenge/challenge.py` — on a real run, every un-grounded answer scores at or below 0.17 against its own target report (one scores 0.00), well under the 0.5 threshold this chapter uses, confirming the check generalizes: it isn’t just tuned to Chapter 9’s specific retrieval setup, it correctly flags “no real evidence used” as unfaithful in general.

10.24. Key takeaways

- A citation only proves retrieval did its job — it proves nothing about whether generation actually used the report it names. Report #21’s example is a true citation attached to a wrong answer.
- A plain `grounded: True/False` flag would have missed report #21’s failure entirely. Checking *which* source an answer verifies against, not just whether one does, is what actually catches it.
- This chapter’s faithfulness check is lexical, not semantic — it reliably catches “used the wrong evidence” (#21) but can miss a factual inversion hiding inside otherwise-matching words (#49). Knowing which failure mode a check does and doesn’t catch matters as much as having the check.
- The same check that catches a wrong-source hallucination also correctly flags a fully un-grounded answer as unfaithful — it isn’t narrowly tuned to one failure mode.
- This chapter’s faithfulness score is an educational baseline, not a validated safety system — report #49’s reversed-direction answer scored 0.80 all the same. It must never be used as an unsupervised safety check on live well operations; a qualified person checking the actual source stays the real safeguard, especially for a handover summary someone might act on.

10.25. Repository files

File	Purpose
<code>code/chapter_10/traceable_outputs.py</code>	Faithfulness scoring and traceable-source verification over Chapter 9’s hybrid system

File	Purpose
<code>code/chapter_10/challenge/challenge.py</code>	Reference solution: faithfulness-checking Chapter 9’s un-grounded baseline answers
<code>notebooks/chapter_10_explore.ipynb</code>	Interactive companion notebook

🔥 CHECKPOINT — Chapter 10

- ☑ Built a real, inspectable faithfulness score between an answer and a source, with no opaque second model involved
- ☑ Found a real case where a true citation was attached to an answer that actually came from a different report
- ☑ Ran the check against all 4 of Chapter 9’s test cases and confirmed it flags exactly the failure Chapter 9’s Field Notes found, plus one Chapter 9 couldn’t distinguish from a success
- ☑ Named a concrete limitation of the check (a factual inversion inside otherwise-matching words) instead of overselling it

💡 WHAT YOU BUILT

traceable_outputs.py — a faithfulness check that scores a generated answer against each source chunk it was shown, and a **verified_sources** list that only keeps the citations an answer is actually traceable to. Applied to Chapter 9’s hybrid system, it turns “here’s what was retrieved” into “here’s what the answer is actually based on” — catching a real, fluent, source-backed-sounding answer that was grounded in the wrong report.

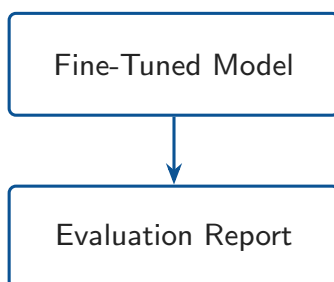
10.26. What can you do now that you couldn’t do before?

You can tell the difference between an answer that cites a real source and an answer that’s actually faithful to it — and you have a concrete, inspectable number to point to instead of reading every answer by eye the way Chapter 9 had to.

10.27. Suggested next step

Coming up in Chapter 11: this chapter checked faithfulness one hand-picked test case at a time — exactly 4 of them, chosen because Chapter 9 already knew what to look for in each. That doesn’t scale, and it doesn’t produce the kind of summary metric a real evaluation needs. Chapter 11 builds a proper evaluation harness: a fixed set of questions with known-correct answers, run automatically, scored consistently, so a change to the model or the retrieval setup can be judged by a number instead of a handful of examples someone happened to check by hand.

11. Evaluating a Fine-Tuned Domain Model



Progress #####. . 11 / 13 · Estimated time: 30–40 min · Difficulty: Intermediate

11.1. Learning objectives

By the end of this chapter, you will be able to:

- Build a real, held-out evaluation set from a report that has never been in any training set in this book — not two hand-picked questions, eight real ones
- Explain why a single metric can make fine-tuning look like it did nothing, and why a second metric can prove that’s wrong
- Compute perplexity — a metric that needs real held-out text, not a known-correct answer — and use it to measure something exact-match can’t see at all
- Read Chapter 8’s own `metrics.csv` for what it’s been quietly tracking every epoch, and turn one column of it into a real evaluation harness

11.2. Operational Problem

Carlos, the completions engineer who asked back in Chapter 5 whether fine-tuning teaches the operation or just memorizes flashcards, has a sharper version of the same question now that Chapters 9 and 10 added retrieval and faithfulness-checking on top: *“We’ve been checking all of this on a handful of questions I picked myself, because I already knew what I was looking for in each one. How do I know any of it holds up on a question I didn’t pick?”*

Chapter 5 answered a narrower version of this with 0/16 and 0/2. Chapter 9 and 10 each checked exactly 4 hand-picked cases. None of that is a real evaluation — it’s a handful of examples someone happened to check by hand, exactly what Chapter 10 ended by admitting doesn’t scale. This chapter builds a real evaluation set — 8 real questions from a report that has never been in any training

11. Evaluating a Fine-Tuned Domain Model

set in this book — and scores it more than one way, because a single number turns out not to be enough to answer Carlos’s question honestly.

11.3. Example: one answer, asked eight different questions

Run Chapter 8’s fine-tuned model against all 8 real held-out questions this chapter builds — no retrieval, no grounding context, exactly the same setup Chapter 5’s original held-out check used — and something worth noticing happens before any scoring at all:

i Eight different questions, from a report the model never trained on

```
Time: 06:00-18:30 (part 1 of 2) -> Production Casing Run Csg & Cement Rig up casing
Time: 18:30-20:30                -> Production Casing Run Csg & Cement Rig up cement
Time: 20:30-21:30                -> Production Casing Run Csg & Cement Rig up cement
Time: 21:30-00:30                -> Production Casing Run Csg & Cement Rig up casing
Time: 00:30-02:30                -> Production Casing Run Csg & Cement Rig up casing
Time: 02:30-03:30                -> Production Casing Run Csg & Cement Rig up cement
Time: 03:30-06:00                -> Production Casing Run Csg & Cement Rig up casing
```

Eight different time windows, seven near-identical answers. Whatever Chapter 8’s fine-tune actually learned, it isn’t “what happened at each of these specific times” — it’s something closer to a single memorized-sounding template the model reaches for regardless of the question. Scoring that against a known-correct answer is one useful number. It is not the only useful number.

11.4. Theory

💡 Engineering Translation: evaluation set

An **evaluation set** is a fixed, reusable list of questions with known-correct answers, held back from training the same way Chapter 2’s held-out report always has been — the difference this chapter makes is *size*. Chapter 3’s held-out check used 2 questions from that report; this chapter uses all 8 real ones the report actually contains, built with Chapter 7’s own chunker.

💡 Engineering Translation: perplexity

Perplexity measures how surprised a model is by a piece of real text — lower means the model found the actual words more expected, higher means it found them more surprising. Unlike exact-match or overlap scoring, perplexity doesn’t need a “correct answer” to compare against at all — just real text the model never trained on. That makes it a useful second opinion when the first metric says “no progress.”

11.5. Why one number isn't enough

The tempting shortcut: run the held-out set once, get an exact-match score, and treat that as the verdict on whether fine-tuning worked. Chapter 8 has quietly been logging exactly that number every epoch, in its own `metrics.csv` — a `held_out_exact_match` column that reads 0/8 at epoch 1, 0/8 at epoch 2, 0/8 at epoch 3. Read on its own, that number says fine-tuning made zero progress on anything it hadn't already memorized. Field Notes below shows a second, real metric that says something very different about the exact same three checkpoints — both numbers are true, and neither one alone tells Carlos what he actually asked.

11.6. Implementation

11.6.1. Step 1: Build a real, held-out evaluation set

11.7. What problem are we solving?

Chapter 3's held-out check used exactly 2 examples — the report's two coarse summary fields. Build a real evaluation set from the same never-trained-on report, at the same granularity Chapter 7 used for training at scale.

11.8. Inputs

- `HELD_OUT_REPORT` (Chapter 2) and `FULL_SET_DIR` (Chapter 6)
- `build_timeline_examples_for_report` (Chapter 7), reused unchanged

11.9. Expected Output

A list of real instruction/input/output examples, one per timeline entry the held-out report actually contains.

```
def build_held_out_eval_set() -> list[dict]:
    examples, _artifacts_filtered =
    ↪ build_timeline_examples_for_report(FULL_SET_DIR / HELD_OUT_REPORT)
    return examples

eval_set = build_held_out_eval_set()
print(f"{len(eval_set)} held-out examples, report never used in any training
    ↪ set")
```

Running this exact code prints:

```
8 held-out examples, report never used in any training set
```

11.10. What just happened?

`build_timeline_examples_for_report` is Chapter 7’s own function, called directly on the one report Chapter 2 reserved and Chapter 5 and 8 both excluded from training. This isn’t new held-out data — it’s the exact same 8 examples Chapter 8’s training script already builds internally for its own `held_out_exact_match` logging. This chapter is the first time this book actually looks past that one column.

11.11. Production Reality

Eight examples is still small by production standards — a real evaluation set for a deployed system would hold back dozens of reports, not one, to make the score statistically meaningful rather than a single report’s quirks. This chapter’s version stays honest about that limit rather than implying 8 examples settles the question; see Key takeaways.

11.11.1. Step 2: Score two ways — exact-match and partial credit

11.12. What problem are we solving?

Reuse Chapter 3’s `matched_expected` check — renamed `exact_match` here, though it still does exactly what Chapter 3 described: it checks whether the generated text *contains* the expected answer, word for word, not whether the two strings are identical. Pair it with Chapter 10’s word-overlap scoring (originally built to check an answer against a *retrieved source*) applied to a different comparison: an answer against its *known-correct output*.

11.13. Inputs

- The fine-tuned model and tokenizer (Chapter 8’s checkpoint)
- The held-out evaluation set from Step 1
- `faithfulness_score` (Chapter 10), reused unchanged

11.14. Expected Output

Every example scored both ways: a strict pass/fail and a 0–1 partial credit score.

```
def exact_match(generated: str, expected: str) -> bool:
    return expected.strip().lower() in generated.strip().lower()

def evaluate(model, tokenizer, eval_set: list[dict]) -> list[dict]:
    results = []
    for example in eval_set:
```

```

prompt = f"{example['instruction']}\n{example['input']}"
generated = generate_reply(model, tokenizer, prompt, max_new_tokens=60)
results.append({
    "input": example["input"],
    "expected": example["output"],
    "generated": generated,
    "exact_match": exact_match(generated, example["output"]),
    "overlap_score": faithfulness_score(generated, example["output"]),
})
return results

results = evaluate(lora_model, tokenizer, eval_set)
matches = sum(r["exact_match"] for r in results)
avg_overlap = sum(r["overlap_score"] for r in results) / len(results)
print(f"Exact-match: {matches}/{len(results)}")
print(f"Average overlap score: {avg_overlap:.2f}")

```

Running this exact code prints:

```

Exact-match: 0/8
Average overlap score: 0.17

```

11.15. What just happened?

0/8 on its own reads like total failure. 0.17 average overlap tells a more precise story: not zero, but low — the generated answers share some vocabulary with the real ones (mostly the archive’s own recurring category words, “Production,” “Drilling”) without capturing the specific content each question actually asked about. Reusing Chapter 10’s exact scoring function for a different comparison is deliberate: the same simple technique generalizes to a second real problem without needing to be reinvented.

11.16. Production Reality

`faithfulness_score` is still the same plain word-overlap check Chapter 10 named the limits of — it can’t detect a paraphrase that uses different words correctly, and it can’t detect a subtle inversion hiding inside otherwise-matching words. Those limits carry over here unchanged; a 0.17 average is a real, low signal, not a precise one.

11.16.1. Step 3: Perplexity — a metric that doesn't need a correct answer

11.17. What problem are we solving?

Measure whether fine-tuning changed how surprised the model is by real held-out text, independent of whether it can reproduce that text exactly.

11.18. Inputs

- A base (pre-fine-tuning) model and Chapter 8's fine-tuned checkpoint
- The same held-out text from Step 1

11.19. Expected Output

A perplexity score for each model on the same real text.

```
import math

def perplexity(model, tokenizer, texts: list[str]) -> float:
    total_loss, total_examples = 0.0, 0
    for text in texts:
        input_ids = tokenizer(text, return_tensors="pt")["input_ids"]
        with torch.no_grad():
            loss = model(input_ids=input_ids, labels=input_ids).loss
        total_loss += loss.item()
        total_examples += 1
    return math.exp(total_loss / total_examples)

held_out_texts = [e["output"] for e in eval_set]
print(f"Base model:    {perplexity(base_model, tokenizer, held_out_texts):.2f}")
print(f"Fine-tuned:    {perplexity(lora_model, tokenizer, held_out_texts):.2f}")
```

Running this exact code prints:

```
Base model:    159.91
Fine-tuned:    25.03
```

11.20. What just happened?

The fine-tuned model found this exact-same real, never-trained-on text about 6.4× less surprising than the base model did — a real, substantial change, on the exact same 8 examples that just scored 0/8 exact-match. Fine-tuning taught the model this archive’s vocabulary and phrasing well enough to make genuinely unseen report text far more expected, even though it still can’t reliably reproduce the *specific* content of a report it hasn’t seen. Chapter 5 called this “shape, not judgment” from a handful of wrong answers; this is the same finding, quantified.

11.21. Practical exercise

Beginner

Try it yourself: run `python code/chapter_11/eval_finetuned_model.py` from inside `book/` and compare its printed output to the transcript above.

You’ll know it worked when: you see `Exact-match: 0/8` and a fine-tuned perplexity noticeably lower than the base model’s, and you can explain in one sentence why both numbers are true at the same time.

11.22. Field notes

! Field notes: the number Chapter 8 already had, and never looked at twice

Query: Chapter 8’s own `checkpoints/chapter_08/*/metrics.csv`, read all the way across instead of just its `held_out_exact_match` column.

Result:

Epoch	Train loss	Held-out exact-match	Held-out perplexity
1	2.755	0/8	27.99
2	2.164	0/8	25.40
3	1.821	0/8	25.03

Why: `held_out_exact_match` was 0/8 at every single checkpoint — by that column alone, epochs 2 and 3 look like wasted training time. Perplexity on the exact same held-out text tells a different, consistent story: it falls at every checkpoint, tracking the falling training loss almost exactly, even while exact-match never moves.

Lesson: a metric that never changes isn’t proof that nothing is changing — it can just be measuring something training isn’t currently improving. Chapter 8 logged the ingredients for this table from its first real run; this chapter is what it took to actually read them together.

11.23. Challenge exercise

Intermediate

Challenge: Chapter 8 saved a checkpoint after every epoch. Compute this chapter’s perplexity metric against all 3 of them and see whether it falls monotonically, the way Field Notes above suggests. A reference solution is at `code/chapter_11/challenge/challenge.py` — on a real run: `checkpoint_1: perplexity 27.99, checkpoint_2: perplexity 25.40, checkpoint_3: perplexity 25.03` — the exact numbers behind Field Notes’ table, falling at every epoch.

11.24. Key takeaways

- A single evaluation metric can make real training progress look like nothing happened. `held_out_exact_match` stayed 0/8 for all 3 of Chapter 8’s epochs; perplexity on the identical text fell at every one of them.
- Perplexity’s advantage is exactly that it doesn’t need a known-correct answer — it only needs real, unseen text, which makes it usable even when you don’t have (or trust) a labeled evaluation set.
- Reusing Chapter 10’s `faithfulness_score` for a completely different comparison (answer vs. known-correct output, not answer vs. source) worked without modification — the same simple technique generalized to a second real problem.
- Eight examples is a real evaluation set, not a toy one — but it’s still one report’s worth of questions. Don’t read 0.17 average overlap or 25.03 perplexity as more statistically solid than they are.

11.25. Repository files

File	Purpose
<code>code/chapter_11/eval_finetuned_model.py</code>	Builds the held-out evaluation set and scores exact-match, overlap, and perplexity
<code>code/chapter_11/challenge/challenge.py</code>	Reference solution: perplexity across all 3 of Chapter 8’s checkpoints
<code>notebooks/chapter_11_explore.ipynb</code>	Interactive companion notebook

CHECKPOINT — Chapter 11

- ☒ Built a real, 8-example held-out evaluation set from a report never used in any training set in this book
- ☒ Scored it two ways — strict exact-match and partial-credit overlap — instead of trusting one number
- ☒ Computed perplexity, a metric that needs no known-correct answer, and found it tells a different story than exact-match on the identical text
- ☒ Read Chapter 8’s own logged metrics all the way across instead of only its headline

column



WHAT YOU BUILT

`eval_finetuned_model.py` — a real evaluation harness: an 8-example held-out set, scored by exact-match, word-overlap partial credit, and perplexity. Applied to Chapter 8's checkpoints, it shows training progress a single exact-match number completely hides.

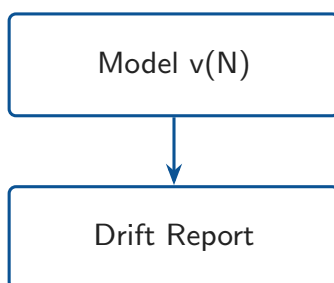
11.26. What can you do now that you couldn't do before?

You can evaluate a fine-tuned model against a real, fixed, held-out question set instead of a handful of examples you happened to check by hand — and you know why a single metric isn't enough to trust the answer.

11.27. Suggested next step

Coming up in Chapter 12: this chapter evaluated one model version against a fixed held-out set, one snapshot in time. A real deployment doesn't stay still — new reports arrive, and eventually someone retrains. Chapter 12 takes this exact evaluation harness and runs it across multiple model versions, to catch the case this chapter can't: a model that used to score well on this held-out set and quietly stops doing so.

12. Detecting Drift Across Model Versions



Progress #####. 12 / 13 · Estimated time: 30–40 min · Difficulty: Intermediate

12.1. Learning objectives

By the end of this chapter, you will be able to:

- Run Chapter 11’s evaluation harness across more than one real model version and compare them directly, instead of judging each version in isolation
- Find a real case where two of Chapter 11’s own metrics disagree about whether a newer model version is better or worse
- Investigate a metric disagreement by reading the model’s actual generated text, instead of trusting either number alone
- Explain why a metric that stays flat across versions can hide a real change underneath it, and why a metric that moves needs a reason, not just a direction

12.2. Operational Problem

Farid, the production engineer who worried back in Chapter 7 about losing information by throwing away most of a shift’s log, has a new worry now that Chapter 11 gave the book a real evaluation harness: *“We’re not going to fine-tune once and stop — we’ll retrain as new reports come in. How do I know a new version isn’t secretly worse than the one it’s replacing?”*

12.3. Example: two checkpoints, two different verdicts

Run Chapter 11’s evaluation harness against Chapter 8’s first two real checkpoints — epoch 1 and epoch 2 of the exact same training run — and compare them directly:

i checkpoint_1 -> checkpoint_2, the same held-out set both times

checkpoint_1: exact_match=0 avg_overlap=0.354 perplexity=27.99

checkpoint_2: exact_match=0 avg_overlap=0.167 perplexity=25.40

checkpoint_1 -> checkpoint_2: exact_match=unchanged avg_overlap=regressed perplexity=improved

Perplexity says checkpoint_2 is a real improvement. The overlap score says it's a real regression — average overlap dropped by more than half. Both numbers come from the exact same two checkpoints, the exact same 8 held-out questions, run the same way Chapter 11 already verified. Neither one is wrong. Neither one is the whole answer either.

12.4. Theory

💡 Engineering Translation: drift / regression

Drift is a model's behavior changing across versions or over time — often, but not always, for the worse. A **regression** is a specific kind of drift: something that used to work (or score well) stops working, in a version that's supposed to be an improvement. Neither term says *why* — only that something changed and is worth a second look. This chapter's drift check means comparing two already-trained checkpoints against one fixed evaluation set. A production ML team also watches for a different kind of drift — a deployed model's live, unlabeled input traffic gradually looking different from what it was trained on — which is a separate, ongoing monitoring problem this book doesn't build.

12.5. Why comparing one metric isn't enough

The tempting shortcut: pick one metric, watch it across versions, and trust whatever it says. Chapter 11 already showed the risk of that with a single version's evaluation; the checkpoint_1 → checkpoint_2 comparison above shows the same risk across versions, more sharply. Trusting perplexity alone here says “ship checkpoint_2, it's better.” Trusting overlap alone says “checkpoint_2 is a regression, investigate before shipping it.” A real drift check needs more than one metric, and when they disagree, it needs a way to actually look at what changed — not just declare one metric the tiebreaker.

12.6. Implementation

12.6.1. Step 1: Summarize one model version across all three metrics

12.7. What problem are we solving?

Turn Chapter 11's three separate metrics into a single, comparable summary per model version.

12.8. Inputs

- A loaded model version (a `PeftModel` wrapping the base model)
- The held-out evaluation set from Chapter 11

12.9. Expected Output

One dictionary per version: exact-match count, average overlap score, and perplexity.

```
def summarize_version(model, tokenizer, eval_set: list[dict]) -> dict:
    results = evaluate(model, tokenizer, eval_set)
    texts = [e["output"] for e in eval_set]
    return {
        "exact_match": sum(r["exact_match"] for r in results),
        "avg_overlap": sum(r["overlap_score"] for r in results) / len(results),
        "perplexity": perplexity(model, tokenizer, texts),
    }
```

12.10. What just happened?

`evaluate` and `perplexity` are Chapter 11’s own functions, reused unchanged. This step doesn’t add a new metric — it packages the three that already exist into a shape that’s easy to compare, version to version.

12.11. Production Reality

Real deployments version far more than a LoRA checkpoint — the retrieval index, the prompt template, even the base model itself can change between versions, and each is its own source of drift. This chapter only compares the fine-tuned adapter, holding everything else fixed, so a real difference is guaranteed to come from the adapter and nothing else.

12.11.1. Step 2: Compare two version summaries

12.12. What problem are we solving?

Turn two summaries into a per-metric verdict: did this metric improve, regress, or stay the same between versions?

12.13. Inputs

- Two `summarize_version` outputs, one “before” and one “after”

12.14. Expected Output

A direction per metric — `improved`, `regressed`, or `unchanged`.

```
def compare_versions(before: dict, after: dict) -> dict:
    def direction(before_value, after_value, lower_is_better=False):
        if after_value == before_value:
            return "unchanged"
        improved = after_value < before_value if lower_is_better else after_value
        <- > before_value
        return "improved" if improved else "regressed"

    return {
        "exact_match": direction(before["exact_match"], after["exact_match"]),
        "avg_overlap": direction(before["avg_overlap"], after["avg_overlap"]),
        "perplexity": direction(before["perplexity"], after["perplexity"],
        <- lower_is_better=True),
    }
```

12.15. What just happened?

`compare_versions` reports direction, not a verdict — it doesn't decide whether `checkpoint_2` is “good enough to ship.” That's deliberate: collapsing three metrics into one automatic pass/fail would hide exactly the kind of disagreement Field Notes below investigates.

12.16. Production Reality

This function has no numeric threshold — any change in direction counts, even a tiny one. A production system would need a real threshold per metric (how much perplexity change is noise vs. signal), tuned against many more than 8 examples. This chapter keeps it simple enough to read every line of, at the cost of being oversensitive to small, possibly meaningless fluctuations.

12.16.1. Step 3: Run it across all 3 real checkpoints

12.17. What problem are we solving?

Apply Steps 1 and 2 to every checkpoint from Chapter 8's real training run, not just two hand-picked ones.

12.18. Inputs

- Chapter 8's 3 real checkpoints (`checkpoint_1`, `checkpoint_2`, `checkpoint_3`)
- The held-out evaluation set

12.19. Expected Output

A summary per checkpoint, and a direction comparison for every consecutive pair.

```
summaries = {}
for checkpoint_dir in checkpoints:
    model, tokenizer = load_version(checkpoint_dir)
    summaries[checkpoint_dir.name] = summarize_version(model, tokenizer,
    ↪ eval_set)

for before_name, after_name in zip(list(summaries), list(summaries)[1:]):
    print(f"{before_name} -> {after_name}:
    ↪ {compare_versions(summaries[before_name], summaries[after_name])}")
```

Running this exact code prints:

```
checkpoint_1 -> checkpoint_2: {'exact_match': 'unchanged', 'avg_overlap': 'regressed', 'perplexity': 'improved'}
checkpoint_2 -> checkpoint_3: {'exact_match': 'unchanged', 'avg_overlap': 'unchanged', 'perplexity': 'improved'}
```

12.20. What just happened?

Two consecutive comparisons, two different outcomes. `checkpoint_2 -> checkpoint_3` is the boring, healthy case: perplexity keeps improving, nothing regresses. `checkpoint_1 -> checkpoint_2` is the one worth stopping for — a real metric disagreement, on real checkpoints, that Field Notes below investigates by reading what the model actually generated.

12.21. Practical exercise

Beginner

Try it yourself: run `python code/chapter_12/detect_model_drift.py` from inside `book/` and compare its printed output to the transcripts above.

You'll know it worked when: you see `avg_overlap: regressed` between `checkpoint_1` and `checkpoint_2`, and you can explain in one sentence why that doesn't automatically mean `checkpoint_2` is a worse model.

12.22. Field notes

⚠ Field notes: same template, different template

Query: what `checkpoint_1` and `checkpoint_2` actually generated for the same 8 held-out questions, side by side.

Result: `checkpoint_1` answers nearly every question with some version of “*Production Casing Run down hole to 9,400’.*” `checkpoint_2` answers nearly every question with some version of “*Production Casing Run Csg & Cement Rig up cement.*” Neither checkpoint is actually answering the specific question asked — both fall back to one memorized-sounding template, the same “shape, not judgment” pattern Chapter 3, 9, and 11 already found.

Why: `checkpoint_1`’s template happens to share more real words (“down hole,” “run,” numbers matching some of the true answers) with this specific 8-question held-out set than `checkpoint_2`’s template does — not because `checkpoint_1` understands the questions better, but because of which arbitrary generic phrase each epoch happened to settle into. The overlap score is measuring that coincidence, not a real improvement or decline in understanding.

Lesson: a regressing metric is a reason to look, not a verdict on its own. Here, looking revealed the “regression” wasn’t really about `checkpoint_2` getting worse at the task — it was two different flavors of the same underlying limitation, one of which happened to score better against this particular held-out set by chance. A real production drift check would need to tell those two situations apart before deciding whether to roll a version back.

12.23. Challenge exercise

Intermediate

Challenge: does the same kind of metric disagreement show up comparing two completely different training regimes, not just adjacent epochs of one run? Compare Chapter 5’s 16-example fine-tune against Chapter 8’s “at scale” checkpoint using this chapter’s own `compare_versions`. A reference solution is at `code/chapter_12/challenge/challenge.py` — on a real run: Chapter 5 scores `avg_overlap=0.5625`, `perplexity=112.28`; Chapter 8 scores `avg_overlap=0.167`, `perplexity=25.03` — the exact same disagreement pattern (`avg_overlap: regressed`, `perplexity: improved`) holds even across two entirely different fine-tunes, not just two epochs of the same one.

12.24. Key takeaways

- Two of Chapter 11’s own metrics can disagree about whether a newer model version is better — `checkpoint_2` looked worse by overlap score and better by perplexity than `checkpoint_1`, on the same held-out set.
- A regressing metric is a signal to investigate, not a verdict. The real cause here was two different memorized templates, not a genuine loss of understanding between epochs.
- `compare_versions` reports direction, deliberately not a pass/fail verdict — collapsing a disagreement into one automatic answer would have hidden exactly the thing worth catching.

- The same disagreement pattern (`avg_overlap` regressed, perplexity improved) showed up comparing two completely different training regimes, not just two epochs of one run — this isn't a one-off coincidence of this specific training run.

12.25. Repository files

File	Purpose
<code>code/chapter_12/detect_model_drift.py</code>	Summarizes and compares model versions across Chapter 11's three metrics
<code>code/chapter_12/challenge/challenge.py</code>	Reference solution: comparing Chapter 5's and Chapter 8's checkpoints as two real versions
<code>notebooks/chapter_12_explore.ipynb</code>	Interactive companion notebook

CHECKPOINT — Chapter 12

- ☒ Ran Chapter 11's evaluation harness across more than one real model version instead of judging each in isolation
- ☒ Found a real case where two metrics disagree about whether a newer version is an improvement
- ☒ Investigated the disagreement by reading the model's actual generated text, and found a real, specific explanation
- ☒ Confirmed the same pattern holds comparing two entirely different training regimes, not just adjacent epochs

WHAT YOU BUILT

`detect_model_drift.py` — `summarize_version()` collapses Chapter 11's three metrics into one comparable summary per model version; `compare_versions()` reports which metrics improved, regressed, or stayed flat between two versions, without collapsing a disagreement into a false single verdict.

12.26. What can you do now that you couldn't do before?

You can compare two real model versions on more than one metric at once, catch the case where they disagree, and investigate why instead of trusting whichever number happens to look best.

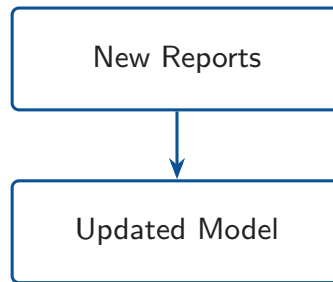
12.27. Suggested next step

Coming up in Chapter 13: this chapter compares model versions that already exist — it doesn't decide when a new one should get trained in the first place, or what to do when this chapter's own comparison flags a disagreement. Chapter 13 closes that loop: retraining as new reports arrive,

12. Detecting Drift Across Model Versions

and running this exact comparison before deciding whether a new version actually replaces the one currently in use.

13. Continuous Fine-Tuning: Keeping the Model Current



Progress ##### 13 / 13 · Estimated time: 45–60 min · Difficulty: Advanced

13.1. Learning objectives

By the end of this chapter, you will be able to:

- Simulate new reports arriving after a model is already in production, using a real chronological split of this book’s own archive
- Continue fine-tuning an existing checkpoint on just the new batch, reusing Chapter 8’s exact save-and-resume mechanism
- Run Chapter 12’s comparison harness on the result to decide whether an updated model is actually worth deploying
- Explain, from a real run, why “new data arrived” is not the same claim as “the new model is better”

13.2. Operational Problem

Sarah, the intervention engineer who first asked back in Chapter 9 whether she could trust an answer or still had to go find the PDF herself, has one last version of that question now that the book is almost done: *“Everything we’ve built assumes the reports we have today. What happens when new ones come in next month — do we just retrain and swap in whatever comes out?”*

13.3. Example: the same question this book has asked all along, one more time

Simulate exactly that. Split this book’s real, quality-gated archive by report number — reports before #60 stand in for “already in production,” reports #60 onward stand in for “just arrived” — train a model on the first group, continue training it on the second, and ask Chapter 12’s comparison harness whether the result is actually better:

i Current model vs. updated model, same held-out set both times

Current model: exact_match=0 avg_overlap=0.464 perplexity=25.92

Updated model: exact_match=0 avg_overlap=0.125 perplexity=25.73

Direction: exact_match=unchanged avg_overlap=regressed perplexity=improved

New data arrived. Training ran without errors. Loss fell. And by one of this book’s own metrics, the result is a regression. This isn’t a hypothetical warning — it’s what actually happened the one time this book ran the full loop for real.

13.4. Theory

💡 Engineering Translation: continuous fine-tuning

Continuous fine-tuning means updating a model again as new data becomes available, instead of training it once and leaving it fixed forever. It answers *how* to incorporate new data. It does not by itself answer *whether* the result should replace what’s currently running — that’s still Chapter 12’s job, run again on the new checkpoint.

13.5. Why “we retrained” isn’t the same claim as “it’s better”

The tempting shortcut this chapter exists to head off: new reports arrived, so retrain, and deploy whatever comes out, because newer data must mean a more current, more capable model. The Example above is the real counter-case — training completed cleanly, loss fell every epoch, and the resulting model still regressed on the same held-out set by `avg_overlap`. Retraining is a mechanical step. Deciding whether the result should replace what’s live is a separate decision, and it needs Chapter 12’s comparison, run again, every time — not assumed from the fact that training happened.

13.6. Implementation

13.6.1. Step 1: Split the archive by a chronological cutoff

13.7. What problem are we solving?

Simulate “new reports arriving” using this book’s own real archive, without touching the one report (#37) reserved as held-out everywhere else in this book.

13.8. Inputs

- `build_archive_records` (Chapter 6), `HELD_OUT_REPORT` (Chapter 2)
- A cutoff report number

13.9. Expected Output

Two lists of real report records: everything before the cutoff, and everything at or after it.

```
CUTOFF_REPORT_NUM = 60

def split_archive_by_cutoff(cutoff: int = CUTOFF_REPORT_NUM) -> tuple[list[dict],
    ↪ list[dict]]:
    records = build_archive_records()
    ok = [r for r in records if r["status"] == "ok" and r["file"] !=
    ↪ HELD_OUT_REPORT]
    current_reports = [r for r in ok if r["rpt_num"] < cutoff]
    new_reports = [r for r in ok if r["rpt_num"] >= cutoff]
    return current_reports, new_reports

current_reports, new_reports = split_archive_by_cutoff()
print(f'Currently in production: {len(current_reports)} reports')
print(f'New reports just arrived: {len(new_reports)} reports')
```

Running this exact code prints:

```
'Currently in production': 57 reports
New reports just arrived: 17 reports
```

13.10. What just happened?

57 + 17 = 74 — every quality-gated, non-held-out report in the archive, split by report number into “before” and “after” a real cutoff. Report #37 never appears in either half, the same reservation every earlier chapter has respected.

13.11. Production Reality

This book’s archive stops in early January 2021 — there’s no genuinely new data to wait for. A production system would use the actual date a report was filed, not a chosen report-number cutoff, and the “new” batch would arrive continuously, a few reports at a time, not as one 17-report batch.

13.11.1. Step 2: Train the current model, then continue on the new batch

13.12. What problem are we solving?

Produce two real checkpoints — one trained on the archive as it stood before the cutoff, one continuing training on only the new batch — using Chapter 8’s exact checkpoint-and-resume mechanism.

13.13. Inputs

- `current_examples` and `new_examples`, built with Chapter 7’s chunker
- `train_with_checkpoints`, `save_checkpoint`, `load_checkpoint` (Chapter 8), reused unchanged

13.14. Expected Output

Four checkpoints: two from training on the current archive, two more continuing on the new batch alone.

```
model, tokenizer = load_model_and_tokenizer(MODEL_NAME)
lora_model = build_lora_model(model)
train_with_checkpoints(lora_model, tokenizer, current_examples,
    ↪ held_out_eval_set, run_dir, logger, num_epochs=2)
current_checkpoint = run_dir / "checkpoint_2"

del model, lora_model
model, tokenizer = load_model_and_tokenizer(MODEL_NAME)
lora_model = load_checkpoint(model, current_checkpoint)
train_with_checkpoints(lora_model, tokenizer, new_examples, held_out_eval_set,
    ↪ run_dir, logger, num_epochs=2, start_epoch=2)
```

```
updated_checkpoint = save_checkpoint(lora_model, run_dir, epoch=4)
```

Running this exact code prints:

```
epoch 1: avg loss 3.0296, held-out 0/8 (410s) -> checkpoint_1
epoch 2: avg loss 2.3672, held-out 0/8 (409s) -> checkpoint_2
epoch 3: avg loss 1.8747, held-out 0/8 (142s) -> checkpoint_3
epoch 4: avg loss 1.5092, held-out 0/8 (145s) -> checkpoint_4
```

13.15. What just happened?

Loss fell at every epoch, in both stages — training worked exactly as intended, on both the current archive and the new batch. `checkpoint_2` is “currently in production”; `checkpoint_4` is “the update after new reports arrived,” produced the same reload-from-disk way Chapter 8 already simulates a crash and restart. Nothing about this step signals a problem. That’s the point of Step 3.

13.16. Production Reality

Two epochs on 179 new examples is a small, fast update by design — a real production system might retrain more or less aggressively depending on how much new data has accumulated and how expensive a full retrain is. This chapter’s fixed 2-epoch continuation is a teaching choice, not a tuned production schedule. It also trains on `new_examples` alone, with none of `current_examples` mixed back in — a real instance of what the field calls **catastrophic forgetting** (or, more precisely here, recency bias): training only on new data can pull a model’s behavior away from what it learned earlier, even without erasing it outright. The standard mitigation — replaying a sample of `current_examples` alongside each new batch — isn’t attempted here; Step 3 and the Field Notes below show part of what skipping it costs.

13.16.1. Step 3: Compare before deciding to deploy

13.17. What problem are we solving?

Answer the question Step 2’s clean training run can’t: is `checkpoint_4` actually an improvement over `checkpoint_2`?

13.18. Inputs

- Both checkpoints from Step 2, reloaded fresh from disk
- `load_version`, `summarize_version`, and `compare_versions` (Chapter 12), reused unchanged

13.19. Expected Output

A direction per metric between the current and updated model.

```
lora_current, tokenizer = load_version(current_checkpoint)
summary_current = summarize_version(lora_current, tokenizer, held_out_eval_set)

lora_updated, _ = load_version(updated_checkpoint)
summary_updated = summarize_version(lora_updated, tokenizer, held_out_eval_set)

print(f"Current model: {summary_current}")
print(f"Updated model: {summary_updated}")
print(f"Direction:      {compare_versions(summary_current, summary_updated)}")
```

Running this exact code prints:

```
Current model: {'exact_match': 0, 'avg_overlap': 0.464, 'perplexity': 25.92}
Updated model: {'exact_match': 0, 'avg_overlap': 0.125, 'perplexity': 25.73}
Direction:     {'exact_match': 'unchanged', 'avg_overlap': 'regressed', 'perplexity': 'improved'}
```

13.20. What just happened?

The same disagreement Chapter 12 found comparing two epochs of one run, and comparing two entirely different training regimes, shows up a third time here, on a genuinely new scenario: real new data, really incorporated. `avg_overlap` says this update made things worse. Perplexity’s “improved” label points the right direction but by a small margin — 25.92 → 25.73 is about 0.7 %, nowhere near the substantial, 6× gap Chapter 11 found between a base and fine-tuned model. Chapter 12’s own Production Reality warned that this comparison has no noise threshold; this run is a live example of why that matters: a real disagreement (`avg_overlap`) sits next to a real but minor “improvement” (`perplexity`), and only one of them is worth acting on.

This step’s loader is worth noticing too: it calls `load_version` (Chapter 12), not Step 2’s `load_checkpoint` (Chapter 8) — the two look interchangeable but aren’t. `load_checkpoint` sets `is_trainable=True`, which leaves the model in **train mode**, dropout included. That’s correct for resuming training in Step 2; loading a checkpoint the same way just to *evaluate* it would leave dropout switched on during scoring, making perplexity noisy from run to run for no real reason. Catching this meant literally checking `model.training` after loading — it printed `True` where it should have printed `False`, on the first attempt at this exact step.

13.21. Practical exercise

Beginner

Try it yourself: run `python code/chapter_13/continuous_finetune.py` from inside `book/` (it takes roughly 20-25 minutes on CPU) and compare its printed output to the transcripts above.

You'll know it worked when: you see `avg_overlap`: regressed in the final comparison, and you can explain in one sentence why that means this particular update shouldn't be deployed without a closer look, even though training itself ran cleanly.

13.22. Field notes

 Field notes: the new batch skews toward a different phase of the well

Query: what the 17 new reports (#60 onward) actually describe, compared to what the held-out evaluation report (#37) describes.

Result: 20 of the new batch's 179 examples mention casing or cement — concentrated in the last several reports, right at the archive's end (*"Rig up and run 7" casing*, *"Cement 7" casing*, *"Production Casing Wait on Cement"*). The updated model's memorized-sounding template shifts to match: *"Production Casing Run Csg & Cement Rig up cementing equipment,"* replacing the current model's *"Production Drilling Cond Mud & Circ Circulate for temperature."* Report #37, the held-out report this chapter still evaluates against, is from the middle of the well program — active drilling, not casing.

Why: a real well program drills first and cases and cements later, near total depth. The new batch isn't random new data; it's disproportionately drawn from a later, different operational phase than the report this chapter evaluates against. Continuing training on it pulled the model's default template toward casing/cementing language, which fits the new reports and doesn't fit report #37.

Lesson: "the new data is real and relevant" and "the new data resembles what you're evaluating against" are different claims. A drop in `avg_overlap` here isn't a training bug — it's an honest reflection of what the new batch actually contains, and exactly the kind of thing a human should look at before deciding whether report #37-style questions are still important enough to weight for, or whether the archive's growing casing/cementing content means the evaluation set itself needs to grow too.

13.23. Challenge exercise

Intermediate

Challenge: this chapter's evaluation set is still Chapter 11's original 8 questions, all from report #37 — a report about active drilling, not casing. Build a second held-out check from one of the new batch's own reports (held out from the "new" training set the same way #37 is held out from everything) and compare the updated model against it instead. A reference solution is at `code/chapter_13/challenge/challenge.py` — on a real run, holding report #65 (2020-12-23, 20 real examples) out of the new-batch continuation and evaluating against it afterward gives `{'exact_match': 0, 'avg_overlap': 0.14, 'perplexity': 16.56}`. Perplexity is noticeably lower here than either model scored against report #37 (~25.8) — the updated model finds a report from its own new batch's operational phase less surprising than one from an earlier phase, even though `avg_overlap` stays just as low. Recency alone didn't fix what changed; matching the evaluation report's operational phase did more than matching its training era.

13.24. Key takeaways

- Training completing without errors, with loss falling every epoch, says nothing about whether the result should replace what’s in production. This chapter’s real run trained cleanly and still regressed on `avg_overlap`.
- The same metric-disagreement pattern Chapter 12 found comparing epochs, and comparing entirely different training regimes, showed up a third time in a genuinely new scenario — this isn’t a one-off quirk of one comparison.
- A regression is worth investigating, not just flagging: the real cause here was traceable to what the new reports actually contain, not a training bug.
- Perplexity moving in the “improved” direction by under 1% is not the same claim as the substantial, 6× gap Chapter 11 found between a base and fine-tuned model. Direction alone, without magnitude, can overstate a trivial change.
- A model left in train mode (dropout still active) scores noisily even in eval-only code — `load_checkpoint’s is_trainable=True` is right for resuming training and wrong for scoring; catching it meant checking `model.training` directly, not just distrusting a number that looked slightly different between runs.

13.25. Repository files

File	Purpose
<code>code/chapter_13/continuous_finetune.py</code>	Splits the archive, trains a current model, continues training on new reports, and compares the result
<code>code/chapter_13/challenge/challenge.py</code>	Reference solution: evaluating against a held-out report from the new batch itself
<code>notebooks/chapter_13_explore.ipynb</code>	Interactive companion notebook



CHECKPOINT — Chapter 13

- ☒ Simulated new reports arriving using a real chronological split of this book’s own archive
- ☒ Continued fine-tuning an existing checkpoint on just the new batch, reusing Chapter 8’s save-and-resume mechanism unchanged
- ☒ Ran Chapter 12’s comparison on the result and found a real regression, not a hypothetical one
- ☒ Traced that regression back to a real, sensible cause instead of treating the metric as unexplainable noise



WHAT YOU BUILT

`continuous_finetune.py` — splits this book’s real archive by a chronological cutoff, trains a “current” model on one side, continues training it on the “new” side using Chapter 8’s exact checkpoint mechanism, and runs Chapter 12’s comparison to decide whether the result actually deserves to replace what’s running. On a real run, it doesn’t.

13.26. What can you do now that you couldn't do before?

You can retrain a model on new data as it arrives, and — because you built Chapter 12's comparison first — you can tell the difference between “we retrained” and “we improved,” instead of deploying whatever a clean training run happens to produce.

13.27. See it side by side

Every real checkpoint sitting in `checkpoints/` right now — from Chapter 5's first fine-tune through this chapter's continuously-updated one — is something the book's companion app (`app/`) can load directly, not a separate demo built beside this book. Its Model Playground puts the base and fine-tuned answers side by side on the same real prompt; Before vs. After Evaluation runs this book's own metrics live across the full held-out set; Dataset Explorer browses the real training examples behind them; and Failure Analysis reruns the book's own documented failure cases — including a real regression like the one this chapter just found — against whichever checkpoint you actually have. See `app/README.md` to run it; nothing in it is pre-recorded, so your own checkpoint may not reproduce this book's exact numbers, and that difference is itself real information.

13.28. Suggested next step

This is the last chapter. Every tool in it runs on your own data, not just this book's Utah FORGE archive: Chapter 6's gate checks a new archive's quality, Chapter 8 fine-tunes it at whatever scale it has, Chapters 9 and 10 add retrieval and a faithfulness check on top, Chapter 11 builds a real evaluation set from whatever you hold out, and Chapters 12 and 13 decide, honestly, whether a new version is actually worth deploying. The book's own archive stops in January 2021. Yours doesn't have to.

13. Continuous Fine-Tuning: Keeping the Model Current

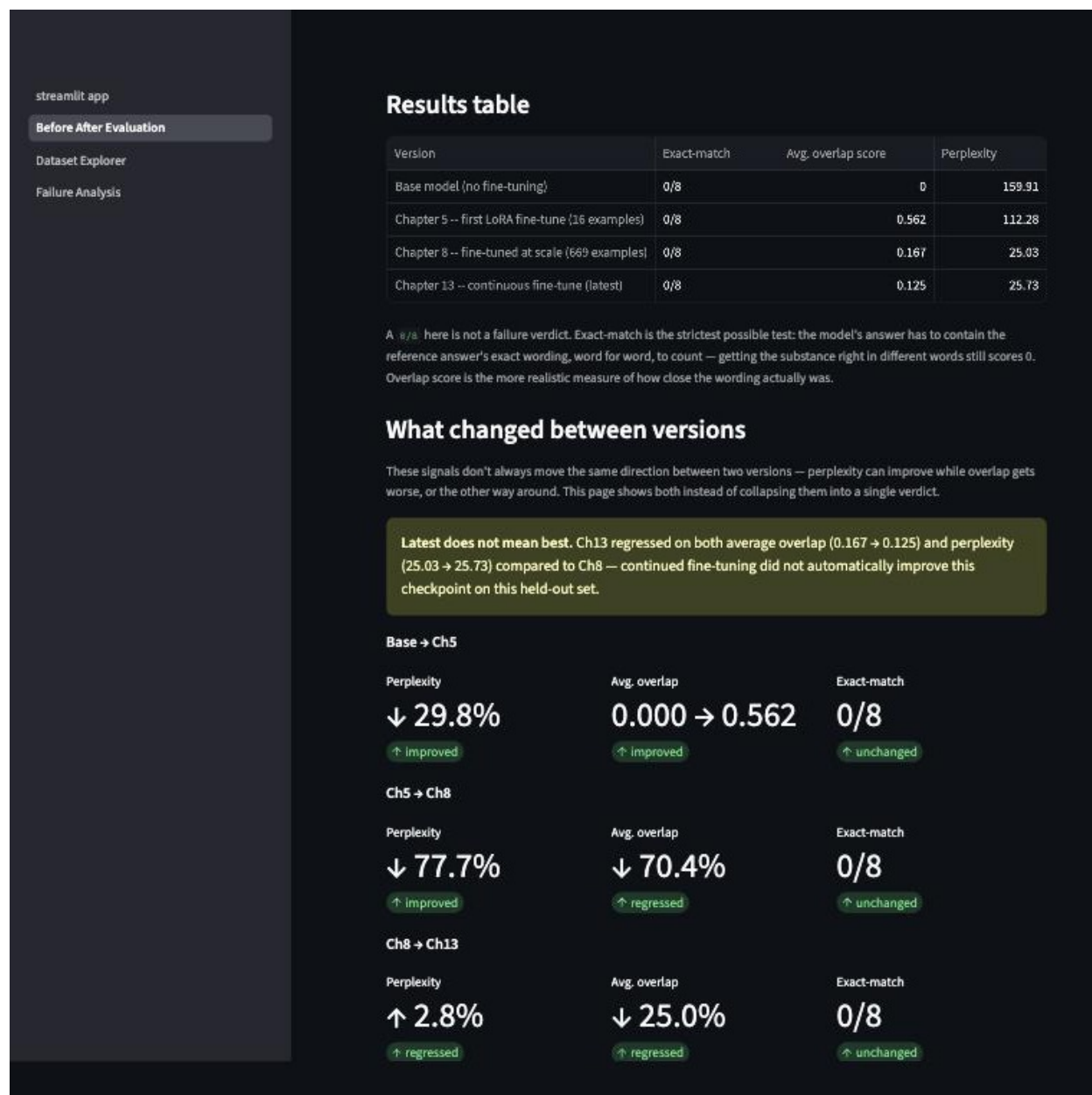


Figure 13.1.: Before vs. After Evaluation, screenshotted from a real local run across this book's own checkpoints – a similar “latest doesn't mean best” regression, found separately by comparing the book's own persisted Chapter 5/8/13 checkpoints (this one regresses on both average overlap and perplexity, not the single metric this chapter's own version-comparison run above flagged – a different checkpoint pair, not the same result), surfaced automatically by the app.

Part IV.

Appendices

Appendix A: Environment Setup

Part 0 covers everything you need to install Python, create a virtual environment, check your hardware, and run the book’s chapter scripts — start there if you haven’t already. This appendix covers a few things Part 0 doesn’t: the training dataset’s layout, rendering this book itself (not just running its code), a data-handling note for your own reports, and troubleshooting for a few less common errors.

If you’d like a guide specific to the editor or notebook environment you chose in Part 0, see Appendices A1–A5 instead.

1. The training dataset

Chapters 1–5 use ten real, curated Daily Drilling and completion reports from the public Utah FORGE archive (well FORGE 16A [78]-32) — already committed as PDFs in `datasets/sample_training_set/`, the same curated subset used in the author’s previous book. Part II’s “at scale” chapters use the full 76-report archive, committed at `datasets/full_training_set/`. Nothing needs to be generated or downloaded to start: cloning the repository (Part 0) gives you the full dataset. See `book/datasets/README.md` for the full policy and license note.

If you want to reproduce the curated subset from the full archive, or extend it with more reports, see `code/chapter_01/build_sample_training_set.py`.

2. Rendering the book with Quarto

Running the chapter scripts (Part 0) doesn’t require anything beyond Python. Rebuilding this book itself — the HTML and PDF you’re reading — is a separate step that needs [Quarto 1.7](#) or later:

```
quarto --version
```

If that’s not recognised, install Quarto from [quarto.org](#) for your operating system.

With Quarto installed and your virtual environment active:

```
quarto render
```

Output is written to `_book/`. For live-editing a single chapter while you write:

```
quarto preview chapters/chapter_01.qmd
```

Or, if you prefer conda/mamba instead of the `.venv` approach from Part 0:

```
conda env create -f environment.yml
conda activate llm-ft-drilling-completions-book
```

3. Hardware notes

Fine-tuning, even parameter-efficiently, is more hardware-sensitive than retrieval:

- **GPU (NVIDIA/CUDA or Apple Silicon/MPS):** the comfortable path from Chapter 5 onward, turning training runs that take minutes on a GPU into ones that take hours on CPU. `bitsandbytes` (used for 4-/8-bit QLoRA fine-tuning) installs on Linux regardless of GPU, but no chapter in this book actually loads a quantized model — every chapter fine-tunes with full-precision LoRA, the path Chapter 5 walks you through, so a GPU here just makes that same LoRA fine-tune faster, not a different technique.
- **CPU only:** still possible for the earliest chapters (loading a model, baseline prompting, tokenization) and for fine-tuning a very small model, but training runs will be noticeably slower. Expect to reduce dataset size and steps for CPU-only exercises.
- **No cloud account or paid API is required anywhere in this book.**

4. A note on data handling

This book's own data (`datasets/sample_training_set/`, `datasets/full_training_set/`) is public and safe to commit, share, and publish — that's true throughout this repository. The moment you point this book's code at your own organisation's reports instead, that changes:

- Never commit real, confidential reports, extracted text, training examples, or model checkpoints trained on them to a public repository.
- If you're experimenting on a shared machine, confirm your organisation's data classification policy covers running third-party Python packages (model downloads, tokenizers) against confidential well data.
- Add your own archive's path to `.gitignore` before dropping any confidential reports into a local `datasets/` folder for testing.
- Before fine-tuning on your own organisation's data, confirm you have the right to use it for this purpose — that determination is outside the scope of this book.

5. Troubleshooting

[Part 0, Section 0.11](#) covers the common setup errors (Python not found, virtual environment not active, and so on). A few more specific to rendering the book or running heavier fine-tuning packages:

Symptom	Likely cause
<code>quarto render</code> fails on a code cell	Activate <code>.venv</code> before rendering — Quarto uses whichever Python is on <code>PATH</code> . Confirm with <code>which python</code> and <code>python -c "import transformers"</code> .
<code>ModuleNotFoundError: peft</code> or <code>trl</code> (Chapter 5+)	Re-run <code>pip install -r requirements.txt</code> ; these are heavier dependencies that pull in <code>torch</code> and sometimes need a platform-specific <code>torch</code> install first.
<code>bitsandbytes</code> fails to import	It's Linux + NVIDIA GPU only (see <code>requirements.txt</code>). On macOS or CPU-only Linux, skip 4-/8-bit quantized fine-tuning and use full-precision LoRA instead.
Training is extremely slow	Confirm a GPU is actually being used (<code>torch.cuda.is_available()</code> or <code>torch.backends.mps.is_available()</code>); on CPU-only hardware, reduce dataset size and steps as suggested in the relevant chapter.

Appendix A1: Using Jupyter Notebook

This appendix assumes you’ve completed **Part 0** — Python installed, `.venv` created and active, packages installed via `pip install -r requirements.txt`. Everything below is specific to running this book’s code inside Jupyter Notebook instead of from a plain terminal.

Install the notebook

`requirements.txt` already installed Jupyter for you in Part 0, so you can likely skip straight to “Start the notebook” below. If you skipped that step, or only want Jupyter on its own:

```
pip install notebook
```

Expected output: a `Successfully installed ...` line mentioning `notebook`.

Start the notebook

With your virtual environment active (you should see `(.venv)` in your terminal prompt) and from inside the `llm-ft-drilling-completions-book` project folder:

```
jupyter notebook
```

What this does: starts a local notebook server and opens a new browser tab showing the contents of your project folder.

If nothing opens automatically, the terminal will print a URL starting with `http://localhost:8888/...` — copy and paste that into your browser.

Open a chapter notebook

Every chapter has an interactive companion notebook. In the browser tab Jupyter opened, click into the `notebooks/` folder, then open `chapter_01_explore.ipynb` (or whichever chapter you’re on).

Run a cell

A notebook is made of cells — small blocks of code you run one at a time. Click into a cell, then press **Shift+Enter** to run it and move to the next one. The output (printed text, or an error) appears directly below the cell.

Expected output: for Chapter 1’s notebook, running the model-loading cell prints a short completion generated by the local base model.

Stop the notebook server

When you’re done, go back to the terminal window where you ran `jupyter notebook` and press **Ctrl+C**, then confirm if asked. Closing the browser tab alone does not stop the server — the terminal is where it’s actually running.

Appendix A2: Using VS Code

This appendix assumes you've completed **Part 0** — Python installed, `.venv` created and active, packages installed via `pip install -r requirements.txt`. Everything below is specific to working with this book's code in Visual Studio Code.

Install VS Code and the Python extension

Download VS Code from code.visualstudio.com, then open the Extensions panel (`Ctrl+Shift+X` / `Cmd+Shift+X`) and install **Python** (by Microsoft).

Open the project folder

File > Open Folder... and select the `llm-ft-drilling-completions-book` folder you cloned in Part 0.

Select the virtual environment as your interpreter

Press `Ctrl+Shift+P` / `Cmd+Shift+P`, type **Python: Select Interpreter**, and choose the `.venv` you created in Part 0 (it will be listed as a recommended interpreter for this folder).

Expected result: the bottom status bar shows `.venv` (or `Python 3.11.x ('.venv')`) instead of a system Python.

Run a chapter script

Open `book/code/chapter_01/load_local_model.py` and press the **Run** button in the top-right corner, or right-click inside the editor and choose **Run Python File in Terminal**. Output appears in VS Code's integrated terminal.

Run the tests

Open the Testing panel (flask icon in the sidebar) and click **Configure Python Tests**, choosing `pyp` and the `book` folder. Tests then appear as a tree you can run individually or all at once.

Troubleshooting

Symptom	Likely cause
VS Code doesn't offer <code>.venv</code> as an interpreter	Make sure you opened the repository root (or <code>book/</code>) as the workspace folder, not a parent folder.
<code>ModuleNotFoundError</code> when running a script	Confirm the selected interpreter (bottom status bar) is <code>.venv</code> , not a system Python.

Appendix A3: Using PyCharm Community

This appendix assumes you've completed **Part 0** — Python installed, `.venv` created and active, packages installed via `pip install -r requirements.txt`. Everything below is specific to working with this book's code in PyCharm Community Edition (free).

Install PyCharm Community

Download it from jetbrains.com/pycharm/download — choose the **Community** edition, not Professional.

Open the project folder

File > Open... and select the `llm-ft-drilling-completions-book` folder you cloned in Part 0.

Point PyCharm at your virtual environment

File > Settings (Windows/Linux) or PyCharm > Settings (macOS) > **Project: llm-ft-drilling-completions-book** > **Python Interpreter** > gear icon > **Add** > **Existing environment**, and browse to the `.venv` folder created in Part 0 (`.venv/bin/python` on macOS/Linux, `.venv\Scripts\python.exe` on Windows).

Expected result: the interpreter dropdown at the bottom of the Settings page shows your `.venv` path.

Run a chapter script

Right-click `book/code/chapter_01/load_local_model.py` in the Project panel and choose **Run** 'load_local_model'. Output appears in the Run panel at the bottom.

Run the tests

Right-click the `book/tests` folder and choose **Run** 'pytest in tests'. PyCharm shows a tree of passing/failing tests.

Troubleshooting

Symptom	Likely cause
Interpreter list doesn't show <code>.venv</code>	Confirm Part 0's virtual environment was actually created inside the project folder, not elsewhere.
Red underlines on valid imports	PyCharm hasn't finished indexing, or the wrong interpreter is selected — check the bottom-right corner of the window.

Appendix A4: Using Positron

This appendix assumes you've completed **Part 0** — Python installed, `.venv` created and active, packages installed via `pip install -r requirements.txt`. Everything below is specific to working with this book's code in [Positron](#), Posit's data-science-focused IDE.

Install Positron

Download it from positron.posit.co for your operating system.

Open the project folder

File > Open Folder... and select the `book/` folder inside your cloned `llm-ft-drilling-completions-book` repository (this matches the author's own workflow described in `book/README.md`).

Select the Python interpreter

Click the interpreter selector in the top-right corner (or bottom status bar) and choose the `.venv` created in Part 0.

Expected result: the console pane shows a Python prompt using that `.venv`'s packages.

Run a chapter script

Open `code/chapter_01/load_local_model.py` and click **Run File**, or select code and press `Ctrl+Enter` / `Cmd+Enter` to run it line by line in the console — useful for inspecting a model's output interactively.

Preview a chapter while writing

From the integrated terminal:

```
quarto preview chapters/chapter_01.qmd
```

This live-reloads a single chapter in a browser tab as you edit it — the author's own drafting workflow.

Troubleshooting

Symptom	Likely cause
Interpreter selector doesn't list <code>.venv</code>	Confirm you opened <code>book/</code> (not the repository root) as the workspace, and that <code>.venv</code> was created inside it.
<code>quarto preview</code> command not found	Quarto isn't installed or isn't on <code>PATH</code> — see Appendix A, Section 2.

Appendix A5: Terminal Only

This appendix assumes you've completed **Part 0** — Python installed, `.venv` created and active, packages installed via `pip install -r requirements.txt`. No editor or notebook tool is required at all; everything in this book can be followed with a terminal and a plain text editor.

Activate your environment

Every time you open a new terminal window to work on this book:

```
cd llm-ft-drilling-completions-book/book
source .venv/bin/activate          # Windows: .venv\Scripts\activate
```

Expected result: your prompt now starts with `(.venv)`.

Edit a file

Any plain text editor works — `nano`, `vim`, `notepad`, or a GUI editor that isn't a full IDE. For example:

```
nano code/chapter_01/load_local_model.py
```

Run a chapter script

```
python code/chapter_01/load_local_model.py
```

Output prints directly to the terminal.

Run the tests

```
pytest -v
```

Render the book (optional)

Rendering the book itself (not just running its code) needs Quarto — see Appendix A, Section 2:

```
quarto render
```

Troubleshooting

Symptom	Likely cause
command not found: python	Try <code>python3</code> instead — some systems only alias <code>python</code> after activating a virtual environment.
Script runs but prints nothing	Confirm you're running it from inside <code>book/</code> , not the repository root — chapter scripts assume that working directory for relative paths.

Appendix B: Drilling, Completions & Fine-Tuning Glossary

This glossary supports Chapter 2’s training-example builder and every later chapter’s “Engineering Translation” callouts. It is split into two parts: common oilfield shorthand you’ll see in raw report text, and the fine-tuning/ML vocabulary this book introduces gradually, gathered here for quick reference. Neither list is exhaustive — abbreviation usage varies by operator, rig contractor, and basin, and the ML terminology here is deliberately scoped to what this book actually uses.

B.1 Common oilfield abbreviations

Abbreviation	Expansion
AFE	authorization for expenditure
BHA	bottom hole assembly
BOP	blowout preventer
CBL	cement bond log
CIRC	circulate
COND	condition
CSG	casing
DFIT	diagnostic fracture injection test
DFS	days from spud
DHSV	downhole safety valve
DLS	dogleg severity
DOL	days on location
DP	drill pipe
ECD	equivalent circulating density
EMW	equivalent mud weight
FIT	formation integrity test
FR	from
GPM	gallons per minute
HWDP	heavy weight drill pipe
KOP	kick-off point
LOT	leak-off test
MD	measured depth
MIRU	move in, rig up
MW	mud weight
MWD	measurement while drilling
NMDC	non-magnetic drill collar
NPT	non-productive time

Abbreviation	Expansion
OBSD	observed
PBTD	plugged back total depth
PDC	polycrystalline diamond compact (bit type)
PJSM	pre-job safety meeting
POOH	pull out of hole
PPG	pounds per gallon
PU	pick up (pipe)
PUW	pick up weight
RIH	run in hole
RKB	rotary kelly bushing
ROP	rate of penetration
RPM	revolutions per minute
SICP	shut-in casing pressure
SIDPP	shut-in drill pipe pressure
SPP	stand pipe pressure
STDS	stands (of drill pipe)
TD	total depth
TIH	trip in hole
TOOH	trip out of hole
TVD	true vertical depth
WOB	weight on bit
WOC	wait on cement
WOW	wait on weather
XO	crossover (sub)

 Abbreviations are not universal

The same three letters can mean different things across operators and basins — build your abbreviation dictionary from *your* archive’s actual usage before trusting it against your own reports.

B.2 Fine-tuning and local LLM vocabulary

Term	Plain-language meaning
Base model	The general-purpose model before any domain fine-tuning — Chapter 1’s starting point.
Fine-tuning	Updating a model’s weights on your own data so it performs better on your specific task or domain.
LoRA (Low-Rank Adaptation)	A parameter-efficient fine-tuning method that trains a small add-on to the model instead of every weight — Chapter 5’s technique.

Term	Plain-language meaning
QLoRA	LoRA fine-tuning on a quantized (compressed) copy of the base model, cutting memory needs further.
Quantization	Storing a model’s weights at lower numerical precision to shrink memory and speed up inference, at some accuracy cost.
Checkpoint	A saved snapshot of a model’s weights at a point during training — Chapter 8.
Tokenizer	The component that splits text into the sub-word units (“tokens”) a model actually processes — Chapter 4.
Instruction/response pair	A single training example: an instruction (and optional input) paired with the desired output — Chapter 2’s core data format.
Epoch	One full pass through the training dataset.
Perplexity	A numeric measure of how well a model predicts held-out text; lower is generally better — Chapter 11.
Hallucination	A fluent but factually unsupported model output — Chapter 10 covers detecting and mitigating it.
Drift	A model’s behavior changing across versions or over time. A regression is drift for the worse – something that used to work stops working in a version meant to be an improvement — Chapter 12 checks this by comparing checkpoints on a fixed evaluation set, not by monitoring a deployed model’s live input traffic (a different, unimplemented problem also called drift).
RAG (retrieval-augmented generation)	Combining a model with a retrieval step over external documents, instead of relying only on what’s in its weights — Chapter 9.
Grounded / grounding	An answer that’s actually built from a real, cited source, not just retrieved alongside it — Chapter 10 checks the difference.
Faithfulness	How well an answer’s actual content matches what a specific source says — Chapter 10’s word-overlap check is one simple way to measure it.
Evaluation set	A fixed, reusable list of questions with known-correct answers, held back from training — Chapter 11 builds a real one from the report Chapter 2 reserved.

Term	Plain-language meaning
Continuous fine-tuning	Updating a model again as new data becomes available, instead of training it once and leaving it fixed — Chapter 13. Answers <i>how</i> to incorporate new data, not <i>whether</i> the result should replace what’s currently deployed; that’s still Chapter 12’s comparison, run again.

Appendix C: Known Limitations

This book’s one non-negotiable rule is that a stated result comes from a real run of this book’s own code, not an estimate. The same standard applies to limitations: everything below is something you can see for yourself by reading the referenced script or rerunning the referenced chapter, not a hypothetical concern.

Four of these are already called out where they happen — in Chapter 2’s, Chapter 4’s, Chapter 6’s, Chapter 7’s, Chapter 8’s, and Chapter 10’s own Production Reality or Field Notes sections — because that’s where a reader hits them first. They’re repeated here, gathered with two that aren’t discussed elsewhere in the book, so you have one place to check before pointing this pipeline at your own archive.

None of these are treated as bugs to quietly fix and forget. There is no single report format across operators, contractors, and basins, so no fixed set of extraction patterns, thresholds, or one small model could honestly claim to be a finished, one-size-fits-all solution. This book’s actual goal is to give you the working knowledge to build and adapt your own pipeline — this list is the specific starting point for that adaptation work, not a warning to wait for a future version that removes it.

C.1 Report parsing breaks on a new report layout (Chapters 2, 6, and 7)

Chapter 2’s and Chapter 7’s extraction scripts both work the same way a spreadsheet macro that looks for a value two cells to the right of the label “WELL NAME:” works: reliable as long as every report is laid out exactly the same way, and silently wrong or empty the moment it isn’t. Chapter 6’s data-quality gate doesn’t add a new regex of its own, but it imports and runs Chapter 2’s `extract_fields/extract_text` unchanged, so this same fixed-layout assumption is what it’s actually checking against the whole 76-report archive.

`code/chapter_02/build_training_examples.py` looks for labels like `WELL NAME:`, `PRESENT OPERATIONS:`, and `ACTIVITY PLANNED:` at fixed positions in the report text using Python’s `re.compile(r"...")` pattern-matching syntax — its own docstring says so directly: “*Only handles the ‘DAILY DRILLING REPORT’ field layout.*” A completion report, which uses different field labels entirely, isn’t handled by the main chapter at all — it’s left to the challenge exercise. `code/chapter_07/format_training_chunks.py` has the same shape: its timeline extraction looks for a literal `TIME BREAKDOWN` header and a fixed column order. Both chapters’ own Production Reality sections already say this plainly: Chapter 2’s calls it “a specific, fixed field layout — the one every `Drilling_NNN` report in this archive happens to share,” and Chapter 7’s calls it “the same kind of assumption that breaks the moment a different operator’s report template shows up.”

Chapter 6 puts a real number on how well that assumption held, at full archive scale, on this one archive: 75/76 reports extracted cleanly, with the single failure being the same completion-report

layout mismatch Chapter 2 already knew about. Chapter 6's own Production Reality section is explicit that this number is a property of this specific archive, not a guarantee: *"This archive comes from a single well, a single operator's report template, over a few months. A multi-well, multi-operator archive would show a much lower pass rate here... A production quality gate would need a library of layout patterns, not one regex per field."*

! This is where a coding background starts to matter

This book is written for a reader with no prior programming experience, and every chapter up to this adaptation step is written to hold that promise. This step doesn't. Editing a `re.compile(r"...")` pattern to match a different TIME BREAKDOWN or OPERATIONS table layout — figuring out which characters in the pattern are literal text and which are wildcards, then testing it against real report text until it matches correctly — is a genuine programming task, not a find-and-replace. If your company's daily drilling reports format these sections differently from Utah FORGE's, expect this to be the point where you need either to build real comfort with Python's `re` module first, or bring in someone who already has it, before Chapter 2's or Chapter 7's scripts will extract anything useful from your own archive.

What this means for you: if your own archive spans multiple operators, contractors, or hand-written morning reports — each with its own layout — expect to write a new set of label patterns per format before Chapter 2's, Chapter 6's, or Chapter 7's scripts will extract anything useful. None of the three attempts to detect or adapt to a layout it wasn't written for.

C.2 The faithfulness check compares words, not meaning (Chapter 10)

Chapter 10's `faithfulness_score` checks whether an answer's words show up somewhere in the source text it's supposed to be grounded in — like a word search, not a careful read of the sentence. It counts matching vocabulary; it has no notion of which word is doing the work in a sentence, so two short, common, opposite words can both "match" equally well.

Chapter 10's own Field Notes catch this in a real run: report #49's grounded answer read *"Trip out of hole with fishing bha"* and scored 0.80 faithfulness — well above the chapter's 0.5 threshold — against a source that actually says *"Pick up Fishing BHA # 33 and trip in hole to 7,996"*. The direction is reversed and the specific depth is dropped, and the score doesn't notice, because `"trip"`, `"fishing"`, and `"bha"` all matched; only `"in"` versus `"out"` — the one word that actually changes the meaning — flipped.

What this means for you: a high faithfulness score tells you an answer's vocabulary came from the right report. It does not tell you the answer's claims are correct. Any check built on this method will have the same blind spot for a reversed direction, a flipped up/down or in/out call, or a swapped number, whenever the rest of the sentence's wording stays similar — exactly the class of error most likely to matter on a real well.

! An educational baseline, not a safety check

This is the limitation most worth taking seriously if you're considering this pipeline for anything beyond learning. `faithfulness_score` is a teaching tool for understanding what "grounded"

does and doesn't guarantee — it has not been validated as a safety system, and field engineers who lean on automated summaries for safety-critical shift handovers should never treat a passing score here as a substitute for a qualified person reading the actual source. Report # 49's reversed "trip in/out of hole" case above scored 0.80, well past this chapter's own 0.5 threshold — proof this check can pass exactly the kind of error that matters most on a live well.

C.3 CPU training times don't demonstrate production scale (Chapters 8 and 13)

Every training run in this book is deliberately sized to finish on a standard laptop with no dedicated GPU: Chapter 8's full multi-epoch run over 669 examples takes about 30-35 minutes on CPU, and Chapter 13's small continuous update — 2 epochs over 179 new examples — takes about 20-25 minutes on CPU. Both are demo-scale by design, not a benchmark of what a production archive would take.

What this means for you: this book never measures, and doesn't claim to know, how long a multi-epoch run over a full company archive of thousands of reports would take on CPU alone, or how often a Chapter-13-style continuous update could realistically run against new reports on a standard corporate laptop. A dedicated GPU — mentioned in Chapter 5 as an optional speedup (`model.to("cuda")`), never benchmarked anywhere in this book — is the likely answer at real scale, but that combination isn't something this book's own code has actually run and measured.

C.4 Resuming a checkpoint forgets the optimizer's memory (Chapters 8 and 13)

Training uses an optimizer (Adam) that keeps a running memory of how each weight has been moving recently — similar to a mechanic remembering roughly how much force a bolt needed last time, so the next turn on it is smoother than starting from scratch. Chapter 8's `save_checkpoint/load_checkpoint` functions save and reload the model's trained weights, but not that running memory — it exists only in the computer's memory while training runs, and is gone the moment training stops, whether that's a real crash or a deliberate pause. The chapter's own Production Reality section says so directly: *"this simple version doesn't persist Adam's momentum across a resume, only the model weights."*

Chapter 13's continuous fine-tuning reuses these same two functions unchanged, so every scheduled update carries the same gap.

What this means for you: expect a small amount of extra training noise — a temporary bump right after training resumes, before it settles back into a smooth improvement — every time a paused or crashed run is picked back up, and every time Chapter 13's continuous update runs against a fresh checkpoint. It's a real, measured cost of this version's simplicity, not a bug; a production setup would checkpoint and restore that optimizer memory too.

C.5 Everything in this book is tested on one small model (Chapters 1-13)

Every chapter from Chapter 1 onward loads the same base model, `Qwen2.5-1.5B-Instruct`, chosen specifically because it fine-tunes on CPU in the timeframes above — a deliberate choice matched to the hardware check in Part 0, not a claim that it’s the best model for this job. `requirements.txt` installs `bitsandbytes`, the package needed for QLoRA (fine-tuning a compressed, lower-memory copy of a larger model — see Appendix B’s glossary), but no chapter script in this book actually loads a quantized model.

What this means for you: whether a larger model — 7B or 14B parameters, fine-tuned with QLoRA instead of this book’s plain LoRA — would learn this same operational vocabulary better, more slowly, or with different failure patterns on the same report archive is a real, open question this book doesn’t answer. If your own use case can afford the extra memory and slower iteration a bigger model requires, that comparison is worth running yourself; this book’s code is not set up to run it for you.

C.6 The tokenizer’s fixed vocabulary splits oilfield shorthand apart (Chapter 4)

A model doesn’t read whole words — it breaks text into pieces from a fixed list decided when the base model was built, the same way a pre-printed form only has boxes for words someone anticipated. Chapter 4 shows this splitting ‘BHA’ into two meaningless pieces, [‘B’, ‘HA’], with no more connection to “bottom hole assembly” than those two letters carry on their own — a real result from that chapter’s own tokenizer script.

Chapter 4’s own Production Reality section already names the fix this book doesn’t take: *“a production system dealing with truly nonstandard shorthand... sometimes trains or extends a custom tokenizer instead of just fine-tuning on top of a generic one; that’s a heavier, rarer move this book doesn’t cover.”* Chapter 5’s fine-tuning never touches the tokenizer itself, only the model’s weights, so this split is present in every chapter that follows.

What this means for you: the fine-tuned model in this book still reads your operator’s shorthand as arbitrary letter fragments, not as a term it has a dedicated slot for — Chapter 4’s own embedding check shows it can still learn a useful association between ‘BHA’ and “bottom hole assembly” without a dedicated token, but that’s a different, weaker fix than giving the term its own vocabulary entry. If your archive leans on internal codes or shorthand severe enough that this isn’t good enough, adding new tokens and resizing the model’s embedding table to match is the heavier step this book doesn’t walk through.

References

- Dettmers, Tim, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. *QLoRA: Efficient Finetuning of Quantized LLMs*. <https://arxiv.org/abs/2305.14314>.
- Hu, Edward J., Yelong Shen, Phillip Wallis, et al. 2021. *LoRA: Low-Rank Adaptation of Large Language Models*. <https://arxiv.org/abs/2106.09685>.
- Ji, Ziwei, Nayeon Lee, Rita Frieske, et al. 2023. “Survey of Hallucination in Natural Language Generation.” In *ACM Computing Surveys*, No. 12, vol. 55. <https://doi.org/10.1145/3571730>.
- Matthes, Eric. 2023. *Python Crash Course: A Hands-on, Project-Based Introduction to Programming*. 3rd ed. No Starch Press.
- Wei, Jason, Maarten Bosma, Vincent Y. Zhao, et al. 2021. *Finetuned Language Models Are Zero-Shot Learners*. <https://arxiv.org/abs/2109.01652>.

